Appendix C

AAPL Agent Walkthrough

notebooks/aapl_agent_walkthrough.ipynb

AAPL Agent Walkthrough

This notebook runs the delivered system end to end on one ticker. The deterministic spine computes the corpus funnel, company resolution, weekly features, statistics, and the baseline arms. Eight LLM-based agent roles and a deterministic Market Lane supply the judgment steps. Six decision sessions run on the highest-coverage demonstration grid, and the memory ledger carries each realized outcome into the next session. The final cells print the AAPL research card and four verification exhibits. exploratory_data_analysis.ipynb profiles the sources and Team-10-Investment-Research-Agent.ipynb proposed the roles. The report describes the full architecture.

0. Config

Sets the ticker, the session count, the demonstration grid, and the random seed. The baseline arms are fixed before the agent sessions run.

In [1]:
```
%matplotlib inline
from collections import Counter, defaultdict
from pathlib import Path
import hashlib
import json
import math
import os
import re
import time
import warnings

warnings.filterwarnings("ignore", message=".*penalty.*deprecated.*")
warnings.filterwarnings("ignore", message=".*Inconsistent values.*")
warnings.filterwarnings("ignore", message=".*invalid value encountered.*")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import requests
import spacy
import yfinance as yf
from datasets import load_dataset
from dotenv import load_dotenv
from scipy.sparse import coo_matrix
from sklearn.cluster import AgglomerativeClustering
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

TICKER = "AAPL"
N_SESSIONS = 6
DEMO_GRID = "active"
RANDOM_STATE = 42
FAMILY_COLORS = {"c": "#4f81bd", "e": "#d28b36", "m": "#8c8c8c"}
NEG = "#c0504d"

REPO_ROOT = Path.cwd().parent
assert (REPO_ROOT / "pyproject.toml").exists(), "run this notebook from notebooks/"
DATA_PROCESSED = REPO_ROOT / "data" / "processed"
LLM_CACHE = REPO_ROOT / "data" / "llm_cache"
DATA_PROCESSED.mkdir(parents=True, exist_ok=True)
LLM_CACHE.mkdir(parents=True, exist_ok=True)
pd.set_option("display.max_colwidth", 90)
pd.set_option("display.width", 170)


def banner(title):
    print(f"\n{'=' * 78}\n{title}\n{'=' * 78}")


def mcnemar_p(a, b):
    n01 = int((~a & b).sum())
    n10 = int((a & ~b).sum())
    n = n01 + n10
```

```
    if n == 0:
        return 1.0
    k = min(n01, n10)
    p = sum(math.comb(n, i) for i in range(k + 1)) / 2 ** n
    return min(1.0, 2 * p)


banner(f"L0 CONFIG  ticker={TICKER}  weekly bins, next-week direction label  the L7 arms are pre-registered before the
agent sessions  demonstration grid={DEMO_GRID} (the {N_SESSIONS} test weeks with the most company coverage)")
```

```
================================================================================
L0 CONFIG  ticker=AAPL  weekly bins, next-week direction label  the L7 arms are pre-registered before the agent sessions  demo
nstration grid=active (the 6 test weeks with the most company coverage)
================================================================================
```

1. Corpus funnel

Splits each daily news block into one row per headline and bins rows by Friday-anchored week. A trigram recurring in 150 or more headlines is flagged as wire boilerplate. Identical normalized text within a week is deduplicated. A funnel chart shows the counts at each stage.

In [2]:
```
banner("L1.1 INGEST · spine step 1/7 — one decade of financial headlines, one row per headline (prompt-chain stage 1/5)")
nifty = load_dataset("raeidsaqur/NIFTY", trust_remote_code=True)
rows = []
for split in ["train", "valid", "test"]:
    fr = nifty[split].to_pandas()[["date", "news"]]
    for d, news in zip(pd.to_datetime(fr["date"]), fr["news"]):
        for h in str(news).split("\n"):
            if h.strip():
                rows.append((d, split, h.strip()))
heads = pd.DataFrame(rows, columns=["date", "split", "headline"])
heads["week"] = heads["date"].dt.to_period("W-FRI")
print(f"{len(heads):,} headlines | {heads['week'].nunique()} weekly bins | {heads['date'].min().date()} to
{heads['date'].max().date()}")
heads[["date", "headline"]].head(3)
```

```
================================================================================
L1.1 INGEST · spine step 1/7 — one decade of financial headlines, one row per headline (prompt-chain stage 1/5)
================================================================================
436,564 headlines | 560 weekly bins | 2010-01-06 to 2020-09-21
```

Out[2]:

	date	headline
0	2010-01-06	China Officials Likely Knew of Bad Milk
1	2010-01-06	Sony's CEO on Strategy in 3-D Technology
2	2010-01-06	Copper Settles at 16-Month High

In [3]:
```
banner("L1.2 WIRE BOILERPLATE · spine step 1/7 — which headlines are publisher formatting rather than news? (prompt-chain
stage 2/5: preprocess)")
TOKEN = re.compile(r"[a-z]{2,}")
heads["tokens"] = heads["headline"].str.lower().map(lambda s: TOKEN.findall(s))
tpl = CountVectorizer(analyzer=lambda toks: [" ".join(toks[i:i + 3]) for i in range(len(toks) - 2)],
                      ngram_range=(1, 1), min_df=150, max_features=30000).fit(heads["tokens"])
# a trigram recurring in 150+ headlines is publisher formatting, not news
tpl_grams = set(tpl.get_feature_names_out())
heads["is_template"] = heads["tokens"].map(
    lambda t: any(" ".join(t[i:i + 3]) in tpl_grams for i in range(len(t) - 2)))
tpl_counts = Counter()
for toks, flag in zip(heads["tokens"], heads["is_template"]):
    if flag:
        for i in range(len(toks) - 2):
            g = " ".join(toks[i:i + 3])
            if g in tpl_grams:
                tpl_counts[g] += 1
print(f"{len(tpl_grams):,} boilerplate trigrams | {heads['is_template'].mean():.1%} of headlines flagged")
top_tpl = tpl_counts.most_common(8)
display(pd.DataFrame({"boilerplate trigram": [g for g, _ in top_tpl],
                      "headlines flagged via": [c for _, c in top_tpl]}))

banner("L1.3 DEDUP — identical normalized text within a week is one story, not two")
heads["norm"] = heads["tokens"].map(" ".join)
clean = heads[~heads["is_template"]].drop_duplicates(subset=["week", "norm"]).reset_index(drop=True)
print(f"{len(clean):,} unique headlines remain ({len(clean) / len(heads):.1%} of raw)")

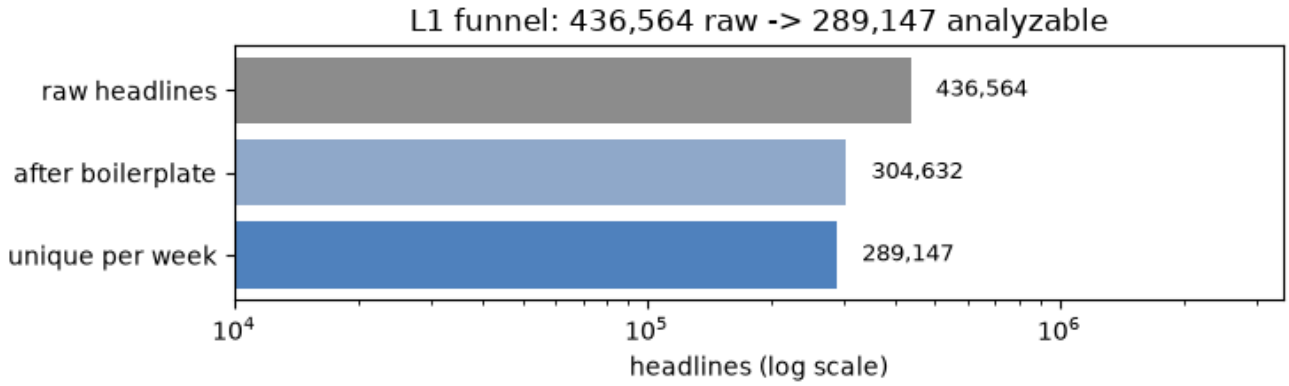
fig, ax = plt.subplots(figsize=(7.5, 2.4))
stages = ["raw headlines", "after boilerplate", "unique per week"]
counts = [len(heads), int((~heads["is_template"]).sum()), len(clean)]
bars = ax.barh(stages[::-1], counts[::-1], color=["#4f81bd", "#8fa8c9", "#8c8c8c"])
ax.set_xscale("log")
```

```
for b, c in zip(bars, counts[::-1]):
    ax.text(c * 1.15, b.get_y() + b.get_height() / 2, f"{c:,}", va="center", fontsize=9)
ax.set_xlim(1e4, len(heads) * 8)
ax.set_xlabel("headlines (log scale)")
ax.set_title(f"L1 funnel: {len(heads):,} raw -> {len(clean):,} analyzable")
plt.tight_layout()
plt.show()
```

```
================================================================================
L1.2 WIRE BOILERPLATE · spine step 1/7 — which headlines are publisher formatting rather than news? (prompt-chain stage 2/5: p
reprocess)
================================================================================
917 boilerplate trigrams | 30.2% of headlines flagged
```

	boilerplate trigram	headlines flagged via
0	net asset value	40426
1	research and markets	21077
2	reg db trackers	18109
3	etf net asset	17274
4	db trackers msci	6278
5	index etf net	5577
6	reg cs etf	4833
7	cs etf ie	4725

```
================================================================================
L1.3 DEDUP — identical normalized text within a week is one story, not two
================================================================================
289,147 unique headlines remain (66.2% of raw)
```



2. Vocabulary and the shared industry family

One spaCy pass over every cleaned headline records lemmas, two-word noun phrases, and named entities with their weeks. Filters then remove repeated wire-service phrases, constant-rate words, and market-process vocabulary, using a vector margin against thing-seed and market-seed lists. The surviving terms are clustered into themes, and each theme contributes one weekly share column.

In [4]:

```
banner("L2.1 VOCABULARY · spine step 2/7 — one pass reads every headline and records its words, lemma-normalized, plus
named entities (prompt-chain stage 4/5: extract)")
nlp = spacy.load("en_core_web_sm", disable=["parser"])
MONTHS = {"january", "february", "march", "april", "may", "june", "july", "august", "september",
          "october", "november", "december", "jan", "feb", "mar", "apr", "jun", "jul", "aug",
          "sep", "oct", "nov", "dec"}
STOP_EXTRA = MONTHS | {
    "plc", "inc", "ltd", "corp", "nv", "sa", "co", "company", "companies", "group",
    "holding", "holdings", "trust", "fund", "funds", "update", "alert", "research",
    "mln", "pct", "bn", "tn", "vs", "qtr", "nr", "gbx", "usd", "gbp", "cny", "yen",
    "bln", "mth", "wk", "yr", "table", "text", "reuters", "afp", "nyse", "nasdaq",
}
SPACY_STOP = nlp.Defaults.stop_words
ENTITY_LABELS = {"ORG", "GPE", "PRODUCT", "NORP", "LAW", "EVENT", "FAC"}

concept_sets, entity_sets, week_of = [], [], []
cw, ew = defaultdict(list), defaultdict(list)
surf = defaultdict(Counter)
occ_total = Counter()
```

```
edge_occ = defaultdict(Counter)
global_edge = Counter()

for start in range(0, len(clean), 40000):
    batch = clean.iloc[start:start + 40000]
    for (_, row), docy in zip(batch.iterrows(), nlp.pipe(batch["headline"], batch_size=1000)):
        toks = [t for t in docy if not t.is_space]
        ntok = len(toks)
        ws = [t.lower_ for t in toks]
        global_edge[("F", " ".join(ws[:2]))] += 1
        global_edge[("F", " ".join(ws[:3]))] += 1
        global_edge[("L", " ".join(ws[-2:]))] += 1
        global_edge[("L", " ".join(ws[-3:]))] += 1

        def rec(store, term, surface, i):
            store[term].append(row["week"])
            surf[term][surface] += 1
            occ_total[term] += 1
            if i < 2:
                edge_occ[term][("F", " ".join(ws[:2]))] += 1
                edge_occ[term][("F", " ".join(ws[:3]))] += 1
            elif i >= ntok - 2:
                edge_occ[term][("L", " ".join(ws[-2:]))] += 1
                edge_occ[term][("L", " ".join(ws[-3:]))] += 1

        cset, eset = set(), set()
        for i, t in enumerate(toks):
            lem = t.lemma_.lower()
            if t.pos_ == "NOUN" and t.is_alpha and len(lem) >= 3 and lem not in STOP_EXTRA and lem not in SPACY_STOP:
                cset.add(lem)
                rec(cw, lem, t.lower_, i)
            if i + 1 < len(toks):
                t2 = toks[i + 1]
                lem2 = t2.lemma_.lower()
                if (t.pos_ in ("NOUN", "ADJ") and t2.pos_ == "NOUN" and t.is_alpha and t2.is_alpha
                        and len(lem) >= 3 and len(lem2) >= 3
                        and lem not in STOP_EXTRA and lem2 not in STOP_EXTRA
                        and lem not in SPACY_STOP and lem2 not in SPACY_STOP):
                    cset.add(lem + " " + lem2)
                    rec(cw, lem + " " + lem2, t.lower_ + " " + t2.lower_, i + 1)
        for ent in docy.ents:
            if ent.label_ not in ENTITY_LABELS:
                continue
            name = re.sub(r"\s+", " ", re.sub(r"[^a-z ]", "", ent.text.lower())).strip()
            if len(name) >= 3 and name not in STOP_EXTRA and not name.isdigit():
                eset.add(name)
                rec(ew, name, ent.text.lower(), ent.start)
        concept_sets.append(frozenset(cset))
        entity_sets.append(frozenset(eset))
        week_of.append(row["week"])
    print(f"   spaCy pass: {min(start + 40000, len(clean)):,}/{len(clean):,}")
print(f"{len(cw):,} concept terms | {len(ew):,} entity terms")
```

```
================================================================================
L2.1 VOCABULARY · spine step 2/7 — one pass reads every headline and records its words, lemma-normalized, plus named entities
(prompt-chain stage 4/5: extract)
================================================================================
   spaCy pass: 40,000/289,147
   spaCy pass: 80,000/289,147
   spaCy pass: 120,000/289,147
   spaCy pass: 160,000/289,147
   spaCy pass: 200,000/289,147
   spaCy pass: 240,000/289,147
   spaCy pass: 280,000/289,147
   spaCy pass: 289,147/289,147
55,630 concept terms | 142,242 entity terms
```

In [5]:

```
banner("L2.2 WIRE RESIDUE — words that appear almost only inside repeated wire phrases are deleted from the vocabulary")
tpl_edge = {k for k, c in global_edge.items() if c >= 150}
artifact = {}
for term, grams in edge_occ.items():
    tot = occ_total[term]
    if tot >= 150 and sum(c for k, c in grams.items() if k in tpl_edge) / tot >= 0.5:
        artifact[term] = tot
print(f"{len(artifact)} terms flagged as wire-format artifacts and excluded from mining")

banner("L2.3 BURSTINESS — vocabulary must appear in >= 20 weeks and in bursts; a word ticking at a constant rate is a feed artifact, not news")
N_WEEKS = heads["week"].nunique()


def burst_ok(wlist):
    per = Counter(wlist)
    mu = len(wlist) / N_WEEKS
    var = (np.array(list(per.values()), float) ** 2).sum() / N_WEEKS - mu ** 2
```

```
    return len(per) >= 20 and (var / mu if mu > 0 else 0) >= 1.2


banner("L2.4 THINGS vs MARKET PROCESS — keep a word only if it names a thing in the world (oil, vaccine, store) rather than
market process (profit, outlook, prices); vector margin > 0.25 or lexicon")
md_vec = spacy.load("en_core_web_md", disable=["tagger", "parser", "ner", "lemmatizer", "attribute_ruler"])
CONCEPT_SEEDS = """oil copper gold silver gas coal iron ore steel aluminum wheat corn coffee cotton
gasoline diesel uranium lithium ethanol phone smartphone device tablet gadget chip semiconductor
computer laptop internet broadband wireless network satellite cloud software app card camera
screen television media film game car truck vehicle airline plane ship rail freight shipping
logistics drug medicine vaccine hospital clinic biotech store supermarket hotel restaurant
farm mine drill refinery pipeline power solar wind electricity battery robot robotics drone
home house building factory machine tool food beer tobacco advertising telecom""".split()
MARKET_SEEDS = """profit loss revenue earnings price prices pricing rating ratings outlook
forecast estimate target jobs job employment sector service services market markets
stock stocks share shares bond bonds yield yields rate rates dollar euro yen yuan franc
inflation debt growth recession risk value index benchmark investor fund portfolio
dividend capital liquidity credit loan merger acquisition ipo trade tariff tax budget
deficit gdp payroll spending stimulus policy demand supply
cut cuts unit buy sell debut discount deal deals stake bid offer""".split()
LEXICON = {"supply chain", "credit card", "big data", "smart home", "internet of things",
           "social media", "e-commerce", "mobile phone", "cell phone", "electric vehicle",
           "electric car", "renewable energy", "wind farm", "solar power", "crude oil",
           "oil price", "gold price", "copper price", "natural gas", "shale oil",
           "oil", "oil spill", "barrel"}
Sv = np.array([md_vec(w).vector for w in CONCEPT_SEEDS])
Sv = Sv[np.linalg.norm(Sv, axis=1) > 0]
Mv = np.array([md_vec(w).vector for w in MARKET_SEEDS])
Mv = Mv[np.linalg.norm(Mv, axis=1) > 0]
Sv /= np.linalg.norm(Sv, axis=1, keepdims=True)
Mv /= np.linalg.norm(Mv, axis=1, keepdims=True)


# concept-vs-market margin: mean top-3 cosine to thing-seeds minus market-process seeds
def vec_margin(term):
    v = md_vec(term).vector
    if not v.any():
        return None
    v = v / np.linalg.norm(v)
    return float(np.sort(Sv @ v)[::-1][:3].mean() - np.sort(Mv @ v)[::-1][:3].mean())


cand_c = [t for t in cw
          if (len(cw[t]) >= (25 if t in LEXICON else 80)) and burst_ok(cw[t]) and t not in artifact]
cand_c.sort(key=lambda t: -len(cw[t]))
cand_e = [t for t in ew if len(ew[t]) >= 60 and burst_ok(ew[t]) and t not in artifact]
cand_e.sort(key=lambda t: -len(ew[t]))
margins = {t: vec_margin(t) for t in cand_c}
kept_c = [t for t in cand_c if t in LEXICON or (margins[t] is not None and margins[t] > 0.25)]
verdicts = pd.DataFrame([
    {"term": t, "margin": round(margins[t], 3) if margins[t] is not None else None,
     "admitted": bool(t in LEXICON or (margins[t] is not None and margins[t] > 0.25))}
    for t in cand_c[:20]])
display(verdicts)
print(f"concept candidates {len(cand_c)} -> kept {len(kept_c)} | entity candidates {len(cand_e)}")
```

```
================================================================================
L2.2 WIRE RESIDUE — words that appear almost only inside repeated wire phrases are deleted from the vocabulary
================================================================================
39 terms flagged as wire-format artifacts and excluded from mining

================================================================================
L2.3 BURSTINESS — vocabulary must appear in >= 20 weeks and in bursts; a word ticking at a constant rate is a feed artifact, n
ot news
================================================================================

================================================================================
L2.4 THINGS vs MARKET PROCESS — keep a word only if it names a thing in the world (oil, vaccine, store) rather than market pro
cess (profit, outlook, prices); vector margin > 0.25 or lexicon
================================================================================
```

	term	margin	admitted
0	share	-0.427	False
1	stock	-0.589	False
2	bank	-0.107	False
3	market	-0.367	False
4	profit	-0.427	False
5	price	-0.191	False
6	sale	-0.063	False
7	rate	-0.290	False
8	investor	-0.427	False
9	year	-0.114	False
10	bond	-0.384	False
11	oil	0.506	True
12	growth	-0.331	False
13	dollar	-0.341	False
14	firm	-0.427	False
15	debt	-0.210	False
16	euro	-0.502	False
17	risk	-0.145	False
18	rise	-0.244	False
19	datum	-0.041	False

```
concept candidates 798 -> kept 68 | entity candidates 260
```

In [6]:

```
banner("L2.5 INDUSTRY FAMILY · spine step 2/7 — group the kept vocabulary into synonym themes (oil/crude/fuel become one theme); this is the GENERIC baseline, not company-specific")


def cluster(pool, sets, weeks_list):
    idx = {t: i for i, t in enumerate(pool)}
    r, c = [], []
    for hi, fs in enumerate(sets):
        for t in fs:
            j = idx.get(t)
            if j is not None:
                r.append(hi)
                c.append(j)
    X = coo_matrix((np.ones(len(r)), (r, c)), shape=(len(sets), len(pool))).tocsr()
    N = X.shape[0]
    co = (X.T @ X).toarray().astype(float)
    px = np.diag(co) / N
    with np.errstate(divide="ignore", invalid="ignore"):
        npmi = np.log((co / N) / (px[:, None] * px[None, :])) / (-np.log(co / N))
    npmi[co == 0] = -1.0
    np.fill_diagonal(npmi, 1.0)
    dist = np.clip(1.0 - npmi, 0, 2)
    np.fill_diagonal(dist, 0.0)
    dist = (dist + dist.T) / 2
    cl = AgglomerativeClustering(n_clusters=None, distance_threshold=0.75,
                                 metric="precomputed", linkage="average").fit(dist)
    groups = defaultdict(list)
    for t, cid in zip(pool, cl.labels_):
        groups[int(cid)].append(t)
    tot = {t: len(weeks_list[t]) for t in pool}
    out = []
    for members in groups.values():
        if len(members) == 1 and members[0] not in LEXICON:
            continue
        members.sort(key=lambda t: -tot[t])
        out.append((members[0], members))
    out.sort(key=lambda x: -sum(tot[t] for t in x[1]))
    return out


themes_c = cluster(kept_c[:400], concept_sets, cw)
themes_e = cluster(cand_e[:150], entity_sets, ew)

banner("L2.6 FAMILY SIGNALS — each theme becomes one weekly number: the share of that week's headlines mentioning it")
wk_index = pd.PeriodIndex(sorted(heads["week"].unique()), freq="W-FRI")
```

```
wk_pos = {w: i for i, w in enumerate(wk_index)}
denom = np.zeros(len(wk_index))
for w in week_of:
    denom[wk_pos[w]] += 1


def intensity_df(themes, sets):
    member_sets = [frozenset(m) for _, m in themes]
    arr = np.zeros((len(themes), len(wk_index)))
    for fs, w in zip(sets, week_of):
        for ti, ms in enumerate(member_sets):
            if fs & ms:
                arr[ti, wk_pos[w]] += 1
    df = pd.DataFrame((arr / np.maximum(denom, 1)).T, index=wk_index,
                      columns=[d for d, _ in themes])
    return df.loc[:, (df > 0).sum() >= 15]


int_c = intensity_df(themes_c, concept_sets)
int_e = intensity_df(themes_e, entity_sets)
shared_features = pd.concat([int_c.add_suffix(" [c]"), int_e.add_suffix(" [e]")], axis=1)
column_map = {}
for fam, themes in [("c", themes_c), ("e", themes_e)]:
    for d, mem in themes:
        column_map[d + f" [{fam}]"] = {"family": fam, "display": d, "members": mem}
print(f"shared non-referencing family: concept {int_c.shape[1]} | entity {int_e.shape[1]} | total
{shared_features.shape[1]} themes over {len(shared_features)} weeks")
```

```
================================================================================
L2.5 INDUSTRY FAMILY · spine step 2/7 — group the kept vocabulary into synonym themes (oil/crude/fuel become one theme); this
is the GENERIC baseline, not company-specific
================================================================================
================================================================================
L2.6 FAMILY SIGNALS — each theme becomes one weekly number: the share of that week's headlines mentioning it
================================================================================
shared non-referencing family: concept 21 | entity 40 | total 61 themes over 560 weeks
```

3. Company resolution

Resolves the ticker to company words from the Yahoo Finance names and counts the headlines that contain any of them. The sample lines below the counts show typical matches.

In [7]:

```
banner("L2 RESOLUTION · spine step 3/7 — which headlines name this company? ticker -> company words; per-word match counts
expose over-matches")
tok_sets = clean["tokens"].map(frozenset).to_numpy()
NAME_STOP = STOP_EXTRA | MONTHS | SPACY_STOP | {"corporation", "incorporated", "limited", "companies"}

info = yf.Ticker(TICKER).info or {}
names = {info.get("shortName"), info.get("longName")} - {None, ""}
words = set()
for name in names:
    for frag in re.findall(r"[A-Z][a-z]+|[A-Z]{2,}", str(name)):
        w = frag.lower()
        if len(w) > 2 and w not in NAME_STOP:
            words.add(w)
    base = re.sub(r"(inc|corp|corporation|ltd|plc|co|company|holdings|group|ag|se|nv|sa)\.?$", "",
                  re.sub(r"[^\w\s]", " ", str(name).lower())).strip()
    for w in base.split():
        if len(w) > 2 and w not in NAME_STOP:
            words.add(w)
resolve = sorted(words)
sym = TICKER.lower()
if len(sym) > 2 and sym.isalpha() and sym not in resolve:
    rate = float(np.mean([sym in s for s in tok_sets]))
    if rate < 0.002:
        resolve.append(sym)
    else:
        print(f"symbol word '{sym}' fails the rarity gate (matches {rate:.2%} of headlines); excluded")
counts = {w: int(sum(w in s for s in tok_sets)) for w in resolve}
print(f"{TICKER} resolved words: " + ", ".join(f"{w} {c}" for w, c in sorted(counts.items(), key=lambda kv: -kv[1])))
m_mask = np.array([bool(s & set(resolve)) for s in tok_sets])
for i in np.where(m_mask)[0][:6]:
    print(f"   - {clean['headline'].iloc[i][:88]}")
```

```
================================================================================
L2 RESOLUTION · spine step 3/7 — which headlines name this company? ticker -> company words; per-word match counts expose over
-matches
================================================================================
```

```
AAPL resolved words: apple 813, aapl 18
   - Apple Press Conference: Insider Report by PowerPennyStocks.com
   - Apple prepares big bet on tablet computing
   - Timeline: Apple product introductions and milestones
   - Instant View: Apple unveils iPad, hoping for new hit
   - US STOCKS-Dow, S&P; dip after Fed; Apple lifts Nasdaq
   - Wall Street gains on Fed's view; Apple lifts Nasdaq
```

4. Tone labels, the company bin, and mined seeds

Trains the sentiment classifier on the unanimous-label sentences and labels every headline. The company bin counts weekly mentions and their tone. Seed mining keeps words that co-occur with the company's mentions at least five times, at twice the baseline rate, and in at least 15 weeks of the corpus.

In [8]:
```
banner("L3.1 TONE LABELS · prompt-chain stage 3/5 CLASSIFY — one classifier trained on unanimous-label financial sentences
labels every headline")
pb_ds = load_dataset("takala/financial_phrasebank", "sentences_allagree", trust_remote_code=True)["train"]
pb = pb_ds.to_pandas()
pb["sentiment"] = pb["label"].map(dict(enumerate(pb_ds.features["label"].names)))
pb = pb.drop_duplicates(subset="sentence").reset_index(drop=True)
clf_senti = Pipeline([
    ("tfidf", TfidfVectorizer(lowercase=True, ngram_range=(1, 2), min_df=2,
                              sublinear_tf=True, max_features=10_000)),
    ("classifier", LogisticRegression(max_iter=1000, class_weight="balanced", random_state=RANDOM_STATE)),
]).fit(pb["sentence"], pb["sentiment"])
fifty = load_dataset("takala/financial_phrasebank", "sentences_50agree", trust_remote_code=True)["train"].to_pandas()
fifty["sentiment"] = fifty["label"].map({0: "negative", 1: "neutral", 2: "positive"})
tail = fifty.drop_duplicates(subset="sentence")
tail = tail[~tail["sentence"].isin(set(pb["sentence"]))]
tail_pred = clf_senti.predict(tail["sentence"])
print(f"trained on {len(pb):,} unanimous-label sentences | disagreement-tail transfer "
      f"accuracy {accuracy_score(tail['sentiment'], tail_pred):.3f} "
      f"(neutral share of predictions {np.mean(tail_pred == 'neutral'):.2f})")
labels_all = clf_senti.predict(clean["headline"])
print(f"labeled {len(labels_all):,} corpus headlines")

banner("L3.2 COMPANY BIN · spine step 3/7 — the company's own weekly coverage: how much, and its tone")
denom_series = pd.Series(denom, index=wk_index)
hits = pd.Series(0.0, index=wk_index)
pos = pd.Series(0.0, index=wk_index)
neg = pd.Series(0.0, index=wk_index)
for i in np.where(m_mask)[0]:
    w = week_of[i]
    hits[w] += 1
    if labels_all[i] == "positive":
        pos[w] += 1
    elif labels_all[i] == "negative":
        neg[w] += 1
company_series = pd.DataFrame({
    "COMPANY|share": (hits / denom_series.replace(0, np.nan)).fillna(0),
    "COMPANY|senti": ((pos - neg) / hits.replace(0, np.nan)).fillna(0),
})
print(f"{TICKER} named in {int(m_mask.sum())} headlines across {int((company_series['COMPANY|share'] > 0).sum())} of
{len(wk_index)} weeks; samples (headline, label):")
for i in np.where(m_mask)[0][:5]:
    print(f"   [{labels_all[i]:>8}] {clean['headline'].iloc[i][:80]}")
```

```
================================================================================
L3.1 TONE LABELS · prompt-chain stage 3/5 CLASSIFY — one classifier trained on unanimous-label financial sentences labels ever
y headline
================================================================================
trained on 2,259 unanimous-label sentences | disagreement-tail transfer accuracy 0.620 (neutral share of predictions 0.79)
labeled 289,147 corpus headlines

================================================================================
L3.2 COMPANY BIN · spine step 3/7 — the company's own weekly coverage: how much, and its tone
================================================================================
AAPL named in 825 headlines across 276 of 560 weeks; samples (headline, label):
   [ neutral] Apple Press Conference: Insider Report by PowerPennyStocks.com
   [ neutral] Apple prepares big bet on tablet computing
   [ neutral] Timeline: Apple product introductions and milestones
   [ neutral] Instant View: Apple unveils iPad, hoping for new hit
   [ neutral] US STOCKS-Dow, S&P; dip after Fed; Apple lifts Nasdaq
```

In [9]:
```
banner(f"L4 SEEDS · spine step 4/7 — which words appear unusually often NEXT TO {TICKER}'s name? (>=5 co-occurrences, >=2x
baseline rate, names a thing, >=15 weeks corpus-wide; lexicon terms get a 3-hit floor)")
corpus_member = Counter()
for s in concept_sets:
    corpus_member.update(s)
```

```
def mine_seeds(min_hits=5, min_lift=2.0, exclude=frozenset(), mask=None, self_words=None):
    # mask/self_words let a second ticker reuse the same miner over its own mention set
    midx = np.where(m_mask if mask is None else mask)[0]
    n_m = len(midx)
    cnt = Counter()
    for i in midx:
        cnt.update(concept_sets[i])
    self_words = set(resolve) if self_words is None else self_words
    rows = []
    for term, mhits in cnt.items():
        # lexicon fixed phrases ride a lowered 3-hit floor; open-vocabulary terms need the full count
        floor = min(3, min_hits) if term in LEXICON else min_hits
        if mhits < floor:
            continue
        if any(w in self_words for w in term.split()):
            continue
        if term in exclude or (set(term.split()) & exclude):
            continue
        total = corpus_member[term]
        if total == 0:
            continue
        lift = (mhits / n_m) / (total / len(concept_sets))
        if lift < min_lift:
            continue
        marg = margins.get(term)
        if marg is None:
            marg = vec_margin(term)
        if not (term in LEXICON or (marg is not None and marg > 0.25)):
            continue
        weeks_sup = len(set(cw[term]))
        if weeks_sup < 15:
            continue
        rows.append({"seed": term, "mention_hits": mhits, "lift": round(lift, 1),
                     "margin": round(marg, 3) if marg is not None else None, "weeks": weeks_sup})
    rows.sort(key=lambda r: -r["mention_hits"])
    return rows


mined = mine_seeds()
display(pd.DataFrame(mined).assign(provenance="mined"))
```

```
================================================================================
L4 SEEDS · spine step 4/7 — which words appear unusually often NEXT TO AAPL's name? (>=5 co-occurrences, >=2x baseline rate, n
ames a thing, >=15 weeks corpus-wide; lexicon terms get a 3-hit floor)
================================================================================
```

	seed	mention_hits	lift	margin	weeks	provenance
0	tablet	7	55.8	0.754	39	mined
1	holiday	7	10.0	0.301	120	mined
2	energy	7	2.4	0.576	274	mined
3	dip	6	3.7	0.364	188	mined
4	watch	5	31.9	0.386	38	mined

5. Feature matrix

Turns each seed into two weekly columns, coverage share and tone, over all headlines. The matrix joins the seed columns, the company bin, and the shared theme family.

In [10]:

```
banner("L5 BINS · spine step 5/7 — every seed becomes two weekly signals over ALL headlines (most never mention the
company): coverage share, and tone of exactly that coverage")
seed_pool = sorted({r["seed"] for r in mined})
seed_set = frozenset(seed_pool)
tw = {s: defaultdict(lambda: [0, 0, 0]) for s in seed_pool}
for i, s in enumerate(concept_sets):
    w = week_of[i]
    lab = labels_all[i]
    for term in (s & seed_set):
        rec_ = tw[term][w]
        rec_[0] += 1
        if lab == "positive":
            rec_[1] += 1
        elif lab == "negative":
            rec_[2] += 1

cols = {}
```

```
for r in mined:
    s = r["seed"]
    cnt = pd.Series({w: tw[s][w][0] for w in wk_index}, index=wk_index, dtype=float)
    pos_ = pd.Series({w: tw[s][w][1] for w in wk_index}, index=wk_index, dtype=float)
    neg_ = pd.Series({w: tw[s][w][2] for w in wk_index}, index=wk_index, dtype=float)
    cols[f"{s}|share"] = (cnt / denom_series.replace(0, np.nan)).fillna(0)
    cols[f"{s}|senti"] = ((pos_ - neg_) / cnt.replace(0, np.nan)).fillna(0)
X0 = pd.DataFrame(cols, index=wk_index).join(company_series)
shared_keep = [c for c in shared_features.columns
               if not (set(resolve) & {w for m in column_map[c]["members"] for w in m.split()})]
X0 = X0.join(shared_features[shared_keep])
print(f"{TICKER}: ticker-specific {2 * len(mined) + 2} cols + shared {X0.shape[1] - 2 * len(mined) - 2} cols =
{X0.shape[1]} features x {X0.shape[0]} weeks")
print("counting rule: a headline counts for seed T in week w when T is in the headline's extracted concept set; share
divides by that week's headline count, senti is (pos - neg) / hits")
X0.iloc[300].head(10)
```

```
================================================================================
L5 BINS · spine step 5/7 — every seed becomes two weekly signals over ALL headlines (most never mention the company): coverage
share, and tone of exactly that coverage
================================================================================
AAPL: ticker-specific 12 cols + shared 60 cols = 72 features x 560 weeks
counting rule: a headline counts for seed T in week w when T is in the headline's extracted concept set; share divides by that
week's headline count, senti is (pos - neg) / hits
```

Out[10]:

```
tablet|share     0.000000
tablet|senti     0.000000
holiday|share    0.002551
holiday|senti    0.000000
energy|share     0.001276
energy|senti     0.000000
dip|share        0.001276
dip|senti       -1.000000
watch|share      0.000000
watch|senti      0.000000
Name: 2015-10-03/2015-10-09, dtype: float64
```

6. Labels and train-window statistics

Downloads prices and the yield series, bins them by week, and labels each week with the next week's close-to-close direction. The statistics table correlates every feature with next-week return, the market control, and the excess return, with split-half checks and Benjamini-Hochberg q values.

In [11]:

```
banner("L6.1 LABELS · spine step 6/7 — does this week's signal predict NEXT week's direction? prices, market state, and the
SPY control (control only, never a feature)")
px = yf.download(" ".join(sorted({TICKER, "SPY"})), start="2010-01-01", end="2020-12-31", auto_adjust=True, progress=False)
["Close"]
wkpx = px.resample("W-FRI").last()
wkpx.index = wkpx.index.to_period("W-FRI")
wkpx["SPY_ret"] = wkpx["SPY"].pct_change()
wkpx["SPY_vol"] = wkpx["SPY_ret"].rolling(8).std()
wkpx["SPY_fwd"] = wkpx["SPY"].shift(-1) / wkpx["SPY"] - 1
fred = pd.read_csv("https://fred.stlouisfed.org/graph/fredgraph.csv?id=DGS10")
fred_w = pd.Series(pd.to_numeric(fred.iloc[:, 1], errors="coerce").values,
                   index=pd.PeriodIndex(pd.to_datetime(fred.iloc[:, 0]), freq="W-FRI"))
wkpx["dgs10_chg"] = fred_w.groupby(level=0).last().reindex(wkpx.index).diff()
print(f"{len(wkpx)} weekly price bins")

TRAIN_END = pd.Period("2017-06-23", "W-FRI")
VALID_END = pd.Period("2019-02-08", "W-FRI")
TR = wk_index <= TRAIN_END
VA = (wk_index > TRAIN_END) & (wk_index <= VALID_END)
TE = wk_index > VALID_END
print(f"split weeks: train {int(TR.sum())} (to 2017-06), valid {int(VA.sum())} (to 2019-02), test {int(TE.sum())} (from
2019-02)")


def ticker_frame():
    df = X0.copy()
    df["ret"] = wkpx[TICKER].pct_change().reindex(df.index)
    df["fwd"] = (wkpx[TICKER].shift(-1) / wkpx[TICKER] - 1).reindex(df.index)
    df["y"] = (df["fwd"] > 0).astype(float)
    for c in ["SPY_ret", "SPY_vol", "SPY_fwd", "dgs10_chg"]:
        df[c] = wkpx[c].reindex(df.index)
    return df
```

```
================================================================================
L6.1 LABELS · spine step 6/7 — does this week's signal predict NEXT week's direction? prices, market state, and the SPY contro
l (control only, never a feature)
================================================================================
```

```
574 weekly price bins
split weeks: train 390 (to 2017-06), valid 85 (to 2019-02), test 85 (from 2019-02)
```

In [12]:

```python
banner(f"L6.2 STATISTICS · spine step 6/7 — does any single signal relate to next-week {TICKER} return? raw (r), market-tracking (r_spy), ticker-specific (r_exc), consistency (halves), honest significance (q)")


def stats_table(through=None):
    df = ticker_frame()
    tr = df[(TR if through is None else df.index <= through) & df["fwd"].notna()]
    fwd, spyf, exc = tr["fwd"], tr["SPY_fwd"], tr["fwd"] - tr["SPY_fwd"]
    rows = []
    for col in X0.columns:
        s = tr[col]
        if s.std() == 0:
            continue
        r = float(np.corrcoef(s, fwd)[0, 1])
        rs = float(np.corrcoef(s, spyf)[0, 1])
        rxc = float(np.corrcoef(s, exc)[0, 1])
        h = len(tr) // 2
        r1 = float(np.corrcoef(s.iloc[:h], fwd.iloc[:h])[0, 1])
        r2 = float(np.corrcoef(s.iloc[h:], fwd.iloc[h:])[0, 1])
        n = len(tr)
        tt = abs(r) * math.sqrt((n - 2) / max(1e-12, 1 - r * r))
        rows.append({"feature": col, "support": int((s > 0).sum()), "r": round(r, 3),
                     "r_spy": round(rs, 3), "r_exc": round(rxc, 3),
                     "half1": round(r1, 3), "half2": round(r2, 3),
                     "stable": bool(np.sign(r1) == np.sign(r2) and abs(r1) > 0.05 and abs(r2) > 0.05),
                     "p": math.erfc(tt / math.sqrt(2))})
    scr = pd.DataFrame(rows)
    m = len(scr)
    if m == 0:
        return scr
    order = np.argsort(scr["p"].values)
    # Benjamini-Hochberg step-up over the screened family
    q = np.minimum.accumulate((scr["p"].values[order] * m / np.arange(1, m + 1))[::-1])[::-1]
    qmap = {o: q[i] for i, o in enumerate(order)}
    scr["q"] = [round(min(1.0, qmap[i]), 3) for i in range(m)]
    return scr.sort_values("r", key=abs, ascending=False).reset_index(drop=True)


scr = stats_table()
print(f"{len(scr)} features screened on {int((TR & ticker_frame()['fwd'].notna()).sum())} train weeks; top 20 by |r| below")
display(scr.head(20))
```

```
================================================================================
L6.2 STATISTICS · spine step 6/7 — does any single signal relate to next-week AAPL return? raw (r), market-tracking (r_spy), ticker-specific (r_exc), consistency (halves), honest significance (q)
================================================================================
72 features screened on 390 train weeks; top 20 by |r| below
```

	feature	support	r	r_spy	r_exc	half1	half2	stable	p	q
0	campaign [c]	156	-0.122	-0.068	-0.105	-0.169	-0.075	True	0.015893	0.464
1	imf [e]	211	0.119	0.042	0.118	0.167	0.018	False	0.018160	0.464
2	drug [c]	166	0.113	0.088	0.082	0.073	0.171	True	0.024983	0.464
3	swiss [e]	175	0.112	0.094	0.078	0.143	0.057	True	0.025802	0.464
4	ibm [e]	176	-0.102	-0.042	-0.097	-0.074	-0.146	True	0.044246	0.545
5	jpmorgan [e]	136	0.101	0.026	0.106	0.180	-0.004	False	0.045420	0.545
6	holiday [c]	134	-0.097	-0.051	-0.086	-0.041	-0.160	False	0.054148	0.557
7	spain [e]	250	-0.087	-0.096	-0.046	-0.107	-0.065	True	0.085239	0.644
8	japan [e]	353	0.085	0.123	0.027	0.129	-0.021	False	0.091876	0.644
9	COMPANY|senti	45	-0.085	-0.081	-0.052	-0.117	-0.052	True	0.093776	0.644
10	treasury [e]	292	0.082	-0.029	0.117	0.063	0.132	True	0.104446	0.644
11	steel [c]	158	-0.079	0.058	-0.131	-0.066	-0.100	True	0.117208	0.644
12	holiday|share	108	-0.079	-0.050	-0.064	-0.013	-0.150	False	0.119411	0.644
13	morgan stanley [e]	228	-0.077	-0.056	-0.059	-0.105	-0.043	False	0.126857	0.644
14	barrel [c]	62	0.076	0.122	0.016	0.130	-0.023	False	0.134278	0.644
15	hit [c]	243	0.074	0.053	0.057	0.128	-0.030	False	0.144747	0.644
16	unit [c]	294	-0.073	-0.012	-0.080	-0.055	-0.096	True	0.152131	0.644
17	russian [e]	219	0.068	0.013	0.074	-0.039	0.151	False	0.177525	0.707
18	italy [e]	209	0.066	0.028	0.063	0.074	0.050	True	0.190363	0.707
19	tie [c]	173	-0.065	-0.047	-0.049	-0.022	-0.157	False	0.199683	0.707

7. NLP-only arms

Fits the pre-registered logistic arms on the training weeks and scores them on the 85 test weeks against the majority and momentum baselines, with McNemar p values. An exploratory horizon check repeats the comparison at daily granularity.

In [13]:

```
banner("L7 ARMS · spine step 7/7 — can any model built on these signals beat always-up? each arm answers one named
question; pre-registered before the agent sessions")
ARM_QUESTIONS = {
    "majority": "always predict the training majority class",
    "momentum": "this week's direction repeats next week",
    "ticker-specific": "the company bin plus this ticker's mined words only",
    "shared family": "generic industry themes only, no company words",
    "all features": "every NLP signal",
    "all+macro": "every NLP signal plus market state",
    "macro-only": "market state alone: do the news signals add anything beyond it?",
    "residualized": "NLP signals with the market component stripped out: is anything left?",
    "stats top-8": "the 8 strongest training correlations only",
    "l1 auto-prune": "everything, sparsity-selected",
}
for k, q in ARM_QUESTIONS.items():
    print(f"   {k:<16} {q}")
MACRO = ["SPY_ret", "SPY_vol", "dgs10_chg"]
CS = [0.03, 0.1, 0.3, 1.0]


# the market component is fit on training weeks only; residuals replace the raw columns
def residualize(df, cols, tr_mask):
    Z = np.column_stack([np.ones(tr_mask.sum()), df.loc[tr_mask, MACRO].fillna(0).to_numpy()])
    beta, *_ = np.linalg.lstsq(Z, df.loc[tr_mask, cols].fillna(0).to_numpy(), rcond=None)
    Zh = np.column_stack([np.ones(len(df)), df[MACRO].fillna(0).to_numpy()])
    resid = df[cols].fillna(0).to_numpy() - Zh @ beta
    return pd.DataFrame(resid, index=df.index, columns=cols)


def run_arms():
    df = ticker_frame()
    tr = df[TR & df["fwd"].notna()]
    va = df[VA & df["fwd"].notna()]
    te = df[TE & df["fwd"].notna()]
    ytr, yva, yte = tr["y"].astype(int), va["y"].astype(int), te["y"].astype(int)
```

```
    trva = pd.concat([tr, va])
    out = {"n": (len(tr), len(va), len(te)), "te": te, "arms": {}}
    maj = float(ytr.mean() >= 0.5)
    pred = np.full(len(te), maj)
    out["arms"]["majority"] = {"pred": pred, "correct": pred == yte.values, "prob": None}
    mom = (te["ret"] > 0).astype(float).values
    out["arms"]["momentum"] = {"pred": mom, "correct": mom == yte.values, "prob": None}

    def make(pen, C):
        return LogisticRegression(max_iter=3000, C=C, penalty=pen,
                                  solver="liblinear" if pen == "l1" else "lbfgs",
                                  random_state=RANDOM_STATE)

    def fit(cols, pen, frames=None):
        dtr, dva, dte, dtrva = frames if frames else (tr, va, te, trva)
        best = None
        for C in CS:
            pipe = Pipeline([("scale", StandardScaler()), ("clf", make(pen, C))])
            pipe.fit(dtr[cols].fillna(0), ytr)
            try:
                auc = roc_auc_score(yva, pipe.predict_proba(dva[cols].fillna(0))[:, 1])
            except ValueError:
                auc = np.nan
            if best is None or (not np.isnan(auc) and (np.isnan(best[0]) or auc > best[0])):
                best = (auc, C)
        pipe = Pipeline([("scale", StandardScaler()), ("clf", make(pen, best[1]))])
        pipe.fit(dtrva[cols].fillna(0), dtrva["y"].astype(int))
        prob = pipe.predict_proba(dte[cols].fillna(0))[:, 1]
        pred = (prob > 0.5).astype(float)
        return {"pred": pred, "correct": pred == yte.values, "prob": prob}

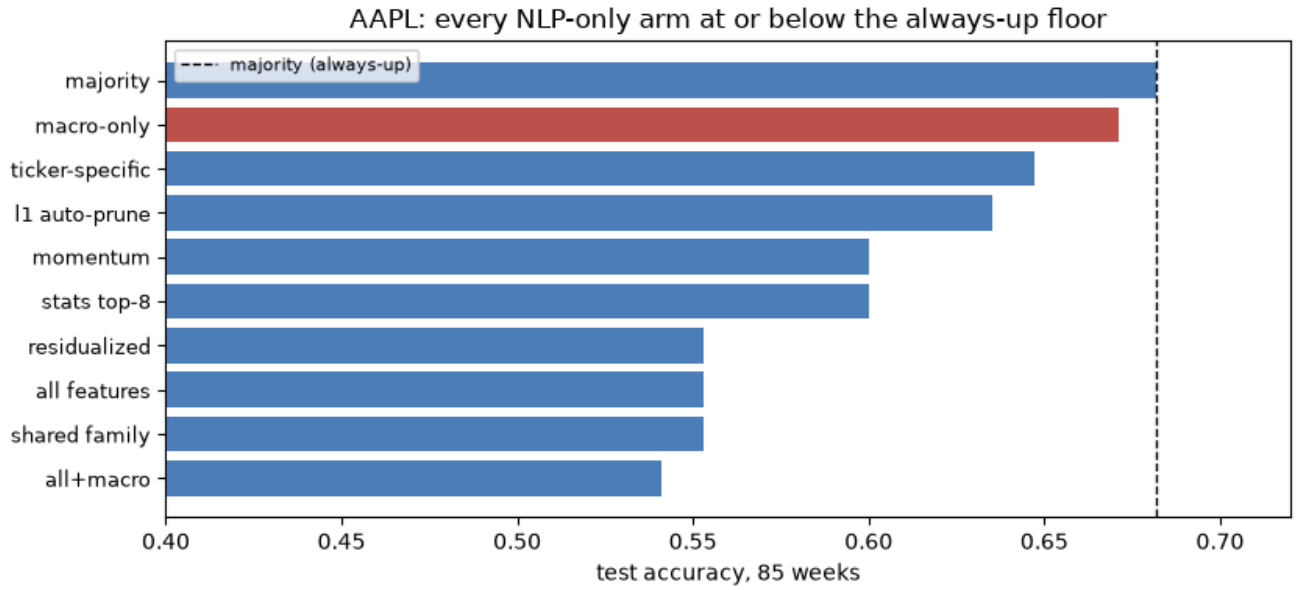
    specific = [c for c in X0.columns if "|" in c]
    shared = [c for c in X0.columns if c.endswith(" [c]") or c.endswith(" [e]")]
    rmap = {}
    for c in X0.columns:
        s = tr[c]
        if s.std() > 0:
            rmap[c] = abs(float(np.corrcoef(s, tr["fwd"])[0, 1]))
    top8 = sorted(rmap, key=rmap.get, reverse=True)[:8]
    resid_all = residualize(df, specific + shared, TR & df["fwd"].notna())
    df_r = df.copy()
    df_r[resid_all.columns] = resid_all
    tr_r = df_r[TR & df["fwd"].notna()]
    va_r = df_r[VA & df["fwd"].notna()]
    te_r = df_r[TE & df["fwd"].notna()]
    trva_r = pd.concat([tr_r, va_r])
    use = [("ticker-specific", specific, None), ("shared family", shared, None),
           ("all features", specific + shared, None), ("all+macro", specific + shared + MACRO, None),
           ("macro-only", MACRO, None), ("stats top-8", top8, None),
           ("residualized", specific + shared, (tr_r, va_r, te_r, trva_r))]
    for name, cols_, frames in use:
        out["arms"][name] = fit(cols_, "l2", frames)
    out["arms"]["l1 auto-prune"] = fit(specific + shared + MACRO, "l1")
    return out


res = run_arms()
yte = res["te"]["y"].astype(int)
maj_ok = res["arms"]["majority"]["correct"]
rows = []
for name, a in res["arms"].items():
    row = {"arm": name, "test accuracy": round(float(a["correct"].mean()), 3),
           "test F1(up)": round(f1_score(yte, a["pred"].astype(int), zero_division=0), 3),
           "long weeks": int(a["pred"].sum()),
           "McNemar p vs majority": round(mcnemar_p(a["correct"], maj_ok), 3)}
    if a["prob"] is not None:
        row["test AUC"] = round(roc_auc_score(yte, a["prob"]), 3)
    rows.append(row)
eval_table = pd.DataFrame(rows).set_index("arm")
print(f"weeks: train/valid/test = {res['n'][0]}/{res['n'][1]}/{res['n'][2]}")
display(eval_table)

fig, ax = plt.subplots(figsize=(8.5, 4))
order = eval_table.sort_values("test accuracy")
ax.barh(range(len(order)), order["test accuracy"],
        color=["#c0504d" if i == "macro-only" else "#4f81bd" for i in order.index])
ax.set_yticks(range(len(order)), order.index, fontsize=9)
ax.axvline(eval_table.loc["majority", "test accuracy"], color="black", ls="--", lw=1, label="majority (always-up)")
ax.set_xlim(0.40, 0.72)
ax.legend(fontsize=8)
ax.set_xlabel(f"test accuracy, {res['n'][2]} weeks")
ax.set_title(f"{TICKER}: every NLP-only arm at or below the always-up floor")
plt.tight_layout()
plt.show()
```

```
================================================================================
L7 ARMS · spine step 7/7 — can any model built on these signals beat always-up? each arm answers one named question; pre-regis
tered before the agent sessions
================================================================================
   majority         always predict the training majority class
   momentum         this week's direction repeats next week
   ticker-specific  the company bin plus this ticker's mined words only
   shared family    generic industry themes only, no company words
   all features     every NLP signal
   all+macro        every NLP signal plus market state
   macro-only       market state alone: do the news signals add anything beyond it?
   residualized     NLP signals with the market component stripped out: is anything left?
   stats top-8      the 8 strongest training correlations only
   l1 auto-prune    everything, sparsity-selected
weeks: train/valid/test = 390/85/85
```

arm	test accuracy	test F1(up)	long weeks	McNemar p vs majority	test AUC
majority	0.682	0.811	85	1.000	NaN
momentum	0.600	0.707	58	0.248	NaN
ticker-specific	0.647	0.779	78	0.453	0.442
shared family	0.553	0.661	54	0.071	0.483
all features	0.553	0.672	58	0.052	0.468
all+macro	0.541	0.667	59	0.029	0.467
macro-only	0.671	0.794	78	1.000	0.467
stats top-8	0.600	0.734	70	0.118	0.508
residualized	0.553	0.672	58	0.052	0.462
l1 auto-prune	0.635	0.763	73	0.388	0.456



In [14]:

```
banner("L7.2 HORIZON CHECK — exploratory, asked after seeing the weekly null: do tone reactions live at the daily horizon
that the weekly bins dilute?")
dpx = px.copy()
dpx["ret0"] = dpx[TICKER].pct_change()
dpx["ret1"] = dpx["ret0"].shift(-1)
dpx["spy0"] = dpx["SPY"].pct_change()
labels_arr = np.array(labels_all)
rows_ = []
means = {}
for lab in ["positive", "neutral", "negative"]:
    idx = np.where(m_mask & (labels_arr == lab))[0]
    dates = pd.DatetimeIndex(clean["date"].iloc[idx])
    r0 = dpx["ret0"].reindex(dates).dropna()
    r1 = dpx["ret1"].reindex(dates).dropna()
    e0 = (dpx["ret0"] - dpx["spy0"]).reindex(dates).dropna()
    means[lab] = (float(r0.mean()) if len(r0) else float("nan"), float(r1.mean()) if len(r1) else float("nan"))
    rows_.append({"tone": lab, "n": len(idx),
                  "mean same-day": f"{means[lab][0]:+.3%}" if len(r0) else "-",
                  "mean next-day": f"{means[lab][1]:+.3%}" if len(r1) else "-",
                  "same-day excess vs SPY": f"{float(e0.mean()):+.3%}" if len(e0) else "-"})
display(pd.DataFrame(rows_))
```

```
================================================================================
L7.2 HORIZON CHECK — exploratory, asked after seeing the weekly null: do tone reactions live at the daily horizon that the wee
kly bins dilute?
================================================================================
```

	tone	n	mean same-day	mean next-day	same-day excess vs SPY
0	positive	74	-0.961%	-0.063%	-0.790%
1	neutral	715	-0.253%	-0.079%	-0.240%
2	negative	36	-2.687%	+0.797%	-2.421%

8. The agent layer

Defines the call pattern shared by every agent role, the memory file, and the tool registry. Every role reads a fixed standing instruction block, and replies are cached by prompt hash at temperature zero. The sections below define the proposer, the planner, the news desk, the earnings desk with its deterministic Market Lane, and the reflection writer.

In [16]:

```
banner("L8.0 LLM LAYER — one call pattern for every agent: temperature 0, disk cache keyed by prompt hash, standing block
printed once, session tables printed in the cell, verbatim reply printed")
load_dotenv(REPO_ROOT / ".env")
GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY")
LLM_MODEL = "gemini-3.5-flash-lite"
PRINTED_BLOCKS = set()
CALL_LOG = []
EVAL_LOG = []
NEWS_TOOL_CALLS = []
TWIN_MISSES = []
ADVOCATE_MISSES = []
SIGN_FLAGS = []
ADVOCATE_CITE_MISSES = []

if GEMINI_API_KEY:
    from google import genai
    from google.genai import types
    gemini_client = genai.Client(api_key=GEMINI_API_KEY)


def parse_json_reply(text):
    text = str(text).strip().removeprefix("```json").removeprefix("```").removesuffix("```")
    start, end = text.find("{"), text.rfind("}")
    try:
        return json.loads(text[start:end + 1])
    except (json.JSONDecodeError, AttributeError):
        return {}


def call_llm(agent, standing_block, tables_block):
    prompt = standing_block + "\n\n" + tables_block
    if agent not in PRINTED_BLOCKS:
        print(f"--- STANDING BLOCK: {agent} (printed once; every {agent} prompt = this block + that session's tables) ---")
        print(standing_block)
        PRINTED_BLOCKS.add(agent)
    key = hashlib.sha256((LLM_MODEL + prompt).encode()).hexdigest()[:24]
    cache_file = LLM_CACHE / f"{key}.json"
    if cache_file.exists():
        reply = json.loads(cache_file.read_text(encoding="utf-8"))
        CALL_LOG.append((agent, "cache"))
    elif GEMINI_API_KEY:
        for attempt in range(3):
            try:
                reply = gemini_client.models.generate_content(model=LLM_MODEL, contents=prompt,
                                                              config=types.GenerateContentConfig(temperature=0.0)).text
                break
            except Exception as exc:
                if attempt == 2:
                    print(f"[{agent}] API call failed after retries: {type(exc).__name__}: {str(exc)[:120]}")
                    return None
                time.sleep(2 * (attempt + 1))
        cache_file.write_text(json.dumps(reply), encoding="utf-8")
        CALL_LOG.append((agent, "live"))
    else:
        print(f"[{agent}] SKIPPED — GEMINI_API_KEY not set; the NLP-only arms above stand alone")
        return None
    print(f"--- {agent} REPLY ---")
    print(reply)
    return parse_json_reply(reply)
```

```
print("LLM layer armed." if GEMINI_API_KEY else "GEMINI_API_KEY not set; agent cells below are skipped with a notice.")
```

```
================================================================================
L8.0 LLM LAYER — one call pattern for every agent: temperature 0, disk cache keyed by prompt hash, standing block printed onc
e, session tables printed in the cell, verbatim reply printed
================================================================================
LLM layer armed.
```

In [17]:
```
banner("L9 MEMORY — the persistent banned-vocabulary ledger, pre-seeded from the deterministic wire/suffix/calendar lists
(the former hardcodes), extended only by planner strikes that generalize; plus the per-ticker call ledger")
MEMORY_FILE = DATA_PROCESSED / "agent_memory.json"
memory = {
    "banned": [{"word": w, "origin": "deterministic", "reason": "wire format / corporate suffix / calendar language"}
               for w in sorted(set(NAME_STOP) | set(STOP_EXTRA))],
    "ledger": [],
    "planner_notes": [],
}
if MEMORY_FILE.exists():
    archived = DATA_PROCESSED / "agent_memory_archived.json"
    archived.write_text(MEMORY_FILE.read_text(encoding="utf-8"), encoding="utf-8")
    print(f"prior memory archived at {archived.name}; every run starts from the deterministic seed, so reruns are
deterministic")
MEMORY_FILE.write_text(json.dumps(memory, indent=1), encoding="utf-8")


def banned_words():
    return {b["word"] for b in memory["banned"]}


# generalize=true strikes must hold for every company, so they persist into every later ticker's admission
def write_ban(word, reason, origin="planner"):
    if word not in banned_words():
        memory["banned"].append({"word": word, "origin": origin, "reason": reason})
        MEMORY_FILE.write_text(json.dumps(memory, indent=1), encoding="utf-8")
        print(f"MEMORY WRITE: '{word}' banned for all later sessions — {reason}")


print(f"{len(banned_words())} pre-seeded bans (origin: deterministic); planner strikes with generalize=true append here and
print when they bite")
```

```
================================================================================
L9 MEMORY — the persistent banned-vocabulary ledger, pre-seeded from the deterministic wire/suffix/calendar lists (the former
hardcodes), extended only by planner strikes that generalize; plus the per-ticker call ledger
================================================================================
prior memory archived at agent_memory_archived.json; every run starts from the deterministic seed, so reruns are deterministic
391 pre-seeded bans (origin: deterministic); planner strikes with generalize=true append here and print when they bite
```

In [18]:
```
banner("L8.1 TOOLS — the registry agent actions dispatch into; dynamic use means the agent's reply decides which tool runs,
with which pre-registered parameters")
EDGAR_CACHE = DATA_PROCESSED / "edgar_cache"


def tool_headline_sample(term, k=5, after_week=None):
    idx = [i for i, s in enumerate(concept_sets) if term in s]
    if after_week is not None:
        idx = [i for i in idx if -int((wk_index[-1] - week_of[i]).n) >= after_week]
    if not idx:
        return []
    seed = RANDOM_STATE + int(hashlib.md5(term.encode()).hexdigest()[:8], 16)
    rng = np.random.default_rng(seed)
    picks = rng.choice(len(idx), size=min(k, len(idx)), replace=False)
    return [{"week_offset": -int((wk_index[-1] - week_of[idx[j]]).n // 7),
             "headline": clean["headline"].iloc[idx[j]], "label": labels_all[idx[j]]} for j in picks]


def tool_company_profile(ticker=TICKER):
    info = yf.Ticker(ticker).info or {}
    return " ".join(str(x) for x in [info.get("longName"), info.get("sector"), info.get("industry"),
info.get("longBusinessSummary")] if x)


def tool_edgar_filings(now):
    EDGAR_CACHE.mkdir(exist_ok=True)
    cf = EDGAR_CACHE / f"{TICKER}.json"
    if cf.exists():
        data = json.loads(cf.read_text(encoding="utf-8"))
    else:
        try:
            headers = {"User-Agent": "AAI-520 University Research (psuranthiran@sandiego.edu)", "Accept":
"application/json"}
            r1 = requests.get("https://www.sec.gov/files/company_tickers.json", headers=headers, timeout=30)
            r1.raise_for_status()
            mapping = r1.json()
            cik = int(next(v["cik_str"] for v in mapping.values() if v["ticker"] == TICKER))
            time.sleep(0.4)
```

```
            r2 = requests.get(f"https://data.sec.gov/submissions/CIK{cik:010d}.json", headers=headers, timeout=30)
            r2.raise_for_status()
            sub = r2.json()
            rec = sub["filings"]["recent"]
            rows = [{"form": f, "date": d} for f, d in zip(rec["form"], rec["filingDate"])]
            for p in sub["filings"].get("files", []):
                if min(r["date"] for r in rows) <= "1995-01-01":
                    break
                time.sleep(0.4)
                rp = requests.get(f"https://data.sec.gov/submissions/{p['name']}", headers=headers, timeout=30)
                rp.raise_for_status()
                fp = rp.json()
                rows += [{"form": f, "date": d} for f, d in zip(fp["form"], fp["filingDate"])]
            data = {"cik": cik, "filings": rows}
            cf.write_text(json.dumps(data), encoding="utf-8")
        except Exception as exc:
            print(f"EDGAR fetch failed for {TICKER} ({type(exc).__name__}: {str(exc)[:100]}); the earnings desk proceeds without filings")
            return []
    out = []
    for f in data["filings"]:
        if f["form"] not in ("10-K", "10-Q", "8-K"):
            continue
        wk = int((now - pd.Timestamp(f["date"])).days // 7)
        if 0 <= wk <= 260:
            out.append(f"{f['form']} ({wk} weeks before now)")
    return out[:12]


def remine_escalated(exclude, mask=None, self_words=None):
    rows = sorted(mine_seeds(min_hits=3, min_lift=1.5, exclude=exclude, mask=mask, self_words=self_words), key=lambda r: -r["mention_hits"])[:6]
    return [{"seed": r["seed"], "provenance": "mined-escalated", "gloss": "surfaced at the escalated floor after planner strikes"}
            for r in rows]


print("registry: headline_sample(term, k, after_week) | company_profile() | edgar_filings(now) | remine(level='escalated', bans)")
```

```
================================================================================
L8.1 TOOLS — the registry agent actions dispatch into; dynamic use means the agent's reply decides which tool runs, with which pre-registered parameters
================================================================================
registry: headline_sample(term, k, after_week) | company_profile() | edgar_filings(now) | remine(level='escalated', bans)
```

In [19]:

```
banner(f"L8.2 PROPOSER — before the corpus gets a say: which words' coverage plausibly transmits to {TICKER}? profile text only; no numbers, no dates; every proposal then passes the same deterministic gates as mined seeds")
PROPOSER_BLOCK = """You are the concept proposer in an investment-research pipeline. You receive one company's self-description and nothing else — no market data, no news, no numbers. Name at most 10 terms whose news coverage would plausibly transmit to this company's business through a channel it actually earns or spends through. Strongly prefer single plain words a financial newswire headline would use (chip, battery, retail, refinery); use a two-word term only for fixed phrases (supply chain, crude oil). For each term give a one-line hunch on the expected transmission. Prefer specific things (products, inputs, demand drivers, regulations it faces); never market-process words (profit, outlook, ratings, sector, prices) — those are never admitted downstream. Reply with JSON only: {"terms": [{"term": "...", "hunch": "..."}]}"""

LAST_ADMIT = []


def propose_terms(ticker=TICKER):
    tables = "COMPANY PROFILE:\n" + tool_company_profile(ticker)[:1800]
    print("--- PROPOSER TABLES (verbatim — the profile text is the proposer's entire input) ---")
    print(tables)
    reply = call_llm("proposer", PROPOSER_BLOCK, tables)
    return [{"seed": str(i.get("term", "")).lower().strip(), "hunch": str(i.get("hunch", ""))[:120]}
            for i in (reply or {}).get("terms", []) if str(i.get("term", "")).strip()][:10]


def admit_proposals(props):
    rows = []
    for p in props:
        term = p["seed"]
        if term in banned_words() or (set(term.split()) & banned_words()):
            rows.append({**p, "fate": "banned in memory"})
            continue
        total = corpus_member.get(term, 0)
        weeks_sup = len(set(cw.get(term, [])))
        marg = vec_margin(term)
        floor_total = 15 if term in LEXICON else 25
        if total < floor_total or weeks_sup < 15:
            rows.append({**p, "fate": f"died on support ({total} headlines, {weeks_sup} weeks; floor {floor_total})"})
        elif not (term in LEXICON or (marg is not None and marg > 0.25)):
            rows.append({**p, "fate": "died on vocabulary test (margin " + ("none" if marg is None else f"{marg:.2f}") + ")"})
        else:
```

```
            rows.append({**p, "fate": "admitted"})
    return rows


def build_roster(props=None):
    global LAST_ADMIT
    rows = [{"seed": r["seed"], "provenance": "mined", "gloss": "co-occurs with this company's mention headlines"} for r in
mined]
    LAST_ADMIT = []
    if props:
        ar = admit_proposals(props)
        LAST_ADMIT = ar
        display(pd.DataFrame(ar))
        for r in ar:
            if r["fate"] == "admitted":
                existing = next((m for m in rows if m["seed"] == r["seed"]), None)
                if existing:
                    existing["provenance"] = "both"
                else:
                    rows.append({"seed": r["seed"], "provenance": "proposed", "gloss": r["hunch"]})
    return rows


print("proposer runs at session 1 below; standing block, tables, and verbatim reply print there")
```

```
==========================================================================
L8.2 PROPOSER — before the corpus gets a say: which words' coverage plausibly transmits to AAPL? profile text only; no number
s, no dates; every proposal then passes the same deterministic gates as mined seeds
==========================================================================
proposer runs at session 1 below; standing block, tables, and verbatim reply print there
```

In [20]:

```
banner("L8.3 PLANNER — routing: one action per candidate; semantic drop reasons only; generalize=true writes a persistent
ban; escalate sends the analyzer for another run at the pre-registered lower floor")
PLANNER_BLOCK = """You plan research for one company. The roster lists candidate signal words, each with provenance (mined
from headlines naming the company, proposed from the company profile, or both) and a gloss. You see no numbers. For each
candidate, exactly one action:
drop — a phrase fragment, generic market-process language, or calendar language; one semantic reason. Set generalize true
ONLY for market-process, calendar, or wire-format language — words that could not be a real transmission channel for ANY
company (profit, outlook, plc, corp). Commodity, sector, and industry words are NOT generalizable — they transmit for some
companies even when not this one; when in doubt set false.
read — the news desk must inspect what the word actually captures; say what you suspect. At most 5 across the roster.
full — route to the statistics lane and the earnings desk.
Also return escalate: true at most once — it re-runs the miner at a lower admission floor with every dropped and banned
word excluded, surfacing replacement candidates; use it only if drops leave the roster thin.
Reply with JSON only: {"actions": [{"term": "...", "action": "drop|read|full", "reason": "...", "generalize": false}],
"escalate": false, "plan_summary": "..."}"""


def planner_tables(roster_rows, ticker=TICKER):
    lines = [f"COMPANY: {ticker}", f"CANDIDATE ROSTER ({len(roster_rows)}):"]
    lines += [f"- {r['seed']} [{r['provenance']}]: {r['gloss']}" for r in roster_rows]
    agent_bans = [b["word"] for b in memory["banned"] if b["origin"] != "deterministic"][-12:]
    if agent_bans:
        lines.append("WORDS ALREADY BANNED BY EARLIER SESSIONS OF THIS SYSTEM (persistent memory; never re-admit them): " +
", ".join(agent_bans))
    return "\n".join(lines)


def plan_research(roster_rows, ticker=TICKER):
    return call_llm("planner", PLANNER_BLOCK, planner_tables(roster_rows, ticker))


def dispatch_plan(plan, roster_rows):
    acts = {str(a.get("term", "")).lower().strip(): a for a in plan.get("actions", [])}
    for r in roster_rows:
        a = acts.get(r["seed"], {"action": "full", "reason": "no planner verdict; defaults to full"})
        r["action"] = str(a.get("action", "full"))
        r["reason"] = str(a.get("reason", ""))
        if r["action"] == "drop" and a.get("generalize"):
            write_ban(r["seed"], r["reason"])
    print(f"PLANNER plan_summary: {plan.get('plan_summary', '')}")
    print("dispatch: " + ", ".join(f"{r['seed']}->{r['action']}" for r in roster_rows))
    return roster_rows

print("planner runs at session 1 below, with its deterministic twin printed beside its choices")
```

```
==========================================================================
L8.3 PLANNER — routing: one action per candidate; semantic drop reasons only; generalize=true writes a persistent ban; escalat
e sends the analyzer for another run at the pre-registered lower floor
==========================================================================
planner runs at session 1 below, with its deterministic twin printed beside its choices
```

In [21]:

```
banner("L8.4 NEWS DESK (INVESTIGATOR) — term quality with a bounded evidence loop: the desk rules on each word, or asks for
more headlines first; its requests are the tool calls, chosen mid-session by the agent's own reply")
NEWS_BLOCK = """You inspect candidate signal words for one company. You initially get three headlines per word, sampled
across the decade (week offset relative to the corpus end; negative = earlier), each with a tone label from a financial
```

```
sentiment classifier. You may ask for more evidence before ruling: reply with {"requests": [{"term": "...", "after_week":
-200, "k": 6}]} and up to three requests per round — each fetches k more headlines for that term from weeks at or after
that offset. Request only when the samples leave the sense genuinely unclear; rule on the words you can already rule on.
When you rule, give for each word:
captures: one line on what the word actually measures across these headlines.
verdict — exactly one of:
clean: one sense, the intended thing.
diluted: the wanted sense dominates but other senses mix in — still usable, noisier.
contaminated: the unwanted sense is most of what weekly counts would track, or two senses fight and cannot be separated;
quote the proving headline in evidence.
Reply with JSON only: {"requests": [...]} while gathering, then {"verdicts": [{"term": "...", "captures": "...", "verdict":
"clean|diluted|contaminated", "evidence": "..."}]}"""


def news_desk(terms):
    lines = [f"COMPANY: {TICKER}"]
    for term in terms:
        lines.append(f"TERM: {term}")
        for s in tool_headline_sample(term, 3):
            lines.append(f"  week {s['week_offset']:>5} | {s['label']:>8} | {s['headline']}")
    print("--- NEWS DESK TABLES (verbatim — the initial evidence the desk rules on) ---")
    print("\n".join(lines))
    reply = {}
    for round_ in range(4):
        reply = call_llm("news", NEWS_BLOCK, "\n".join(lines)) or {}
        reqs = [r for r in (reply.get("requests") or []) if str(r.get("term", "")).strip()][:3]
        if not reqs or round_ == 3:
            return reply
        for rq in reqs:
            term = str(rq.get("term", "")).lower().strip()
            after = int(rq.get("after_week", -9999)) if str(rq.get("after_week", "-9999")).lstrip("-").isdigit() else -9999
            k = int(rq.get("k", 5)) if str(rq.get("k", "5")).isdigit() else 5
            k = min(max(k, 1), 8)
            NEWS_TOOL_CALLS.append((term, after, k))
            print(f"TOOL CALL: headline_sample(term='{term}', after_week={after}, k={k}) — the agent's reply chose this
call")
            lines.append(f"MORE EVIDENCE for '{term}' (samples at or after week {after}):")
            for s in tool_headline_sample(term, k, after_week=after):
                fetched = f"  week {s['week_offset']:>5} | {s['label']:>8} | {s['headline']}"
                lines.append(fetched)
                print(fetched)
    return reply


print("news desk runs on planner-assigned read terms at session 1; every evidence request prints as a tool call, with the
boundary audit after")
```

```
================================================================================
L8.4 NEWS DESK (INVESTIGATOR) — term quality with a bounded evidence loop: the desk rules on each word, or asks for more headl
ines first; its requests are the tool calls, chosen mid-session by the agent's own reply
================================================================================
news desk runs on planner-assigned read terms at session 1; every evidence request prints as a tool call, with the boundary au
dit after
```

In [22]:

```
banner("L8.5 EARNINGS DESK + MARKET LANE — transmission channels judged on the profile, recent filings, and the news desk's
captures; the statistics lane is deterministic math with pre-registered weights (no LLM)")
EARNINGS_BLOCK = """You judge transmission channels for one company. You receive the company profile, a list of recent SEC
filings (form and weeks-before-now), and each candidate signal word with the news desk's description of what its coverage
actually captures. For each candidate:
channel: revenue | cost | demand | financing | none — through which line of this company's business the captured coverage
transmits.
direction: helps | hurts | unclear.
mechanism: one line. For channel none, the mechanism line must say why it does not transmit to this company.
Judge the described coverage, not the bare word. Reply with JSON only: {"channels": [{"term": "...", "channel": "...",
"direction": "...", "mechanism": "..."}]}"""


def earnings_desk(terms, captures, now):
    lines = [f"COMPANY: {TICKER}", "PROFILE:", tool_company_profile()[:900], "RECENT SEC FILINGS:"]
    filings = tool_edgar_filings(now)
    lines += [("  " + f) for f in filings] or ["  none within the window"]
    lines.append("CANDIDATES (with what each one's coverage captures, per the news desk):")
    lines += [f"- {term}: {captures.get(term, '')}" for term in terms]
    print("--- EARNINGS DESK TABLES (verbatim — profile excerpt, filings recency, candidates with captures) ---")
    print("\n".join(lines))
    reply = call_llm("earnings", EARNINGS_BLOCK, "\n".join(lines))
    out = {}
    for c in (reply or {}).get("channels", []):
        out[str(c.get("term", "")).lower().strip()] = c
    return out


MARKET_W = {"abs_r": 0.45, "stable": 0.20, "r_exc": 0.25, "support": 0.10}
print("market lane weights, pre-registered before the sessions: " + json.dumps(MARKET_W))
```

```
def market_lane(terms, through):
    scr_ = stats_table(through=through)
    if scr_.empty:
        return {term: {} for term in terms}
    scr_ = scr_.set_index("feature")
    out = {}
    for term in terms:
        best = None
        for suf in ("share", "senti"):
            col = f"{term}|{suf}"
            if col in scr_.index and (best is None or abs(scr_.loc[col, "r"]) > abs(scr_.loc[best, "r"])):
                best = col
        if best is None:
            out[term] = {}
            continue
        row = scr_.loc[best]
        score = (MARKET_W["abs_r"] * min(abs(row["r"]), 0.3) / 0.3
                 + MARKET_W["stable"] * int(bool(row["stable"]))
                 + MARKET_W["r_exc"] * min(abs(row["r_exc"]), 0.3) / 0.3
                 + MARKET_W["support"] * min(row["support"], 300) / 300)
        out[term] = {"r": row["r"], "r_spy": row["r_spy"], "r_exc": row["r_exc"], "q": row["q"],
                     "market_score": round(float(score), 3),
                     "tracking": bool(abs(row["r_spy"]) >= 0.8 * abs(row["r"]) and abs(row["r_exc"]) < 0.05)}
    return out
```

```
==========================================================================
L8.5 EARNINGS DESK + MARKET LANE — transmission channels judged on the profile, recent filings, and the news desk's captures;
the statistics lane is deterministic math with pre-registered weights (no LLM)
==========================================================================
market lane weights, pre-registered before the sessions: {"abs_r": 0.45, "stable": 0.2, "r_exc": 0.25, "support": 0.1}
```

In [23]:

```
banner("L8.6 REFLECTION — after outcomes realize, the system writes its own lesson: one call over the glossed ledger; the
lesson joins memory and is injected into later sessions (learning across runs)")
REFLECTION_BLOCK = """You are the memory of an investment-research pipeline. You see this system's own prior calls for one
company, each already glossed by the pipeline relative to the call made: directional calls are marked correct or incorrect
on direction; abstained calls held no position, so the move printed beside them is context (a move sat out), never a gain,
loss, or error of the call. Write exactly one lesson of at most two sentences that a later decision for this company could
use. Judge directional calls on their direction; judge abstentions only on the context they were made on (the printed
coverage and market state), never on the move that followed. Use only the printed rows; quote numbers, never compute; no
advice beyond what the rows support. Do not merely restate a row — state the pattern across rows, or say the rows are too
few to support one.
Reply with JSON only: {"lesson": "..."}"""


def call_outcome_gloss(direction, fwd):
    """Render one outcome relative to the call: the pipeline, not the model, decides what a hold means."""
    if direction == "buy":
        return f"correct ({fwd:+.1%} realized, called up)" if fwd > 0 else f"incorrect ({fwd:+.1%} realized, called up)"
    if direction == "sell":
        return f"correct ({fwd:+.1%} realized, called down)" if fwd < 0 else f"incorrect ({fwd:+.1%} realized, called
down)"
    sat = "a down move sat out, not a loss" if fwd < 0 else "an up move sat out, not a gain"
    return f"abstained — no position held; the ticker moved {fwd:+.1%} that week ({sat})"


def call_context_gloss(w):
    """One deterministic line of the state a call was made on: coverage, tone, market."""
    n_, p_, g_ = bin_counts(w)
    cov = "quiet coverage" if n_ == 0 else (f"thin coverage ({n_} stories)" if n_ < 3 else f"active coverage ({n_}
stories)")
    tone = "tone undefined on fewer than 3 stories" if n_ < 3 else ("tone positive" if p_ > g_ else "tone negative" if g_ >
p_ else "tone mixed")
    m_ = float(wkpx["SPY_ret"].reindex(X0.index).loc[w])
    v_ = float(wkpx["SPY_vol"].reindex(X0.index).loc[w])
    return f"{cov}, {tone}, market {'up' if m_ > 0 else 'down'}, volatility {'elevated' if v_ > 0.03 else 'calm'}"


def ledger_row(ticker, session, direction, strength, fwd, w):
    return {"ticker": ticker, "session": session, "direction": direction, "strength": strength,
            "realized": fwd, "gloss": call_outcome_gloss(direction, fwd), "context": call_context_gloss(w)}


def ledger_lines(led):
    return [f"  session {r['session']}: {r['direction']} strength {r['strength']} -> {r['gloss']} (made on:
{r.get('context', '')})" for r in led]


def deterministic_summary(led):
    holds = [r for r in led if r["direction"] == "hold"]
    calls = [r for r in led if r["direction"] in ("buy", "sell")]
    ok = [r for r in calls if (r["direction"] == "buy") == (r["realized"] > 0)]
    dn = [r["realized"] for r in holds if r["realized"] < 0]
    up = [r["realized"] for r in holds if r["realized"] > 0]
    parts = [f"{len(holds)} abstentions — {len(dn)} sat out down weeks (mean {np.mean(dn):+.1%})" if dn else f"{len(holds)}
```

```
abstentions — none sat out a down week",
             f"{len(up)} sat out up weeks (mean {np.mean(up):+.1%})" if up else "none sat out an up week",
             f"{len(calls)} directional calls — {len(ok)} correct on direction"]
    return "; ".join(parts) + ". Quote these; do not recompute."


def reflect():
    led = [r for r in memory["ledger"] if r["ticker"] == TICKER]
    if not led:
        return None
    tables = ("PRIOR CALLS AND REALIZED OUTCOMES (pipeline-glossed, call-relative):\n"
              + "\n".join(ledger_lines(led))
              + "\nDETERMINISTIC SUMMARY (pipeline-computed): " + deterministic_summary(led))
    print('--- REFLECTION TABLES (verbatim — the lesson writer sees exactly this) ---')
    print(tables)
    reply = call_llm("reflection", REFLECTION_BLOCK, tables)
    return str((reply or {}).get("lesson", "")).strip() or None


print("reflection runs inside the sessions loop below, after each outcome realizes; the ledger rows it reads are glossed
here first")
```

```
================================================================================
L8.6 REFLECTION — after outcomes realize, the system writes its own lesson: one call over the glossed ledger; the lesson joins
memory and is injected into later sessions (learning across runs)
================================================================================
reflection runs inside the sessions loop below, after each outcome realizes; the ledger rows it reads are glossed here first
```

9. The sessions

The sessions run on the six test weeks with the most company coverage. Coverage is an input, so the choice uses no outcome information. Session 1 runs the proposer, the planner, the news desk, and the earnings desk, then freezes the feature roster. Every session prints its as-of tables, runs the advocate, the decision call, and the evaluator, and writes the realized outcome to the ledger after the week elapses.

All agent replies below are cached replays of the recorded run — the call log reports 0 live calls — and the printed tables are exactly what that run saw.

In [24]:

```
banner("L10 SESSIONS — each session's tables are computed as-of its week and printed in full; agents never see outcomes or
calendar dates; the ledger of realized prior calls and the system's own written lessons are injected into later sessions
(learning across runs)")
SYNTHESIS_BLOCK = """You are the decision layer of an investment-research pipeline for one company. Your job: call the
direction of this company's coming week — buy = you expect the coming week's close higher, sell = lower. hold is abstention
and is available only when the printed evidence genuinely balances; a pipeline that always abstains provides nothing, but
never manufacture conviction the tables do not show. strength grades your conviction: 1 = marginal, 2 = supported by the
tables, 3 = coherent across several features.
You receive the frozen verdict table (each surviving signal word with what its coverage captures, its transmission channel,
and its verbalized statistics), this week's semantic levels and binned news counts, the company's own coverage bin, the
market state, recent headlines, and this system's ledger of its own prior calls, their realized outcomes, and lessons it
has written for itself.
Keep every listed feature unless you give a per-feature removal reason. Then: direction = buy | sell | hold; strength = 1 |
2 | 3; evidence = the printed values your call rests on; strength_boundary = what evidence would move you one grade down;
rationale = at most two sentences. Where a feature's statistics conflict with what it captures, say so. A relation printed
as 'no clear historical relation' carries no directional weight, and a relation printed as not significant after multiple
testing may temper strength but never set direction: with no relation surviving multiple testing, the honest call is hold.
Address the devil's advocate brief: name its strongest item in advocate_response and state, on printed evidence, why your
call survives it, or concede to hold. The ledger is short — it may temper strength, never override evidence. Its rows
arrive pre-glossed by the pipeline, call-relative: for an abstained call the printed move is context (a move sat out),
never this system's realized gain or loss. Use nothing outside these tables; do no arithmetic.
Reply with JSON only: {"final_features": ["..."], "removals": [{"term": "...", "reason": "..."}], "advocate_response":
"...", "direction": "...", "strength": 1, "evidence": ["..."], "strength_boundary": "...", "rationale": "..."}"""

EVALUATOR_BLOCK = """You audit a decision against exactly the tables its author saw. Check both directions:
1. every feature the decision lists exists in the verdict table; 2. every verdict-table survivor is retained or has a
stated removal reason; 3. every cited value matches the printed tables; 4. no claim untraceable to the tables (outside
knowledge); 5. no arithmetic performed by the decision (numbers may be quoted, never computed).
Also return audit_note: one sentence stating what your acceptance or revision rests on.
Reply with JSON only: {"checks": {"listed_valid": true, "survivors_accounted": true, "citations_match": true,
"no_outside_knowledge": true, "no_arithmetic": true}, "violations": ["..."], "audit_note": "...", "verdict":
"accept|revise"}"""

ADVOCATE_BLOCK = """You are the devil's advocate in an investment-research pipeline. You receive exactly the tables the
decision layer will see, before it decides. Argue the strongest case for the OPPOSITE of the direction the tables lean
toward — if they lean constructive, argue the bear case; if defensive, the bull case; if they appear balanced, argue that
the balance is an artifact of sparse coverage. Use only the printed tables; quote values, never compute; at most four
sentences.
```

```
Also return counter_direction (buy, sell, or hold — the direction you would call from these tables) and strongest_item (the single printed item the decision layer is most likely to underweight).
Reply with JSON only: {"brief": "...", "counter_direction": "buy|sell|hold", "strongest_item": "..."}"""


def week_off(w):
    return -int((wk_index[-1] - w).n)


def verbal_rel(rr, unit):
    r_v, r_e, r_s = float(rr["r"]), float(rr["r_exc"]), float(rr["r_spy"])
    rel = (f"{unit} historically preceded weaker weeks" if r_v < -0.05
           else f"{unit} historically preceded stronger weeks" if r_v > 0.05
           else "no clear historical relation")
    spec = ("ticker-specific, more than the market" if abs(r_e) > 0.03 and abs(r_e) > abs(r_s)
            else "mostly mirrors the market" if abs(r_s) >= 0.8 * abs(r_v) and abs(r_e) < 0.05
            else "unclear specificity")
    q_v = float(rr.get("q", 1.0))
    sig = "survives multiple testing" if q_v <= 0.05 else "not significant after multiple testing"
    return f"{rel} (r={r_v:+.3f}, r_spy={r_s:+.3f}, r_exc={r_e:+.3f}); specificity: {spec}; significance: q={q_v:.2f} —
{sig}"


def bin_counts(w_):
    return int(hits.loc[w_]), int(pos.loc[w_]), int(neg.loc[w_])


def syn_tables(w, vt, include_memory=True):
    lines = [f"COMPANY: {TICKER} (week {week_off(w)} relative to corpus end)",
             "VERDICT TABLE (frozen at session 1):"]
    lines += vt["rows"] or ["  (no surviving concept features; the company bin and market state carry this decision)"]
    lines.append("THIS WEEK'S VALUES (pipeline-computed against each feature's own history up to this week — these features
are sparse: most weeks are zero):")
    for col in [f"{term}|share" for term in vt["survivors"]] + ["COMPANY|share"]:
        if col in X0.columns:
            hist = X0[col].loc[:w]
            nz = hist[hist > 0]
            now_v = float(hist.iloc[-1])
            if now_v == 0:
                lines.append(f"  {col}: quiet (zero coverage this week; active in {len(nz)} of {len(hist)} weeks to date)")
            else:
                pct = float((nz <= now_v).mean())
                lines.append(f"  {col}: now {now_v:+.4f} | percentile among its own active weeks {pct:.0%} | active in
{len(nz)} of {len(hist)} weeks | level: {'elevated' if pct >= 0.9 else 'typical'}")
    for term in vt["survivors"] + ["COMPANY"]:
        col = f"{term}|senti"
        if col not in X0.columns:
            continue
        if term == "COMPANY":
            n_, p_, g_ = bin_counts(w)
        else:
            rec_ = tw[term][w]
            n_, p_, g_ = rec_[0], rec_[1], rec_[2]
        if n_ == 0:
            lines.append(f"  {col}: no coverage this week — tone undefined")
        elif n_ < 3:
            lines.append(f"  {col}: tone uninformative ({n_} stories this week; {p_} positive, {g_} negative) — now
{float(X0[col].loc[w]):+.4f}")
        else:
            v_ = float(X0[col].loc[w])
            tone = "positive" if v_ > 0.05 else ("negative" if v_ < -0.05 else "mixed")
            lines.append(f"  {col}: tone {tone} ({p_} positive, {g_} negative of {n_} stories) — now {v_:+.4f}")
    lines.append("RECENT NEWS BINS (the company's own coverage, last 4 weekly bins):")
    pw = wk_index.get_loc(w)
    for j in range(max(0, pw - 3), pw + 1):
        ww = wk_index[j]
        n_, p_, g_ = bin_counts(ww)
        if n_ >= 3:
            lines.append(f"  week {week_off(ww):>4}: {n_} stories — {p_} positive, {g_} negative, {n_ - p_ - g_} neutral |
tone {float(X0['COMPANY|senti'].loc[ww]):+.2f}")
        elif n_:
            lines.append(f"  week {week_off(ww):>4}: {n_} stor{'y' if n_ == 1 else 'ies'} — {p_} positive, {g_} negative,
{n_ - p_ - g_} neutral | tone uninformative (fewer than 3 stories)")
        else:
            lines.append(f"  week {week_off(ww):>4}: no stories — tone undefined")
    lines.append("MARKET STATE THIS WEEK:")
    for c in ["SPY_ret", "SPY_vol", "dgs10_chg"]:
        v = float(wkpx[c].reindex(X0.index).loc[w])
        gloss = (" (an up week for the market)" if c == "SPY_ret" and v > 0
                 else " (a down week for the market)" if c == "SPY_ret" and v < 0
                 else " (elevated volatility)" if c == "SPY_vol" and v > 0.03
                 else " (calm volatility)" if c == "SPY_vol" else "")
        lines.append(f"  {c}: {v:+.4f}{gloss}")
    ret_a = wkpx[TICKER].pct_change().loc[:w].iloc[-52:]
    ret_s = wkpx["SPY"].pct_change().loc[:w].iloc[-52:]
```

```
    rho = float(np.corrcoef(ret_a, ret_s)[0, 1])
    tgloss = ("moves closely with the market" if rho >= 0.5 else "moves moderately with the market" if rho >= 0.3 else
"moves largely independently of the market")
    lines.append(f"  ticker-market link: {TICKER}'s weekly returns correlate {rho:+.2f} with SPY over the trailing 52 weeks
— {tgloss}")
    dfr = ticker_frame()
    m_asof = (dfr.index <= w) & dfr["fwd"].notna() & dfr["SPY_ret"].notna()
    if int(m_asof.sum()) > 50:
        r_mkt = float(np.corrcoef(dfr.loc[m_asof, "SPY_ret"], dfr.loc[m_asof, "fwd"])[0, 1])
        mrel = ("down market weeks historically preceded weaker weeks for this ticker" if r_mkt > 0.03
                else "down market weeks historically preceded stronger weeks for this ticker" if r_mkt < -0.03
                else "market weeks had no clear relation to this ticker's next week")
        lines.append(f"  market-relation (window to date): {mrel} (r={r_mkt:+.3f}) — not significant after multiple
testing")
    rec = sorted([i for i in np.where(m_mask)[0] if 0 <= (w - week_of[i]).n <= 4], key=lambda i: week_of[i])[-5:]
    lines.append("RECENT HEADLINES NAMING THIS COMPANY (last month, with tone labels):")
    if rec:
        for i in rec:
            lines.append(f"  [{labels_all[i]:>8}] {clean['headline'].iloc[i][:90]}")
    else:
        lines.append("  (no company-naming headlines in the last month)")
    if include_memory:
        led = [r for r in memory["ledger"] if r["ticker"] == TICKER]
        lines.append("MEMORY LEDGER (this system's prior calls for this company, pipeline-glossed call-relative
outcomes):")
        if led:
            lines += ledger_lines(led)
        else:
            lines.append("  (empty — first call for this company)")
        lines.append("LESSONS THIS SYSTEM WROTE FROM ITS OWN REALIZED OUTCOMES:")
        if memory.get("lessons"):
            lines += [f"  - {l_[:220]}" for l_ in memory["lessons"][-3:]]
        else:
            lines.append("  (none yet)")
    return "\n".join(lines)


def citation_twin(syn, tables_text):
    ev = syn.get("evidence", [])
    cited = re.findall(r"[-+]?\d+\.\d+", " ".join(ev) if isinstance(ev, list) else str(ev))
    norm = re.sub(r"[-+]?\d+\.\d+", lambda m: f"{float(m.group()):+.4f}", tables_text)
    return sorted({n for n in cited if f"{float(n):+.4f}" not in norm})


te_weeks = wk_index[TE]
if DEMO_GRID == "active":
    cov = company_series["COMPANY|share"].loc[te_weeks]
    aapl_sessions = sorted(cov.sort_values(ascending=False).index[:N_SESSIONS])
    print(f"demonstration grid: the {N_SESSIONS} test weeks with the most company-naming coverage")
else:
    grid = te_weeks[np.unique(np.round(np.linspace(0, len(te_weeks) - 1, 36)).astype(int))]
    aapl_sessions = [grid[j * 6] for j in range(N_SESSIONS)]
    print(f"even grid: {N_SESSIONS} sessions spread across the {int(TE.sum())} test weeks")

frozen, session_log, session_failures, fate_log = None, [], [], []
memory["ledger"] = []
memory["planner_notes"] = []
memory["lessons"] = []
MEMORY_FILE.write_text(json.dumps(memory, indent=1), encoding="utf-8")

# every table below is computed as-of this week; agents see week offsets, never calendar dates
for si, w in enumerate(aapl_sessions):
    print(f"\n--- SESSION {si + 1}/{N_SESSIONS}: {TICKER}, week {week_off(w)} ---")
    try:
        if frozen is None:
            props = propose_terms() if GEMINI_API_KEY else []
            ros = build_roster(props)
            for p in (LAST_ADMIT or []):
                if p["fate"] != "admitted":
                    fate_log.append({"term": p["seed"], "provenance": "proposed", "planner": None,
                                     "planner_note": "", "news": None, "channel": None,
                                     "fate": "died at admission (" + p["fate"] + ")"})
            plan_ = plan_research(ros) if GEMINI_API_KEY and ros else None
            nv = {}
            if plan_:
                ros = dispatch_plan(plan_, ros)
                hits_map = {m["seed"]: m["mention_hits"] for m in mined}
                twin = sorted(ros, key=lambda r: -hits_map.get(r["seed"], 0))[:5]
                print(f"twin (deterministic): no drops, read = top-5 by mention hits {[r['seed'] for r in twin]}")
                if plan_.get("escalate"):
                    print("TOOL CALL: remine(level='escalated', bans=memory + planner strikes)")
                    repl = remine_escalated({r["seed"] for r in ros} | banned_words())
                    print("replacements: " + (", ".join(r["seed"] for r in repl) or "none surfaced"))
                    if repl:
                        ros += dispatch_plan(plan_research(repl), repl)
```

```
            for r in ros:
                fate_log.append({"term": r["seed"], "provenance": r["provenance"], "planner": r.get("action", "full"),
                                 "planner_note": r.get("reason", ""), "news": None, "channel": None, "fate": None})
            read_ = [r["seed"] for r in ros if r.get("action") == "read"]
            if GEMINI_API_KEY and read_:
                for v in (news_desk(read_) or {}).get("verdicts", []):
                    nv[str(v.get("term", "")).lower().strip()] = v
                print(f"news desk twin: all {len(read_)} words already passed the deterministic wire and vocabulary gates")
                cleared = [x for x in read_ if nv.get(x, {}).get("verdict") == "clean"]
                if not cleared:
                    print("boundary audit: no read term was verdicted clean this run")
                if cleared:
                    t0 = cleared[0]
                    print(f"boundary audit on cleared term '{t0}': five other containing headlines")
                    for s in tool_headline_sample(t0, 5):
                        print(f"   week {s['week_offset']:>5} | {s['label']:>8} | {s['headline']}")
            cand = [r["seed"] for r in ros if r.get("action") != "drop"]
            gloss = {r["seed"]: r.get("gloss", "") for r in ros}
            caps = {term: nv.get(term, {}).get("captures") or gloss.get(term, "") for term in cand}
            now_ts = w.to_timestamp(how="end").normalize()
            ec = earnings_desk(cand, caps, now_ts) if GEMINI_API_KEY and cand else {}
            surv = [term for term in cand
                    if not (nv.get(term, {}).get("verdict") == "contaminated"
                            and (ec.get(term, {}) or {}).get("channel") in (None, "none"))]
            for fr in fate_log:
                if fr["fate"] is not None or fr["planner"] is None:
                    continue
                term = fr["term"]
                if fr["planner"] == "drop":
                    fr["fate"] = f"dropped by planner: {fr['planner_note'][:60]}"
                    continue
                fr["news"] = nv.get(term, {}).get("verdict")
                fr["channel"] = (ec.get(term, {}) or {}).get("channel")
                if fr["news"] == "contaminated" and fr["channel"] in (None, "none"):
                    fr["fate"] = f"struck on two axes: news contaminated + earnings channel {fr['channel'] or 'unknown'}"
                elif fr["news"] == "contaminated":
                    fr["fate"] = "kept, flagged: contaminated coverage but a live transmission channel"
                else:
                    fr["fate"] = "frozen survivor"
            w_prior = wk_index[wk_index.get_loc(w) - 1]
            ml = market_lane(surv, w_prior)
            scr_now = stats_table(through=w_prior)
            if not scr_now.empty:
                scr_now = scr_now.set_index("feature")
            rows_, cov_rel, senti_rel, senti_r = [], {}, {}, {}
            company_tone_r = None
            for term in surv:
                m, ch = ml.get(term, {}), ec.get(term, {})
                flag = " [flagged: diluted]" if nv.get(term, {}).get("verdict") == "diluted" else ""
                rows_.append(f"- {term}{flag}: captures: {caps[term]} | channel: {ch.get('channel', '?')}
{ch.get('direction', '?')} — {ch.get('mechanism', '')}")
                if m:
                    cov_rel[term] = verbal_rel(m, "higher coverage")
                    rows_.append(f"  coverage relation: {cov_rel[term]}")
                if f"{term}|senti" in scr_now.index:
                    senti_rel[term] = verbal_rel(scr_now.loc[f"{term}|senti"], "more positive tone")
                    senti_r[term] = float(scr_now.loc[f"{term}|senti"]["r"])
                    rows_.append(f"  tone relation: {senti_rel[term]}")
            if not scr_now.empty:
                for c, gloss_ in [("COMPANY|share", "the volume of headlines naming the company"),
                                  ("COMPANY|senti", "the tone of headlines naming the company")]:
                    if c in scr_now.index:
                        rr = scr_now.loc[c]
                        if c == "COMPANY|senti":
                            company_tone_r = float(rr["r"])
                        unit = "more coverage" if c.endswith("share") else "more positive tone"
                        rows_.append(f"- COMPANY {'coverage' if c.endswith('share') else 'tone'}: captures: {gloss_} |
{verbal_rel(rr, unit)}")
            frozen = {"survivors": surv, "rows": rows_, "market": ml, "cov_rel": cov_rel,
                      "senti_rel": senti_rel, "senti_r": senti_r, "company_tone_r": company_tone_r,
                      "caps": {t: caps[t] for t in surv},
                      "chan": {t: ec.get(t, {}) for t in surv}}
            memory["planner_notes"].append(f"{TICKER}: {(plan_ or {}).get('plan_summary', 'no plan (no API key)')[:200]}")
            MEMORY_FILE.write_text(json.dumps(memory, indent=1), encoding="utf-8")
            print("CANDIDATE FATE (this ticker's full candidate stream):")
            for fr in fate_log:
                print(f"   {fr['term']:<16} [{fr['provenance']:<14}] -> {fr['fate']}")
            print(f"FROZEN at session 1: {len(surv)} survivors — {', '.join(surv) or 'none; company bin + market state
only'}")
        n_before = len([r for r in memory["ledger"] if r["ticker"] == TICKER])
        for i, prev in enumerate(session_log):
            if i >= n_before:
                memory["ledger"].append(ledger_row(TICKER, i + 1, prev["direction"], prev["strength"], prev["fwd"],
prev["week"]))
        MEMORY_FILE.write_text(json.dumps(memory, indent=1), encoding="utf-8")
```

```
        if GEMINI_API_KEY and len(memory["ledger"]) > n_before:
            les = reflect()
            if les:
                memory["lessons"].append(les)
                MEMORY_FILE.write_text(json.dumps(memory, indent=1), encoding="utf-8")
                print(f"MEMORY WRITE (lesson): {les}")
        if not GEMINI_API_KEY:
            print("synthesis/evaluator SKIPPED (no API key)")
            continue
        tbl = syn_tables(w, frozen)
        print("--- SESSION TABLES (verbatim — the synthesis and the evaluator each see exactly this text) ---")
        print(tbl)
        adv = call_llm("advocate", ADVOCATE_BLOCK, tbl) or {}
        brief = str(adv.get("brief", "")).strip()
        adv_dir = str(adv.get("counter_direction", "")).lower().strip() or None
        strongest = str(adv.get("strongest_item", "")).strip()
        adv_missing = citation_twin({"evidence": [brief]}, tbl) if brief else []
        if adv_missing:
            ADVOCATE_CITE_MISSES.append((si + 1, adv_missing))
            print(f"ADVOCATE TWIN: brief cites numbers absent from the tables: {adv_missing}")
        if brief:
            tbl = tbl + "\nDEVIL'S ADVOCATE BRIEF (argue with this where your evidence is stronger):\n" + brief
            if strongest:
                tbl += f"\nADVOCATE'S STRONGEST ITEM: {strongest}"
        syn = call_llm("synthesis", SYNTHESIS_BLOCK, tbl)
        if not syn:
            continue
        ev = call_llm("evaluator", EVALUATOR_BLOCK, tbl + "\n\nDECISION UNDER AUDIT:\n" + json.dumps(syn)) or {}
        EVAL_LOG.append({"session": si + 1, "verdict": ev.get("verdict"),
                         "violations": ev.get("violations", []) if isinstance(ev.get("violations"), list) else [],
                         "note": ev.get("audit_note", "")})
        if ev.get("verdict") == "revise":
            print("EVALUATOR: revise — one resubmission follows")
            syn2 = call_llm("synthesis", SYNTHESIS_BLOCK,
                            tbl + "\n\nEVALUATOR FEEDBACK (one revision allowed, address the violations):\n" +
json.dumps(ev))
            if syn2 and syn2.get("direction"):
                syn = syn2
        missing = citation_twin(syn, tbl)
        if missing:
            TWIN_MISSES.append((si + 1, missing))
            print(f"MECHANICAL TWIN: cited numbers absent from the tables: {missing}")
        direction = str(syn.get("direction", "hold")).lower()
        strength = syn.get("strength", 1)
        strength = int(strength) if str(strength).isdigit() and 1 <= int(strength) <= 3 else 1
        if strongest:
            adv_toks = set(re.findall(r"[a-z]{4,}", strongest.lower()))
            resp_toks = set(re.findall(r"[a-z]{4,}", str(syn.get("advocate_response", "")).lower()))
            if len(adv_toks & resp_toks) < 2:
                strength = 1
                ADVOCATE_MISSES.append(si + 1)
                print("ADVOCATE UNADDRESSED — strength capped by contract (the reply must engage the advocate's strongest
item)")
        reply_text = json.dumps(syn).lower()
        tone_states = []
        for term in frozen["survivors"]:
            rec_ = tw[term][w]
            if rec_[0] >= 3 and term in frozen.get("senti_r", {}):
                tone_states.append((term, float(X0[f"{term}|senti"].loc[w]), frozen["senti_r"][term]))
        n_c, _, _ = bin_counts(w)
        if n_c >= 3 and frozen.get("company_tone_r") is not None:
            tone_states.append(("COMPANY tone", float(X0["COMPANY|senti"].loc[w]), frozen["company_tone_r"]))
        for tname, v_now, r_rel in tone_states:
            if abs(v_now) <= 0.05 or direction not in ("buy", "sell"):
                continue
            implied = "buy" if v_now * r_rel > 0 else "sell"
            if direction != implied and f"r={r_rel:+.3f}" in reply_text:
                SIGN_FLAGS.append((si + 1, tname))
                print(f"SIGN TWIN: decision '{direction}' contradicts the sign of the cited {tname} relation (r=
{r_rel:+.3f} with this week's tone {v_now:+.2f} implies '{implied}')")
        fwd_realized = float(ticker_frame()["fwd"].loc[w])
        session_log.append({"week": w, "direction": direction, "strength": strength,
                            "fwd": fwd_realized, "syn": syn, "vt": frozen, "advocate": adv_dir})
        print(f"DECISION: {direction} strength {strength} | realized next week {fwd_realized:+.1%} | features
{len(syn.get('final_features', []))}")
    except Exception as exc:
        session_failures.append((str(w), f"{type(exc).__name__}: {exc}"))
        print(f"SESSION FAILED ({type(exc).__name__}): {exc}")

for i, prev in enumerate(session_log):
    if i >= len([r for r in memory["ledger"] if r["ticker"] == TICKER]):
        memory["ledger"].append(ledger_row(TICKER, i + 1, prev["direction"], prev["strength"], prev["fwd"], prev["week"]))
MEMORY_FILE.write_text(json.dumps(memory, indent=1), encoding="utf-8")
if GEMINI_API_KEY and session_log:
    les = reflect()
```

```
    if les:
        memory["lessons"].append(les)
        MEMORY_FILE.write_text(json.dumps(memory, indent=1), encoding="utf-8")
        print(f"MEMORY WRITE (lesson, final): {les}")

print(f"\nsessions complete: {len(session_log)} decisions logged | memory: {len(memory['ledger'])} realized ledger rows,
{len(memory['banned'])} bans ({sum(1 for b in memory['banned'] if b['origin'] != 'deterministic')} added by agents),
{len(memory.get('lessons', []))} self-written lessons")
if session_failures:
    for w_, err in session_failures:
        print(f"  failed session {w_}: {err}")
```

```
================================================================================
L10 SESSIONS — each session's tables are computed as-of its week and printed in full; agents never see outcomes or calendar da
tes; the ledger of realized prior calls and the system's own written lessons are injected into later sessions (learning across
runs)
================================================================================
demonstration grid: the 6 test weeks with the most company-naming coverage

--- SESSION 1/6: AAPL, week -77 ---
```

--- PROPOSER TABLES (verbatim — the profile text is the proposer's entire input) ---
COMPANY PROFILE:
Apple Inc. Technology Consumer Electronics Apple Inc. designs, manufactures, and markets smartphones, personal computers, tablets, wearables, and accessories worldwide. The company offers iPhone, a line of smartphones; Mac, a line of personal computers; iPad, a line of multi-purpose tablets; and wearables, home, and accessories comprising AirPods, Apple Vision Pro, Apple TV, Apple Watch, Beats products, and HomePod, as well as Apple branded and third-party accessories. It also provides AppleCare support and cloud services; and operates various platforms, including the App Store that allows customers to discover and download applications and digital content, such as books, music, video, games, and podcasts, as well as advertising services include third-party licensing arrangements and its own advertising platforms. In addition, the company offers various subscription-based services, such as Apple Arcade, a game subscription service; Apple Fitness+, a personalized fitness service; Apple Music, which offers users a curated listening experience with on-demand radio stations; Apple News+, a subscription news and magazine service; Apple TV, which offers original content and live sports; Apple Card, a co-branded credit card; and Apple Pay, a cashless payment service, as well as licenses its intellectual property. The company serves consumers, and small and mid-sized businesses; and the education, enterprise, and government markets. It distributes third-party applications for its products through the App Store. The company also sells its products through its retail and online stores, and direct sales force; and third-party cellular network carriers and resellers. The company was formerly known as Apple Computer, Inc. and changed its name to Apple Inc. in January 2007. Apple Inc
--- STANDING BLOCK: proposer (printed once; every proposer prompt = this block + that session's tables) ---
You are the concept proposer in an investment-research pipeline. You receive one company's self-description and nothing else — no market data, no news, no numbers. Name at most 10 terms whose news coverage would plausibly transmit to this company's business through a channel it actually earns or spends through. Strongly prefer single plain words a financial newswire headline would use (chip, battery, retail, refinery); use a two-word term only for fixed phrases (supply chain, crude oil). For each term give a one-line hunch on the expected transmission. Prefer specific things (products, inputs, demand drivers, regulations it faces); never market-process words (profit, outlook, ratings, sector, prices) — those are never admitted downstream. Reply with JSON only: {"terms": [{"term": "...", "hunch": "..."}]}
--- proposer REPLY ---

```json
{
  "terms": [
    {
      "term": "chip",
      "hunch": "Changes in semiconductor availability and component costs directly impact hardware manufacturing margins for iPhones and Macs."
    },
    {
      "term": "battery",
      "hunch": "Material costs and regulatory requirements regarding energy storage directly affect portable devices and wearables."
    },
    {
      "term": "tariff",
      "hunch": "Import duties and trade policy shifts alter the cost of goods manufactured overseas and imported for major consumer electronics."
    },
    {
      "term": "antitrust",
      "hunch": "Regulatory enforcement and legal challenges target App Store commission structures and service revenue models."
    },
    {
      "term": "advertising",
      "hunch": "Shifts in digital privacy rules and ad spend directly influence platform and licensing revenue streams."
    },
    {
      "term": "retail",
      "hunch": "Consumer spending trends and foot traffic fluctuations impact direct-to-consumer sales performance."
    },
    {
      "term": "supply chain",
      "hunch": "Logistics disruptions or manufacturing bottlenecks in assembly regions delay product availability and fulfillment."
    },
    {
      "term": "music",
      "hunch": "Licensing fees and digital content consumption trends dictate operational expenses and subscriber acquisition for media services."
    },
    {
      "term": "gaming",
      "hunch": "Content acquisition costs and user engagement levels drive performance in subscription gaming and app marketplace fees."
    },
    {
      "term": "credit",
      "hunch": "Default rates and lending partner terms affect the economics of co-branded financial services and payment processing."
    }
  ]
}
```

	seed	hunch	fate
0	chip	Changes in semiconductor availability and component costs directly impact hardware man...	died on vocabulary test (margin 0.24)
1	battery	Material costs and regulatory requirements regarding energy storage directly affect po...	died on support (19 headlines, 15 weeks; floor 25)
2	tariff	Import duties and trade policy shifts alter the cost of goods manufactured overseas an...	died on vocabulary test (margin -0.22)
3	antitrust	Regulatory enforcement and legal challenges target App Store commission structures and...	died on support (2 headlines, 2 weeks; floor 25)
4	advertising	Shifts in digital privacy rules and ad spend directly influence platform and licensing...	admitted
5	retail	Consumer spending trends and foot traffic fluctuations impact direct-to-consumer sales...	died on vocabulary test (margin -0.29)
6	supply chain	Logistics disruptions or manufacturing bottlenecks in assembly regions delay product a...	admitted
7	music	Licensing fees and digital content consumption trends dictate operational expenses and...	died on support (19 headlines, 18 weeks; floor 25)
8	gaming	Content acquisition costs and user engagement levels drive performance in subscription...	admitted
9	credit	Default rates and lending partner terms affect the economics of co-branded financial s...	died on vocabulary test (margin -0.19)

--- STANDING BLOCK: planner (printed once; every planner prompt = this block + that session's tables) ---
You plan research for one company. The roster lists candidate signal words, each with provenance (mined from headlines naming the company, proposed from the company profile, or both) and a gloss. You see no numbers. For each candidate, exactly one action:
drop — a phrase fragment, generic market-process language, or calendar language; one semantic reason. Set generalize true ONLY for market-process, calendar, or wire-format language — words that could not be a real transmission channel for ANY company (profit, outlook, plc, corp). Commodity, sector, and industry words are NOT generalizable — they transmit for some companies even when not this one; when in doubt set false.
read — the news desk must inspect what the word actually captures; say what you suspect. At most 5 across the roster.
full — route to the statistics lane and the earnings desk.
Also return escalate: true at most once — it re-runs the miner at a lower admission floor with every dropped and banned word excluded, surfacing replacement candidates; use it only if drops leave the roster thin.
Reply with JSON only: {"actions": [{"term": "...", "action": "drop|read|full", "reason": "...", "generalize": false}], "escalate": false, "plan_summary": "..."}
--- planner REPLY ---
```json
{
  "actions": [
    {
      "term": "tablet",
      "action": "full",
      "reason": "Core hardware product line directly impacting revenue and shipment statistics.",
      "generalize": false
    },
    {
      "term": "holiday",
      "action": "drop",
      "reason": "Calendar language representing seasonal shopping periods.",
      "generalize": true
    },
    {
      "term": "energy",
      "action": "drop",
      "reason": "Generic sector or commodity-related term not specific to company operations.",
      "generalize": false
    },
    {
      "term": "dip",
      "action": "drop",
      "reason": "Generic market-process language describing price movements.",
      "generalize": true
    },
    {
      "term": "watch",
      "action": "full",
      "reason": "Core wearable device line driving ecosystem and hardware revenue.",
      "generalize": false
    },
    {
      "term": "advertising",
      "action": "read",
      "reason": "Suspect captures digital services revenue growth and privacy policy impacts.",
      "generalize": false
    },
    {
      "term": "supply chain",
      "action": "read",
      "reason": "Suspect captures manufacturing bottlenecks, logistics costs, and component availability.",
      "generalize": false
    },
    {
      "term": "gaming",
      "action": "read",
      "reason": "Suspect captures App Store commission trends and subscription gaming performance.",
      "generalize": false
    }
  ],
  "escalate": false,
  "plan_summary": "Processed 8 candidate terms for AAPL, routing core hardware lines to full analysis, evaluating structural business drivers with the news desk, and dropping calendar/market noise."
}
```
MEMORY WRITE: 'holiday' banned for all later sessions — Calendar language representing seasonal shopping periods.
MEMORY WRITE: 'dip' banned for all later sessions — Generic market-process language describing price movements.
PLANNER plan_summary: Processed 8 candidate terms for AAPL, routing core hardware lines to full analysis, evaluating structural business drivers with the news desk, and dropping calendar/market noise.
dispatch: tablet->full, holiday->drop, energy->drop, dip->drop, watch->full, advertising->read, supply chain->read, gaming->read
twin (deterministic): no drops, read = top-5 by mention hits ['tablet', 'holiday', 'energy', 'dip', 'watch']
--- NEWS DESK TABLES (verbatim — the initial evidence the desk rules on) ---
COMPANY: AAPL
TERM: advertising
  week   -31 | neutral | Britain's Guardian's loss deepens on advertising slump
  week   -74 | neutral | Advertising and Lead-Generation Firm The 800 Exchange Sees Revenue Double Amid Rapid...
  week   -74 | neutral | Top U.S. Advertising Agencies Adopt Transis, Industry's First Digital Media Buying Automation Softwa

```
re
TERM: supply chain
  week   -72 |  neutral | UPDATE 1-Fujitsu, Boeing in aircraft supply chain deal - Nikkei
  week   -74 |  neutral | Businesses that scaled back supply during recession may be caught off guard as economy recovers warn
s economist, insurer and supply chain professionals
  week   -70 |  neutral | Japan could cause auto supply chain rethink: expert
TERM: gaming
  week   -69 |  neutral | Gaming; Win or Lose? - Equity Research on Pandora Media Inc. and Shanda Interactive Entertainment Li
mited
  week   -40 |  neutral | Gaming and Leisure Raises Bid for Pinnacle’s Real Estate
  week   -50 | positive | CORRECTED-UPDATE 1-Genting to spend $3-4 bln in Las Vegas in push for U.S. gaming empire
--- STANDING BLOCK: news (printed once; every news prompt = this block + that session's tables) ---
You inspect candidate signal words for one company. You initially get three headlines per word, sampled across the decade (wee
k offset relative to the corpus end; negative = earlier), each with a tone label from a financial sentiment classifier. You ma
y ask for more evidence before ruling: reply with {"requests": [{"term": "...", "after_week": -200, "k": 6}]} and up to three
requests per round — each fetches k more headlines for that term from weeks at or after that offset. Request only when the sam
ples leave the sense genuinely unclear; rule on the words you can already rule on. When you rule, give for each word:
captures: one line on what the word actually measures across these headlines.
verdict — exactly one of:
clean: one sense, the intended thing.
diluted: the wanted sense dominates but other senses mix in — still usable, noisier.
contaminated: the unwanted sense is most of what weekly counts would track, or two senses fight and cannot be separated; quote
the proving headline in evidence.
Reply with JSON only: {"requests": [...]} while gathering, then {"verdicts": [{"term": "...", "captures": "...", "verdict": "c
lean|diluted|contaminated", "evidence": "..."}]}
--- news REPLY ---
{"requests": [{"term": "advertising", "after_week": -500, "k": 6}, {"term": "supply chain", "after_week": -500, "k": 6}, {"ter
m": "gaming", "after_week": -500, "k": 6}]}
TOOL CALL: headline_sample(term='advertising', after_week=-500, k=6) — the agent's reply chose this call
  week   -67 |  neutral | PROFNET EXPERT ALERTS: Facebook Advertising / IRS Tax Issues / Google Wallet
  week   -37 |  neutral | Advertising Week 2015 and The Nasdaq Disruption Series to Ring The Nasdaq Stock Market Closing Bell
  week   -18 |  neutral | Orexigen to Use Some Bankruptcy Financing on Advertising
  week   -64 |  neutral | FrogAds.com Combines Online Auctioning and Advertising, Making it Today's Most Interactive Online Ma
rket
  week   -60 |  neutral | Sanoma cuts 2012 outlook on weak advertising markets
  week   -40 | positive | BRIEF-Cofina sees recovery of revenues from advertising in 2015
TOOL CALL: headline_sample(term='supply chain', after_week=-500, k=6) — the agent's reply chose this call
  week   -33 | positive | Wal-Mart aims to switch to cage-free egg supply chain by 2025
  week   -67 |  neutral | Toyota aims for quake-proof supply chain
  week   -33 | positive | UPDATE 2-Wal-Mart to switch to cage-free egg supply chain by 2025
  week   -68 |  neutral | CN and China's CNBM Forest Products Trading Ltd. strike deal for B.C. lumber export supply chain anc
hored on Vancouver transload facility
  week   -34 |  neutral | Companies have environmental "blind spot" in supply chains -study
  week   -39 |  neutral | BRIEF-Yinyi Real Estate plans to boost capital of a supply chain management firm by 200 mln yuan
TOOL CALL: headline_sample(term='gaming', after_week=-500, k=6) — the agent's reply chose this call
  week   -41 |  neutral | RPT-Japan's Nintendo, DeNA announce gaming, capital tie-up
  week   -45 |  neutral | Noteholders say Caesars fraudulently transferred gaming assets
  week   -59 |  neutral | U.S. Gaming Sector on a Roll
  week   -55 |  neutral | Education, gaming, toy companies to pitch for seed funding at ShowStoppers Launch.it during Sandbox
Summit @ MIT
  week   -49 |  neutral | REGIONAL ECONOMIC MODELS INC., SPECTRUM GAMING GROUP FORM GAMING IMPACTS TASK FORCE
  week   -62 |  neutral | TEXT-S&P; says new gaming supply may test Asia-Pacific gaming firms
--- news REPLY ---
{"verdicts": [{"term": "advertising", "captures": "general macroeconomic advertising markets and third-party ad agencies/spend
ing rather than Apple's own ad business.", "verdict": "contaminated", "evidence": "Britain's Guardian's loss deepens on advert
ising slump"}, {"term": "supply chain", "captures": "general macroeconomic supply chain disruptions, logistics, and other comp
anies' supply chain deals.", "verdict": "contaminated", "evidence": "UPDATE 1-Fujitsu, Boeing in aircraft supply chain deal -
Nikkei"}, {"term": "gaming", "captures": "casino gambling, general video game industry news, and other companies' gaming secto
r developments.", "verdict": "contaminated", "evidence": "Gaming and Leisure Raises Bid for Pinnacle\u2019s Real Estate"}]}
news desk twin: all 3 words already passed the deterministic wire and vocabulary gates
boundary audit: no read term was verdicted clean this run
--- EARNINGS DESK TABLES (verbatim — profile excerpt, filings recency, candidates with captures) ---
COMPANY: AAPL
PROFILE:
Apple Inc. Technology Consumer Electronics Apple Inc. designs, manufactures, and markets smartphones, personal computers, tabl
ets, wearables, and accessories worldwide. The company offers iPhone, a line of smartphones; Mac, a line of personal computer
s; iPad, a line of multi-purpose tablets; and wearables, home, and accessories comprising AirPods, Apple Vision Pro, Apple TV,
Apple Watch, Beats products, and HomePod, as well as Apple branded and third-party accessories. It also provides AppleCare sup
port and cloud services; and operates various platforms, including the App Store that allows customers to discover and downloa
d applications and digital content, such as books, music, video, games, and podcasts, as well as advertising services include
third-party licensing arrangements and its own advertising platforms. In addition, the company offers various subscription-bas
ed services, such
RECENT SEC FILINGS:
  8-K (4 weeks before now)
  8-K (8 weeks before now)
  10-Q (9 weeks before now)
  8-K (9 weeks before now)
  8-K (13 weeks before now)
  10-K (21 weeks before now)
  8-K (22 weeks before now)
  10-Q (35 weeks before now)
  8-K (35 weeks before now)
  8-K (47 weeks before now)
  10-Q (48 weeks before now)
  8-K (48 weeks before now)
```

CANDIDATES (with what each one's coverage captures, per the news desk):
- tablet: co-occurs with this company's mention headlines
- watch: co-occurs with this company's mention headlines
- advertising: general macroeconomic advertising markets and third-party ad agencies/spending rather than Apple's own ad business.
- supply chain: general macroeconomic supply chain disruptions, logistics, and other companies' supply chain deals.
- gaming: casino gambling, general video game industry news, and other companies' gaming sector developments.
--- STANDING BLOCK: earnings (printed once; every earnings prompt = this block + that session's tables) ---
You judge transmission channels for one company. You receive the company profile, a list of recent SEC filings (form and weeks-before-now), and each candidate signal word with the news desk's description of what its coverage actually captures. For each candidate:
channel: revenue | cost | demand | financing | none — through which line of this company's business the captured coverage transmits.
direction: helps | hurts | unclear.
mechanism: one line. For channel none, the mechanism line must say why it does not transmit to this company.
Judge the described coverage, not the bare word. Reply with JSON only: {"channels": [{"term": "...", "channel": "...", "direction": "...", "mechanism": "..."}]}
--- earnings REPLY ---
```json
{"channels": [{"term": "tablet", "channel": "revenue", "direction": "helps", "mechanism": "Directly relates to sales of the company's iPad line of multi-purpose tablets."}, {"term": "watch", "channel": "revenue", "direction": "helps", "mechanism": "Directly relates to sales of the company's Apple Watch wearables."}, {"term": "advertising", "channel": "none", "direction": "unclear", "mechanism": "Does not transmit to Apple because the coverage captures general macroeconomic advertising markets and third-party agencies rather than Apple's own ad business."}, {"term": "supply chain", "channel": "none", "direction": "unclear", "mechanism": "Does not transmit to Apple because the coverage captures general macroeconomic logistics and other companies' supply chain deals rather than Apple's specific operations."}, {"term": "gaming", "channel": "none", "direction": "unclear", "mechanism": "Does not transmit to Apple because the coverage captures casino gambling and general video game industry news rather than Apple's App Store gaming ecosystem."}]}
```

```
CANDIDATE FATE (this ticker's full candidate stream):
   chip             [proposed      ] -> died at admission (died on vocabulary test (margin 0.24))
   battery          [proposed      ] -> died at admission (died on support (19 headlines, 15 weeks; floor 25))
   tariff           [proposed      ] -> died at admission (died on vocabulary test (margin -0.22))
   antitrust        [proposed      ] -> died at admission (died on support (2 headlines, 2 weeks; floor 25))
   retail           [proposed      ] -> died at admission (died on vocabulary test (margin -0.29))
   music            [proposed      ] -> died at admission (died on support (19 headlines, 18 weeks; floor 25))
   credit           [proposed      ] -> died at admission (died on vocabulary test (margin -0.19))
   tablet           [mined         ] -> frozen survivor
   holiday          [mined         ] -> dropped by planner: Calendar language representing seasonal shopping periods.
   energy           [mined         ] -> dropped by planner: Generic sector or commodity-related term not specific to com
   dip              [mined         ] -> dropped by planner: Generic market-process language describing price movements.
   watch            [mined         ] -> frozen survivor
   advertising      [proposed      ] -> struck on two axes: news contaminated + earnings channel none
   supply chain     [proposed      ] -> struck on two axes: news contaminated + earnings channel none
   gaming           [proposed      ] -> struck on two axes: news contaminated + earnings channel none
FROZEN at session 1: 2 survivors — tablet, watch
--- SESSION TABLES (verbatim — the synthesis and the evaluator each see exactly this text) ---
COMPANY: AAPL (week -77 relative to corpus end)
VERDICT TABLE (frozen at session 1):
- tablet: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of th
e company's iPad line of multi-purpose tablets.
  coverage relation: no clear historical relation (r=-0.043, r_spy=+0.020, r_exc=-0.064); specificity: ticker-specific, more t
han the market; significance: q=0.84 — not significant after multiple testing
  tone relation: no clear historical relation (r=+0.032, r_spy=+0.112, r_exc=-0.031); specificity: mostly mirrors the market;
significance: q=0.84 — not significant after multiple testing
- watch: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of the
company's Apple Watch wearables.
  coverage relation: higher coverage historically preceded weaker weeks (r=-0.053, r_spy=+0.021, r_exc=-0.077); specificity: t
icker-specific, more than the market; significance: q=0.84 — not significant after multiple testing
  tone relation: no clear historical relation (r=+0.006, r_spy=-0.013, r_exc=+0.015); specificity: mostly mirrors the market;
significance: q=0.94 — not significant after multiple testing
- COMPANY coverage: captures: the volume of headlines naming the company | no clear historical relation (r=-0.011, r_spy=+0.07
9, r_exc=-0.062); specificity: unclear specificity; significance: q=0.94 — not significant after multiple testing
- COMPANY tone: captures: the tone of headlines naming the company | more positive tone historically preceded weaker weeks (r=
-0.081, r_spy=-0.064, r_exc=-0.057); specificity: unclear specificity; significance: q=0.50 — not significant after multiple t
esting
THIS WEEK'S VALUES (pipeline-computed against each feature's own history up to this week — these features are sparse: most wee
ks are zero):
  tablet|share: quiet (zero coverage this week; active in 39 of 483 weeks to date)
  watch|share: quiet (zero coverage this week; active in 38 of 483 weeks to date)
  COMPANY|share: now +0.0163 | percentile among its own active weeks 97% | active in 255 of 483 weeks | level: elevated
  tablet|senti: no coverage this week — tone undefined
  watch|senti: no coverage this week — tone undefined
  COMPANY|senti: tone uninformative (2 stories this week; 0 positive, 0 negative) — now +0.0000
RECENT NEWS BINS (the company's own coverage, last 4 weekly bins):
  week  -80: no stories — tone undefined
  week  -79: no stories — tone undefined
  week  -78: 1 story — 0 positive, 0 negative, 1 neutral | tone uninformative (fewer than 3 stories)
  week  -77: 2 stories — 0 positive, 0 negative, 2 neutral | tone uninformative (fewer than 3 stories)
MARKET STATE THIS WEEK:
  SPY_ret: +0.0216 (an up week for the market)
  SPY_vol: +0.0172 (calm volatility)
  dgs10_chg: +0.0900
  ticker-market link: AAPL's weekly returns correlate +0.53 with SPY over the trailing 52 weeks — moves closely with the marke
t
  market-relation (window to date): down market weeks historically preceded stronger weeks for this ticker (r=-0.065) — not si
gnificant after multiple testing
RECENT HEADLINES NAMING THIS COMPANY (last month, with tone labels):
  [ neutral] The Apple Card Is Here, With Cash Back and Privacy Promises
  [ neutral] Aramco Emerges Ahead of Apple as World’s Most Profitable Company
  [ neutral] Stocks to Watch: Apple, Chipotle, Tesla
MEMORY LEDGER (this system's prior calls for this company, pipeline-glossed call-relative outcomes):
  (empty — first call for this company)
LESSONS THIS SYSTEM WROTE FROM ITS OWN REALIZED OUTCOMES:
  (none yet)
--- STANDING BLOCK: advocate (printed once; every advocate prompt = this block + that session's tables) ---
You are the devil's advocate in an investment-research pipeline. You receive exactly the tables the decision layer will see, b
efore it decides. Argue the strongest case for the OPPOSITE of the direction the tables lean toward — if they lean constructiv
e, argue the bear case; if defensive, the bull case; if they appear balanced, argue that the balance is an artifact of sparse
coverage. Use only the printed tables; quote values, never compute; at most four sentences.
Also return counter_direction (buy, sell, or hold — the direction you would call from these tables) and strongest_item (the si
ngle printed item the decision layer is most likely to underweight).
Reply with JSON only: {"brief": "...", "counter_direction": "buy|sell|hold", "strongest_item": "..."}
--- advocate REPLY ---
```json
{
  "brief": "While the decision layer might read elevated company share as a bullish surge, this apparent activity is merely an
artifact of sparse coverage and only two neutral stories this week. Furthermore, historical data shows that more positive tone
precedes weaker weeks (r=-0.81), and higher watch coverage historically preceded weaker weeks. Therefore, the seemingly constr
uctive activity actually masks an underlying vulnerability that warrants caution.",
  "counter_direction": "sell",
  "strongest_item": "COMPANY tone: more positive tone historically preceded weaker weeks (r=-0.081)"
}
```
```

ADVOCATE TWIN: brief cites numbers absent from the tables: ['-0.81']
--- STANDING BLOCK: synthesis (printed once; every synthesis prompt = this block + that session's tables) ---
You are the decision layer of an investment-research pipeline for one company. Your job: call the direction of this company's coming week — buy = you expect the coming week's close higher, sell = lower. hold is abstention and is available only when the printed evidence genuinely balances; a pipeline that always abstains provides nothing, but never manufacture conviction the tables do not show. strength grades your conviction: 1 = marginal, 2 = supported by the tables, 3 = coherent across several features.
You receive the frozen verdict table (each surviving signal word with what its coverage captures, its transmission channel, and its verbalized statistics), this week's semantic levels and binned news counts, the company's own coverage bin, the market state, recent headlines, and this system's ledger of its own prior calls, their realized outcomes, and lessons it has written for itself.
Keep every listed feature unless you give a per-feature removal reason. Then: direction = buy | sell | hold; strength = 1 | 2 | 3; evidence = the printed values your call rests on; strength_boundary = what evidence would move you one grade down; rationale = at most two sentences. Where a feature's statistics conflict with what it captures, say so. A relation printed as 'no clear historical relation' carries no directional weight, and a relation printed as not significant after multiple testing may temper strength but never set direction: with no relation surviving multiple testing, the honest call is hold. Address the devil's advocate brief: name its strongest item in advocate_response and state, on printed evidence, why your call survives it, or concede to hold. The ledger is short — it may temper strength, never override evidence. Its rows arrive pre-glossed by the pipeline, call-relative: for an abstained call the printed move is context (a move sat out), never this system's realized gain or loss. Use nothing outside these tables; do no arithmetic.
Reply with JSON only: {"final_features": ["..."], "removals": [{"term": "...", "reason": "..."}], "advocate_response": "...", "direction": "...", "strength": 1, "evidence": ["..."], "strength_boundary": "...", "rationale": "..."}
--- synthesis REPLY ---
```json
{
  "final_features": [
    "tablet",
    "watch",
    "COMPANY coverage",
    "COMPANY tone"
  ],
  "removals": [],
  "advocate_response": "The advocate correctly notes historical tendencies, but all features in the verdict table—including COMPANY tone and watch coverage—are not significant after multiple testing (q >= 0.50) and carry no surviving directional relationships.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "tablet: q=0.84 — not significant after multiple testing",
    "watch: q=0.84 — not significant after multiple testing",
    "COMPANY coverage: q=0.94 — not significant after multiple testing",
    "COMPANY tone: q=0.50 — not significant after multiple testing"
  ],
  "strength_boundary": "No boundary applies as all tested relations are non-significant, mandating a hold.",
  "rationale": "With no relationship surviving multiple testing across any feature, the honest call is hold. The pipeline rules dictate that when no relations survive multiple testing, abstention is required."
}
```
--- STANDING BLOCK: evaluator (printed once; every evaluator prompt = this block + that session's tables) ---
You audit a decision against exactly the tables its author saw. Check both directions:
1. every feature the decision lists exists in the verdict table; 2. every verdict-table survivor is retained or has a stated removal reason; 3. every cited value matches the printed tables; 4. no claim untraceable to the tables (outside knowledge); 5. no arithmetic performed by the decision (numbers may be quoted, never computed).
Also return audit_note: one sentence stating what your acceptance or revision rests on.
Reply with JSON only: {"checks": {"listed_valid": true, "survivors_accounted": true, "citations_match": true, "no_outside_knowledge": true, "no_arithmetic": true}, "violations": ["..."], "audit_note": "...", "verdict": "accept|revise"}
--- evaluator REPLY ---
```json
{"checks": {"listed_valid": true, "survivors_accounted": true, "citations_match": true, "no_outside_knowledge": true, "no_arithmetic": true}, "violations": [], "audit_note": "Acceptance rests on all features, removals, citations, and non-significance claims strictly matching the verdict table without outside knowledge or arithmetic.", "verdict": "accept"}
```
DECISION: hold strength 1 | realized next week +0.9% | features 4

--- SESSION 2/6: AAPL, week -61 ---
--- REFLECTION TABLES (verbatim — the lesson writer sees exactly this) ---
PRIOR CALLS AND REALIZED OUTCOMES (pipeline-glossed, call-relative):
  session 1: hold strength 1 -> abstained — no position held; the ticker moved +0.9% that week (an up move sat out, not a gain) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
DETERMINISTIC SUMMARY (pipeline-computed): 1 abstentions — none sat out a down week; 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction. Quote these; do not recompute.
--- STANDING BLOCK: reflection (printed once; every reflection prompt = this block + that session's tables) ---
You are the memory of an investment-research pipeline. You see this system's own prior calls for one company, each already glossed by the pipeline relative to the call made: directional calls are marked correct or incorrect on direction; abstained calls held no position, so the move printed beside them is context (a move sat out), never a gain, loss, or error of the call. Write exactly one lesson of at most two sentences that a later decision for this company could use. Judge directional calls on their direction; judge abstentions only on the context they were made on (the printed coverage and market state), never on the move that followed. Use only the printed rows; quote numbers, never compute; no advice beyond what the rows support. Do not merely restate a row — state the pattern across rows, or say the rows are too few to support one.
Reply with JSON only: {"lesson": "..."}
--- reflection REPLY ---
{"lesson": "With only 1 abstention sitting out a \"+0.9%\" up week in a \"calm\" volatility market under \"thin coverage (2 stories)\", the rows are too few to support a pattern."}
MEMORY WRITE (lesson): With only 1 abstention sitting out a "+0.9%" up week in a "calm" volatility market under "thin coverage (2 stories)", the rows are too few to support a pattern.

--- SESSION TABLES (verbatim — the synthesis and the evaluator each see exactly this text) ---
COMPANY: AAPL (week -61 relative to corpus end)
VERDICT TABLE (frozen at session 1):
- tablet: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of the company's iPad line of multi-purpose tablets.
  coverage relation: no clear historical relation (r=-0.043, r_spy=+0.020, r_exc=-0.064); specificity: ticker-specific, more than the market; significance: q=0.84 — not significant after multiple testing
  tone relation: no clear historical relation (r=+0.032, r_spy=+0.112, r_exc=-0.031); specificity: mostly mirrors the market; significance: q=0.84 — not significant after multiple testing
- watch: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of the company's Apple Watch wearables.
  coverage relation: higher coverage historically preceded weaker weeks (r=-0.053, r_spy=+0.021, r_exc=-0.077); specificity: ticker-specific, more than the market; significance: q=0.84 — not significant after multiple testing
  tone relation: no clear historical relation (r=+0.006, r_spy=-0.013, r_exc=+0.015); specificity: mostly mirrors the market; significance: q=0.94 — not significant after multiple testing
- COMPANY coverage: captures: the volume of headlines naming the company | no clear historical relation (r=-0.011, r_spy=+0.079, r_exc=-0.062); specificity: unclear specificity; significance: q=0.94 — not significant after multiple testing
- COMPANY tone: captures: the tone of headlines naming the company | more positive tone historically preceded weaker weeks (r=-0.081, r_spy=-0.064, r_exc=-0.057); specificity: unclear specificity; significance: q=0.50 — not significant after multiple testing
THIS WEEK'S VALUES (pipeline-computed against each feature's own history up to this week — these features are sparse: most weeks are zero):
  tablet|share: quiet (zero coverage this week; active in 39 of 499 weeks to date)
  watch|share: quiet (zero coverage this week; active in 38 of 499 weeks to date)
  COMPANY|share: now +0.0160 | percentile among its own active weeks 96% | active in 260 of 499 weeks | level: elevated
  tablet|senti: no coverage this week — tone undefined
  watch|senti: no coverage this week — tone undefined
  COMPANY|senti: tone uninformative (2 stories this week; 0 positive, 0 negative) — now +0.0000
RECENT NEWS BINS (the company's own coverage, last 4 weekly bins):
  week -64: no stories — tone undefined
  week -63: no stories — tone undefined
  week -62: no stories — tone undefined
  week -61: 2 stories — 0 positive, 0 negative, 2 neutral | tone uninformative (fewer than 3 stories)
MARKET STATE THIS WEEK:
  SPY_ret: +0.0163 (an up week for the market)
  SPY_vol: +0.0173 (calm volatility)
  dgs10_chg: +0.0300
  ticker-market link: AAPL's weekly returns correlate +0.66 with SPY over the trailing 52 weeks — moves closely with the market
  market-relation (window to date): down market weeks historically preceded stronger weeks for this ticker (r=-0.065) — not significant after multiple testing
RECENT HEADLINES NAMING THIS COMPANY (last month, with tone labels):
  [neutral] Apple Design Chief Jony Ive to Depart
  [neutral] SoftBank Looks to Goldman, Apple for Sequel to $100 Billion Vision Fund
  [neutral] Intel Agrees to Sell Smartphone-Chip Assets to Apple
MEMORY LEDGER (this system's prior calls for this company, pipeline-glossed call-relative outcomes):
  session 1: hold strength 1 -> abstained — no position held; the ticker moved +0.9% that week (an up move sat out, not a gain) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
LESSONS THIS SYSTEM WROTE FROM ITS OWN REALIZED OUTCOMES:
  - With only 1 abstention sitting out a "+0.9%" up week in a "calm" volatility market under "thin coverage (2 stories)", the rows are too few to support a pattern.
--- advocate REPLY ---
```json
{
  "brief": "Despite low headline volume and uninformative tone, the company's share level is elevated at 96% among its active weeks while the market prints an up week with calm volatility. The historical relation notes that more positive tone historically preceded weaker weeks (r=-0.081), but current tone is uninformative and coverage is thin rather than positive. Furthermore, down market weeks historically preceded stronger weeks (r=-0.065), and this week the market is up (+0.0163).",
  "counter_direction": "sell",
  "strongest_item": "COMPANY|share: now +0.0160 | percentile among its own active weeks 96% | active in 260 of 499 weeks | level: elevated"
}
```
--- synthesis REPLY ---
```json
{
  "final_features": [
    "tablet",
    "watch",
    "COMPANY coverage",
    "COMPANY tone"
  ],
  "removals": [],
  "advocate_response": "The advocate correctly notes that COMPANY|share is elevated at the 96th percentile, but all available features show no clear historical relation or lack significance after multiple testing.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "COMPANY|share: now +0.0160 | percentile among its own active weeks 96% | active in 260 of 499 weeks | level: elevated",
    "tablet|share: quiet",
    "watch|share: quiet",
    "COMPANY|senti: tone uninformative"
  ],
  "strength_boundary": "No features survive multiple testing, so any conviction above 1 is unsupported.",
  "rationale": "All available features have relations that are not significant after multiple testing. With no statistically s
```

urviving relations, the honest call is hold."
}
```
--- evaluator REPLY ---
```json
{"checks": {"listed_valid": true, "survivors_accounted": true, "citations_match": true, "no_outside_knowledge": true, "no_arithmetic": true}, "violations": [], "audit_note": "Every feature, value, and claim strictly matches the printed tables and no outside knowledge or unauthorized arithmetic was used.", "verdict": "accept"}
```
DECISION: hold strength 1 | realized next week -1.8% | features 4

--- SESSION 3/6: AAPL, week -54 ---
--- REFLECTION TABLES (verbatim — the lesson writer sees exactly this) ---
PRIOR CALLS AND REALIZED OUTCOMES (pipeline-glossed, call-relative):
  session 1: hold strength 1 -> abstained — no position held; the ticker moved +0.9% that week (an up move sat out, not a gain) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 2: hold strength 1 -> abstained — no position held; the ticker moved -1.8% that week (a down move sat out, not a loss) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
DETERMINISTIC SUMMARY (pipeline-computed): 2 abstentions — 1 sat out down weeks (mean -1.8%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction. Quote these; do not recompute.
--- reflection REPLY ---
```json
{"lesson": "Across 2 abstentions — 1 sat out down weeks (mean -1.8%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction, the pipeline consistently abstained during thin coverage (2 stories) with undefined tone."}
```
MEMORY WRITE (lesson): Across 2 abstentions — 1 sat out down weeks (mean -1.8%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction, the pipeline consistently abstained during thin coverage (2 stories) with undefined tone.
--- SESSION TABLES (verbatim — the synthesis and the evaluator each see exactly this text) ---
COMPANY: AAPL (week -54 relative to corpus end)
VERDICT TABLE (frozen at session 1):
- tablet: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of the company's iPad line of multi-purpose tablets.
  coverage relation: no clear historical relation (r=-0.043, r_spy=+0.020, r_exc=-0.064); specificity: ticker-specific, more than the market; significance: q=0.84 — not significant after multiple testing
  tone relation: no clear historical relation (r=+0.032, r_spy=+0.112, r_exc=-0.031); specificity: mostly mirrors the market; significance: q=0.84 — not significant after multiple testing
- watch: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of the company's Apple Watch wearables.
  coverage relation: higher coverage historically preceded weaker weeks (r=-0.053, r_spy=+0.021, r_exc=-0.077); specificity: ticker-specific, more than the market; significance: q=0.84 — not significant after multiple testing
  tone relation: no clear historical relation (r=+0.006, r_spy=-0.013, r_exc=+0.015); specificity: mostly mirrors the market; significance: q=0.94 — not significant after multiple testing
- COMPANY coverage: captures: the volume of headlines naming the company | no clear historical relation (r=-0.011, r_spy=+0.079, r_exc=-0.062); specificity: unclear specificity; significance: q=0.94 — not significant after multiple testing
- COMPANY tone: captures: the tone of headlines naming the company | more positive tone historically preceded weaker weeks (r=-0.081, r_spy=-0.064, r_exc=-0.057); specificity: unclear specificity; significance: q=0.50 — not significant after multiple testing
THIS WEEK'S VALUES (pipeline-computed against each feature's own history up to this week — these features are sparse: most weeks are zero):
  tablet|share: quiet (zero coverage this week; active in 39 of 506 weeks to date)
  watch|share: quiet (zero coverage this week; active in 38 of 506 weeks to date)
  COMPANY|share: now +0.0187 | percentile among its own active weeks 97% | active in 262 of 506 weeks | level: elevated
  tablet|senti: no coverage this week — tone undefined
  watch|senti: no coverage this week — tone undefined
  COMPANY|senti: tone uninformative (2 stories this week; 0 positive, 0 negative) — now +0.0000
RECENT NEWS BINS (the company's own coverage, last 4 weekly bins):
  week -57: no stories — tone undefined
  week -56: no stories — tone undefined
  week -55: no stories — tone undefined
  week -54: 2 stories — 0 positive, 0 negative, 2 neutral | tone uninformative (fewer than 3 stories)
MARKET STATE THIS WEEK:
  SPY_ret: +0.0102 (an up week for the market)
  SPY_vol: +0.0196 (calm volatility)
  dgs10_chg: +0.3500
  ticker-market link: AAPL's weekly returns correlate +0.68 with SPY over the trailing 52 weeks — moves closely with the market
  market-relation (window to date): down market weeks historically preceded stronger weeks for this ticker (r=-0.062) — not significant after multiple testing
RECENT HEADLINES NAMING THIS COMPANY (last month, with tone labels):
  [neutral] Apple Wants Gamers to Hit ‘Subscribe’ With Apple Arcade
  [neutral] Are Apple’s Shares Approaching a Speed Bump?
MEMORY LEDGER (this system's prior calls for this company, pipeline-glossed call-relative outcomes):
  session 1: hold strength 1 -> abstained — no position held; the ticker moved +0.9% that week (an up move sat out, not a gain) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 2: hold strength 1 -> abstained — no position held; the ticker moved -1.8% that week (a down move sat out, not a loss) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
LESSONS THIS SYSTEM WROTE FROM ITS OWN REALIZED OUTCOMES:
  - With only 1 abstention sitting out a "+0.9%" up week in a "calm" volatility market under "thin coverage (2 stories)", the rows are too few to support a pattern.
  - Across 2 abstentions — 1 sat out down weeks (mean -1.8%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction, the pipeline consistently abstained during thin coverage (2 stories) with undefined
--- advocate REPLY ---
```json
{
  "brief": "Despite elevated headline share at the 97th percentile, company tone is uninformative with only 2 stories and zero

positive coverage. Furthermore, historical data warns that more positive tone historically preceded weaker weeks (r=-0.81), an
d higher watch coverage historically preceded weaker weeks (r=-0.053). With sparse activity and zero coverage on product lines
like tablet and watch this week, the apparent activity is an artifact of minimal engagement rather than true bullish momentum.
Therefore, the constructive tilt in headline volume is misleading and warrants a sell stance.",
  "counter_direction": "sell",
  "strongest_item": "COMPANY tone: more positive tone historically preceded weaker weeks (r=-0.081, r_spy=-0.064, r_exc=-0.05
7)"
}
```
ADVOCATE TWIN: brief cites numbers absent from the tables: ['-0.81']
--- synthesis REPLY ---
```json
{
  "final_features": [
    "tablet",
    "watch",
    "COMPANY coverage",
    "COMPANY tone"
  ],
  "removals": [],
  "advocate_response": "While the advocate notes historical inverse relations for tone and watch coverage, none of these relat
ions are statistically significant after multiple testing (q=0.50 and q=0.84), and this week's tone is strictly uninformative
with zero net positive or negative stories. As instructed, a relation not significant after multiple testing can temper streng
th but never set direction, and with no surviving significant relation, the honest call is hold.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "tablet|share: quiet (zero coverage this week)",
    "watch|share: quiet (zero coverage this week)",
    "COMPANY|share: now +0.0187 (elevated, 97th percentile)",
    "COMPANY|senti: tone uninformative (2 stories this week, 0 positive, 0 negative)"
  ],
  "strength_boundary": "If any of the historical relations had reached statistical significance after multiple testing, or if
coverage had been robust rather than sparse, a directional call might have been warranted.",
  "rationale": "All surviving signals lack statistical significance after multiple testing, and company tone is uninformative
with only two neutral stories. Per instructions, with no relation surviving multiple testing and conflicting or absent signal
statistics, the honest call is hold."
}
```
--- evaluator REPLY ---
```json
{
  "checks": {
    "listed_valid": true,
    "survivors_accounted": true,
    "citations_match": true,
    "no_outside_knowledge": true,
    "no_arithmetic": true
  },
  "violations": [],
  "audit_note": "All listed features, values, and absence of statistical significance are faithfully sourced from the provided
tables and text without arithmetic or outside knowledge.",
  "verdict": "accept"
}
```
DECISION: hold strength 1 | realized next week -0.5% | features 4

--- SESSION 4/6: AAPL, week -35 ---
--- REFLECTION TABLES (verbatim — the lesson writer sees exactly this) ---
PRIOR CALLS AND REALIZED OUTCOMES (pipeline-glossed, call-relative):
  session 1: hold strength 1 -> abstained — no position held; the ticker moved +0.9% that week (an up move sat out, not a gai
n) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 2: hold strength 1 -> abstained — no position held; the ticker moved -1.8% that week (a down move sat out, not a los
s) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 3: hold strength 1 -> abstained — no position held; the ticker moved -0.5% that week (a down move sat out, not a los
s) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
DETERMINISTIC SUMMARY (pipeline-computed): 3 abstentions — 2 sat out down weeks (mean -1.1%); 1 sat out up weeks (mean +0.9%);
0 directional calls — 0 correct on direction. Quote these; do not recompute.
--- reflection REPLY ---
{"lesson": "Across 3 abstentions — 2 sat out down weeks (mean -1.1%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0
correct on direction, all abstained calls occurred under thin coverage (2 stories) with tone undefined on fewer than 3 storie
s, market up, and volatility calm."}
MEMORY WRITE (lesson): Across 3 abstentions — 2 sat out down weeks (mean -1.1%); 1 sat out up weeks (mean +0.9%); 0 directiona
l calls — 0 correct on direction, all abstained calls occurred under thin coverage (2 stories) with tone undefined on fewer th
an 3 stories, market up, and volatility calm.
--- SESSION TABLES (verbatim — the synthesis and the evaluator each see exactly this text) ---
COMPANY: AAPL (week -35 relative to corpus end)
VERDICT TABLE (frozen at session 1):
- tablet: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of th
e company's iPad line of multi-purpose tablets.
  coverage relation: no clear historical relation (r=-0.043, r_spy=+0.020, r_exc=-0.064); specificity: ticker-specific, more t
han the market; significance: q=0.84 — not significant after multiple testing
  tone relation: no clear historical relation (r=+0.032, r_spy=+0.112, r_exc=-0.031); specificity: mostly mirrors the market;
significance: q=0.84 — not significant after multiple testing
- watch: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of the
```

company's Apple Watch wearables.
  coverage relation: higher coverage historically preceded weaker weeks (r=-0.053, r_spy=+0.021, r_exc=-0.077); specificity: t
icker-specific, more than the market; significance: q=0.84 — not significant after multiple testing
  tone relation: no clear historical relation (r=+0.006, r_spy=-0.013, r_exc=+0.015); specificity: mostly mirrors the market;
significance: q=0.94 — not significant after multiple testing
- COMPANY coverage: captures: the volume of headlines naming the company | no clear historical relation (r=-0.011, r_spy=+0.07
9, r_exc=-0.062); specificity: unclear specificity; significance: q=0.94 — not significant after multiple testing
- COMPANY tone: captures: the tone of headlines naming the company | more positive tone historically preceded weaker weeks (r=
-0.081, r_spy=-0.064, r_exc=-0.057); specificity: unclear specificity; significance: q=0.50 — not significant after multiple t
esting
THIS WEEK'S VALUES (pipeline-computed against each feature's own history up to this week — these features are sparse: most wee
ks are zero):
  tablet|share: quiet (zero coverage this week; active in 39 of 525 weeks to date)
  watch|share: quiet (zero coverage this week; active in 38 of 525 weeks to date)
  COMPANY|share: now +0.0152 | percentile among its own active weeks 95% | active in 265 of 525 weeks | level: elevated
  tablet|senti: no coverage this week — tone undefined
  watch|senti: no coverage this week — tone undefined
  COMPANY|senti: tone uninformative (1 stories this week; 1 positive, 0 negative) — now +1.0000
RECENT NEWS BINS (the company's own coverage, last 4 weekly bins):
  week -38: no stories — tone undefined
  week -37: no stories — tone undefined
  week -36: no stories — tone undefined
  week -35: 1 story — 1 positive, 0 negative, 0 neutral | tone uninformative (fewer than 3 stories)
MARKET STATE THIS WEEK:
  SPY_ret: -0.0096 (a down week for the market)
  SPY_vol: +0.0093 (calm volatility)
  dgs10_chg: -0.1400
  ticker-market link: AAPL's weekly returns correlate +0.71 with SPY over the trailing 52 weeks — moves closely with the marke
t
  market-relation (window to date): down market weeks historically preceded stronger weeks for this ticker (r=-0.059) — not si
gnificant after multiple testing
RECENT HEADLINES NAMING THIS COMPANY (last month, with tone labels):
  [positive] Jobs, Cook, Ive—Blevins? The Rise of Apple’s Cost Cutter
MEMORY LEDGER (this system's prior calls for this company, pipeline-glossed call-relative outcomes):
  session 1: hold strength 1 -> abstained — no position held; the ticker moved +0.9% that week (an up move sat out, not a gai
n) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 2: hold strength 1 -> abstained — no position held; the ticker moved -1.8% that week (a down move sat out, not a los
s) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 3: hold strength 1 -> abstained — no position held; the ticker moved -0.5% that week (a down move sat out, not a los
s) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
LESSONS THIS SYSTEM WROTE FROM ITS OWN REALIZED OUTCOMES:
  - With only 1 abstention sitting out a "+0.9%" up week in a "calm" volatility market under "thin coverage (2 stories)", the
rows are too few to support a pattern.
  - Across 2 abstentions — 1 sat out down weeks (mean -1.8%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct
on direction, the pipeline consistently abstained during thin coverage (2 stories) with undefined
  - Across 3 abstentions — 2 sat out down weeks (mean -1.1%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct
on direction, all abstained calls occurred under thin coverage (2 stories) with tone undefined on
--- advocate REPLY ---
```json
{
  "brief": "Despite the tables leaning defensive with quiet tablet and watch channels and thin company coverage of just 1 stor
y, the bull case rests on the market-relation historical tendency where down market weeks (SPY_ret -0.0096) historically prece
ded stronger weeks for this ticker (r=-0.059). Furthermore, the company tone is recorded as positive (+1.0000) alongside an el
evated company share percentile (95%) among its active weeks. Finally, historical sessions show the system abstaining during t
hin coverage while sitting out potential moves, suggesting upside risk is being discounted.",
  "counter_direction": "buy",
  "strongest_item": "down market weeks historically preceded stronger weeks for this ticker (r=-0.059) — not significant after
multiple testing"
}
```
--- synthesis REPLY ---
```json
{
  "final_features": [
    "tablet",
    "watch",
    "COMPANY coverage",
    "COMPANY tone"
  ],
  "removals": [],
  "advocate_response": "The advocate's strongest point regarding down market weeks preceding stronger weeks has no significanc
e after multiple testing (q-values and tests across features show no surviving significance). Furthermore, all active signal r
elations fail multiple testing significance, dictating an honest hold.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "tablet|share: quiet (zero coverage this week)",
    "watch|share: quiet (zero coverage this week)",
    "COMPANY|share: now +0.0152 | percentile 95%",
    "COMPANY|senti: tone uninformative (1 stories this week; +1.0000)"
  ],
  "strength_boundary": "Any single surviving signal with statistical significance after multiple testing would move convictio
n.",
  "rationale": "All featured relations (tablet, watch, company coverage, company tone, and market-relation) are not significan
t after multiple testing. With no relation surviving multiple testing, the honest call is hold."
```

}
```
--- evaluator REPLY ---
```json
{"checks": {"listed_valid": true, "survivors_accounted": true, "citations_match": true, "no_outside_knowledge": true, "no_arithmetic": true}, "violations": [], "audit_note": "The decision correctly lists all valid features, accounts for all survivors, cites verifiable table and pipeline values without outside knowledge or arithmetic, and supports an honest hold.", "verdict": "accept"}
```
DECISION: hold strength 1 | realized next week -2.8% | features 4

--- SESSION 5/6: AAPL, week -31 ---
--- REFLECTION TABLES (verbatim — the lesson writer sees exactly this) ---
PRIOR CALLS AND REALIZED OUTCOMES (pipeline-glossed, call-relative):
  session 1: hold strength 1 -> abstained — no position held; the ticker moved +0.9% that week (an up move sat out, not a gain) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 2: hold strength 1 -> abstained — no position held; the ticker moved -1.8% that week (a down move sat out, not a loss) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 3: hold strength 1 -> abstained — no position held; the ticker moved -0.5% that week (a down move sat out, not a loss) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 4: hold strength 1 -> abstained — no position held; the ticker moved -2.8% that week (a down move sat out, not a loss) (made on: thin coverage (1 stories), tone undefined on fewer than 3 stories, market down, volatility calm)
DETERMINISTIC SUMMARY (pipeline-computed): 4 abstentions — 3 sat out down weeks (mean -1.7%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction. Quote these; do not recompute.
--- reflection REPLY ---
{"lesson": "The pipeline recorded \"4 abstentions — 3 sat out down weeks (mean -1.7%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction\" under thin coverage of 1 to 2 stories with an undefined tone. These rows are too few to support a broader pattern beyond consistently abstaining during periods of thin coverage."}
MEMORY WRITE (lesson): The pipeline recorded "4 abstentions — 3 sat out down weeks (mean -1.7%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction" under thin coverage of 1 to 2 stories with an undefined tone. These rows are too few to support a broader pattern beyond consistently abstaining during periods of thin coverage.
--- SESSION TABLES (verbatim — the synthesis and the evaluator each see exactly this text) ---
COMPANY: AAPL (week -31 relative to corpus end)
VERDICT TABLE (frozen at session 1):
- tablet: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of the company's iPad line of multi-purpose tablets.
  coverage relation: no clear historical relation (r=-0.043, r_spy=+0.020, r_exc=-0.064); specificity: ticker-specific, more than the market; significance: q=0.84 — not significant after multiple testing
  tone relation: no clear historical relation (r=+0.032, r_spy=+0.112, r_exc=-0.031); specificity: mostly mirrors the market; significance: q=0.84 — not significant after multiple testing
- watch: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of the company's Apple Watch wearables.
  coverage relation: higher coverage historically preceded weaker weeks (r=-0.053, r_spy=+0.021, r_exc=-0.077); specificity: ticker-specific, more than the market; significance: q=0.84 — not significant after multiple testing
  tone relation: no clear historical relation (r=+0.006, r_spy=-0.013, r_exc=+0.015); specificity: mostly mirrors the market; significance: q=0.94 — not significant after multiple testing
- COMPANY coverage: captures: the volume of headlines naming the company | no clear historical relation (r=-0.011, r_spy=+0.079, r_exc=-0.062); specificity: unclear specificity; significance: q=0.94 — not significant after multiple testing
- COMPANY tone: captures: the tone of headlines naming the company | more positive tone historically preceded weaker weeks (r=-0.081, r_spy=-0.064, r_exc=-0.057); specificity: unclear specificity; significance: q=0.50 — not significant after multiple testing
THIS WEEK'S VALUES (pipeline-computed against each feature's own history up to this week — these features are sparse: most weeks are zero):
  tablet|share: quiet (zero coverage this week; active in 39 of 529 weeks to date)
  watch|share: quiet (zero coverage this week; active in 38 of 529 weeks to date)
  COMPANY|share: now +0.0441 | percentile among its own active weeks 99% | active in 266 of 529 weeks | level: elevated
  tablet|senti: no coverage this week — tone undefined
  watch|senti: no coverage this week — tone undefined
  COMPANY|senti: tone negative (0 positive, 1 negative of 3 stories) — now -0.3333
RECENT NEWS BINS (the company's own coverage, last 4 weekly bins):
  week -34: no stories — tone undefined
  week -33: no stories — tone undefined
  week -32: no stories — tone undefined
  week -31: 3 stories — 0 positive, 1 negative, 2 neutral | tone -0.33
MARKET STATE THIS WEEK:
  SPY_ret: -0.0122 (a down week for the market)
  SPY_vol: +0.0183 (calm volatility)
  dgs10_chg: -0.1300
  ticker-market link: AAPL's weekly returns correlate +0.75 with SPY over the trailing 52 weeks — moves closely with the market
  market-relation (window to date): down market weeks historically preceded stronger weeks for this ticker (r=-0.055) — not significant after multiple testing
RECENT HEADLINES NAMING THIS COMPANY (last month, with tone labels):
  [positive] Jobs, Cook, Ive—Blevins? The Rise of Apple’s Cost Cutter
  [neutral] Apple’s Virus Warning Takes Bite Out of Chip-Maker Stocks
  [negative] U.S. Stocks Fall After Apple’s Sales Warning
  [neutral] Warren Buffett Found His ‘Elephant’ With Giant Stake in Apple
MEMORY LEDGER (this system's prior calls for this company, pipeline-glossed call-relative outcomes):
  session 1: hold strength 1 -> abstained — no position held; the ticker moved +0.9% that week (an up move sat out, not a gain) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 2: hold strength 1 -> abstained — no position held; the ticker moved -1.8% that week (a down move sat out, not a loss) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 3: hold strength 1 -> abstained — no position held; the ticker moved -0.5% that week (a down move sat out, not a loss) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 4: hold strength 1 -> abstained — no position held; the ticker moved -2.8% that week (a down move sat out, not a los
```

s) (made on: thin coverage (1 stories), tone undefined on fewer than 3 stories, market down, volatility calm)
LESSONS THIS SYSTEM WROTE FROM ITS OWN REALIZED OUTCOMES:
  - Across 2 abstentions — 1 sat out down weeks (mean -1.8%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction, the pipeline consistently abstained during thin coverage (2 stories) with undefined
  - Across 3 abstentions — 2 sat out down weeks (mean -1.1%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction, all abstained calls occurred under thin coverage (2 stories) with tone undefined on
  - The pipeline recorded "4 abstentions — 3 sat out down weeks (mean -1.7%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction" under thin coverage of 1 to 2 stories with an undefined tone. These
--- advocate REPLY ---
```json
{
  "brief": "Despite negative company tone of -0.3333 and a down market week where SPY_ret is -0.0122, history shows that down market weeks historically preceded stronger weeks for this ticker with r=-0.055. Furthermore, COMPANY coverage is elevated at percentile 99% among its own active weeks, showing intense focus rather than sparse noise. Finally, more positive tone historically preceded weaker weeks (r=-0.081), implying that negative headlines may contrarily foreshadow strength.",
  "counter_direction": "buy",
  "strongest_item": "COMPANY|share: now +0.0441 | percentile among its own active weeks 99% | active in 266 of 529 weeks | level: elevated"
}
```
--- synthesis REPLY ---
```json
{
  "final_features": [
    "tablet",
    "watch",
    "COMPANY coverage",
    "COMPANY tone"
  ],
  "removals": [],
  "advocate_response": "While COMPANY coverage is elevated at the 99th percentile and down market weeks show a mild historical tendency (r=-0.055) not significant after multiple testing, all featured relations carry q >= 0.50 and are not significant after multiple testing. Under the pipeline rules, with no relation surviving multiple testing, the honest call is hold.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "tablet: q=0.84 — not significant after multiple testing",
    "watch: q=0.84 — not significant after multiple testing",
    "COMPANY coverage: q=0.94 — not significant after multiple testing",
    "COMPANY tone: q=0.50 — not significant after multiple testing"
  ],
  "strength_boundary": "A shift to significant multiple-testing survival on directional features would move conviction.",
  "rationale": "All available signal relations are not significant after multiple testing. Therefore, the honest call is hold."
}
```
--- evaluator REPLY ---
```json
{"checks": {"listed_valid": true, "survivors_accounted": true, "citations_match": true, "no_outside_knowledge": true, "no_arithmetic": true}, "violations": [], "audit_note": "All listed features and citations accurately reflect the verdict table, with no outside knowledge or unauthorized arithmetic used.", "verdict": "accept"}
```
DECISION: hold strength 1 | realized next week -12.7% | features 4

--- SESSION 6/6: AAPL, week -2 ---
--- REFLECTION TABLES (verbatim — the lesson writer sees exactly this) ---
PRIOR CALLS AND REALIZED OUTCOMES (pipeline-glossed, call-relative):
  session 1: hold strength 1 -> abstained — no position held; the ticker moved +0.9% that week (an up move sat out, not a gain) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 2: hold strength 1 -> abstained — no position held; the ticker moved -1.8% that week (a down move sat out, not a loss) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 3: hold strength 1 -> abstained — no position held; the ticker moved -0.5% that week (a down move sat out, not a loss) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 4: hold strength 1 -> abstained — no position held; the ticker moved -2.8% that week (a down move sat out, not a loss) (made on: thin coverage (1 stories), tone undefined on fewer than 3 stories, market down, volatility calm)
  session 5: hold strength 1 -> abstained — no position held; the ticker moved -12.7% that week (a down move sat out, not a loss) (made on: active coverage (3 stories), tone negative, market down, volatility calm)
DETERMINISTIC SUMMARY (pipeline-computed): 5 abstentions — 4 sat out down weeks (mean -4.4%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction. Quote these; do not recompute.
--- reflection REPLY ---
```json
{"lesson": "Across 5 sessions, the pipeline abstained from all calls under thin or active coverage when tone was undefined or negative, sitting out 4 down weeks (mean -4.4%) and 1 up week (mean +0.9%). These rows are too few to support a directional pattern as 0 directional calls were made."}
```
MEMORY WRITE (lesson): Across 5 sessions, the pipeline abstained from all calls under thin or active coverage when tone was undefined or negative, sitting out 4 down weeks (mean -4.4%) and 1 up week (mean +0.9%). These rows are too few to support a directional pattern as 0 directional calls were made.
--- SESSION TABLES (verbatim — the synthesis and the evaluator each see exactly this text) ---
COMPANY: AAPL (week -2 relative to corpus end)
VERDICT TABLE (frozen at session 1):
- tablet: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of the company's iPad line of multi-purpose tablets.
  coverage relation: no clear historical relation (r=-0.043, r_spy=+0.020, r_exc=-0.064); specificity: ticker-specific, more than the market; significance: q=0.84 — not significant after multiple testing

  tone relation: no clear historical relation (r=+0.032, r_spy=+0.112, r_exc=-0.031); specificity: mostly mirrors the market; significance: q=0.84 — not significant after multiple testing
- watch: captures: co-occurs with this company's mention headlines | channel: revenue helps — Directly relates to sales of the company's Apple Watch wearables.
  coverage relation: higher coverage historically preceded weaker weeks (r=-0.053, r_spy=+0.021, r_exc=-0.077); specificity: ticker-specific, more than the market; significance: q=0.84 — not significant after multiple testing
  tone relation: no clear historical relation (r=+0.006, r_spy=-0.013, r_exc=+0.015); specificity: mostly mirrors the market; significance: q=0.94 — not significant after multiple testing
- COMPANY coverage: captures: the volume of headlines naming the company | no clear historical relation (r=-0.011, r_spy=+0.079, r_exc=-0.062); specificity: unclear specificity; significance: q=0.94 — not significant after multiple testing
- COMPANY tone: captures: the tone of headlines naming the company | more positive tone historically preceded weaker weeks (r=-0.081, r_spy=-0.064, r_exc=-0.057); specificity: unclear specificity; significance: q=0.50 — not significant after multiple testing
THIS WEEK'S VALUES (pipeline-computed against each feature's own history up to this week — these features are sparse: most weeks are zero):
  tablet|share: quiet (zero coverage this week; active in 39 of 558 weeks to date)
  watch|share: quiet (zero coverage this week; active in 38 of 558 weeks to date)
  COMPANY|share: now +0.0217 | percentile among its own active weeks 97% | active in 276 of 558 weeks | level: elevated
  tablet|senti: no coverage this week — tone undefined
  watch|senti: no coverage this week — tone undefined
  COMPANY|senti: tone uninformative (1 stories this week; 0 positive, 1 negative) — now -1.0000
RECENT NEWS BINS (the company's own coverage, last 4 weekly bins):
  week   -5: no stories — tone undefined
  week   -4: no stories — tone undefined
  week   -3: 1 story — 0 positive, 0 negative, 1 neutral | tone uninformative (fewer than 3 stories)
  week   -2: 1 story — 0 positive, 1 negative, 0 neutral | tone uninformative (fewer than 3 stories)
MARKET STATE THIS WEEK:
  SPY_ret: -0.0248 (a down week for the market)
  SPY_vol: +0.0209 (calm volatility)
  dgs10_chg: -0.0500
  ticker-market link: AAPL's weekly returns correlate +0.79 with SPY over the trailing 52 weeks — moves closely with the market
  market-relation (window to date): down market weeks historically preceded stronger weeks for this ticker (r=-0.037) — not significant after multiple testing
RECENT HEADLINES NAMING THIS COMPANY (last month, with tone labels):
  [neutral] Apple, Tesla Shares Keep Rising After Stock Splits
  [negative] Tesla, Apple, Amazon Down Premarket
MEMORY LEDGER (this system's prior calls for this company, pipeline-glossed call-relative outcomes):
  session 1: hold strength 1 -> abstained — no position held; the ticker moved +0.9% that week (an up move sat out, not a gain) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 2: hold strength 1 -> abstained — no position held; the ticker moved -1.8% that week (a down move sat out, not a loss) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 3: hold strength 1 -> abstained — no position held; the ticker moved -0.5% that week (a down move sat out, not a loss) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 4: hold strength 1 -> abstained — no position held; the ticker moved -2.8% that week (a down move sat out, not a loss) (made on: thin coverage (1 stories), tone undefined on fewer than 3 stories, market down, volatility calm)
  session 5: hold strength 1 -> abstained — no position held; the ticker moved -12.7% that week (a down move sat out, not a loss) (made on: active coverage (3 stories), tone negative, market down, volatility calm)
LESSONS THIS SYSTEM WROTE FROM ITS OWN REALIZED OUTCOMES:
  - Across 3 abstentions — 2 sat out down weeks (mean -1.1%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction, all abstained calls occurred under thin coverage (2 stories) with tone undefined on
  - The pipeline recorded "4 abstentions — 3 sat out down weeks (mean -1.7%); 1 sat out up weeks (mean +0.9%); 0 directional calls — 0 correct on direction" under thin coverage of 1 to 2 stories with an undefined tone. These
  - Across 5 sessions, the pipeline abstained from all calls under thin or active coverage when tone was undefined or negative, sitting out 4 down weeks (mean -4.4%) and 1 up week (mean +0.9%). These rows are too few to supp
--- advocate REPLY ---
```json
{
  "brief": "Despite negative tone and elevated headline share, the historical relation shows that more positive tone historically preceded weaker weeks (r=-0.081) and down market weeks historically preceded stronger weeks for this ticker (r=-0.037). With tablet and watch share quiet and the current COMPANY senti at -1.0000 on minimal stories, the apparent bearish signal is undermined by sparse coverage. Therefore, the bull case rests on treating this negative flash as noise, pointing toward a rebound.",
  "counter_direction": "buy",
  "strongest_item": "COMPANY|senti: tone uninformative (1 stories this week; 0 positive, 1 negative) — now -1.0000"
}
```
--- synthesis REPLY ---
```json
{
  "final_features": [
    "tablet",
    "watch",
    "COMPANY coverage",
    "COMPANY tone"
  ],
  "removals": [],
  "advocate_response": "The devil's advocate points to the negative tone on minimal stories as potentially uninformative noise, but all listed features have significance q >= 0.50 and carry no clear historical relation or are not significant after multiple testing.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "tablet significance q=0.84 — not significant after multiple testing",
    "watch significance q=0.84 — not significant after multiple testing",
```

```
    "COMPANY coverage significance q=0.94 — not significant after multiple testing",
    "COMPANY tone significance q=0.50 — not significant after multiple testing"
  ],
  "strength_boundary": "No relation surviving multiple testing permits a directional call; any single significant relation wou
ld move strength to 2.",
  "rationale": "With no relation surviving multiple testing across the verdict table, the honest call is hold. The memory ledg
er and current sparse statistics do not override the lack of statistically significant directional relations."
}
```
--- evaluator REPLY ---
```json
{"checks": {"listed_valid": true, "survivors_accounted": true, "citations_match": true, "no_outside_knowledge": true, "no_arit
hmetic": true}, "violations": [], "audit_note": "The decision is fully grounded in the provided verdict table and pipeline sta
tistics with all features and citations matching accurately and no outside knowledge or unauthorized arithmetic used.", "verdi
ct": "accept"}
```
DECISION: hold strength 1 | realized next week -4.6% | features 4
--- REFLECTION TABLES (verbatim — the lesson writer sees exactly this) ---
PRIOR CALLS AND REALIZED OUTCOMES (pipeline-glossed, call-relative):
  session 1: hold strength 1 -> abstained — no position held; the ticker moved +0.9% that week (an up move sat out, not a gai
n) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 2: hold strength 1 -> abstained — no position held; the ticker moved -1.8% that week (a down move sat out, not a los
s) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 3: hold strength 1 -> abstained — no position held; the ticker moved -0.5% that week (a down move sat out, not a los
s) (made on: thin coverage (2 stories), tone undefined on fewer than 3 stories, market up, volatility calm)
  session 4: hold strength 1 -> abstained — no position held; the ticker moved -2.8% that week (a down move sat out, not a los
s) (made on: thin coverage (1 stories), tone undefined on fewer than 3 stories, market down, volatility calm)
  session 5: hold strength 1 -> abstained — no position held; the ticker moved -12.7% that week (a down move sat out, not a lo
ss) (made on: active coverage (3 stories), tone negative, market down, volatility calm)
  session 6: hold strength 1 -> abstained — no position held; the ticker moved -4.6% that week (a down move sat out, not a los
s) (made on: thin coverage (1 stories), tone undefined on fewer than 3 stories, market down, volatility calm)
DETERMINISTIC SUMMARY (pipeline-computed): 6 abstentions — 5 sat out down weeks (mean -4.5%); 1 sat out up weeks (mean +0.9%);
0 directional calls — 0 correct on direction. Quote these; do not recompute.
--- reflection REPLY ---
```json
{"lesson": "Across 6 abstentions, the pipeline consistently held no position during thin and active coverage periods while sit
ting out down weeks averaging -4.5% and up weeks averaging +0.9%."}
```
MEMORY WRITE (lesson, final): Across 6 abstentions, the pipeline consistently held no position during thin and active coverage
periods while sitting out down weeks averaging -4.5% and up weeks averaging +0.9%.

sessions complete: 6 decisions logged | memory: 6 realized ledger rows, 393 bans (2 added by agents), 6 self-written lessons
```

# 10. Evaluation

Scores the six decisions against baselines computed from data up to each week. The paper portfolio applies the strength-to-position map with a 15 basis point round-trip cost and models sells as short positions. The spot check reports the next-week and next-month move beside each call. The ablation reruns each decision with the memory ledger withheld. Six sessions support no statistical comparison. Section 7 reports the measured 85-week baseline.

The SPYdn rule column is a fixed baseline: it predicts AAPL's next-week direction to continue this week's SPY direction (down then down, up then up), and reads ok when the direction continued and miss when it reversed.

In [25]:
```
banner("L11 EVALUATION — every baseline is computed as-of each session week; direction accuracy on non-abstentions,
abstentions reported separately")
rows = []
for s in session_log:
    w = s["week"]
    df_ = ticker_frame()
    hist = df_[(df_.index < w) & df_["fwd"].notna()]
    cols = list(X0.columns) + MACRO
    maj = float(hist["y"].mean() >= 0.5)
    mom = bool(df_["ret"].loc[:w].iloc[-1] > 0)
    pipe = Pipeline([("scale", StandardScaler()),
                     ("clf", LogisticRegression(max_iter=3000, C=0.1, random_state=RANDOM_STATE))]
).fit(hist[cols].fillna(0), hist["y"].astype(int))
    logit = bool(pipe.predict(df_[cols].fillna(0).loc[[w]])[0] > 0.5)
    up = bool(s["fwd"] > 0)
    spy_dn = bool(float(wkpx["SPY_ret"].loc[s["week"]]) < 0)
    excess_up = bool((s["fwd"] - float(wkpx["SPY_fwd"].loc[s["week"]])) > 0)
    agent_call = s["direction"] in ("buy", "sell")
    rows.append({"week": str(s["week"]), "call": s["direction"], "str": s["strength"],
                 "advocate": s.get("advocate") or "-", "next up": up,
                 "excess up": excess_up, "SPYdn rule": "ok" if spy_dn != up else "miss",
```
```

```
                "agent": ("ok" if (s["direction"] == "buy") == up else "miss") if agent_call else "hold",
                "majority": "ok" if (maj > 0.5) == up else "miss",
                "momentum": "ok" if mom == up else "miss",
                "admit-all logit": "ok" if logit == up else "miss",
                "realized": round(s["fwd"] * 100, 2)})
dec = pd.DataFrame(rows)
if dec.empty:
    print("no decisions logged — the sessions need GEMINI_API_KEY")
else:
    display(dec)
    scored = dec[dec["agent"] != "hold"]
    acc = {c: (scored[c] == "ok").mean() for c in ["agent", "majority", "momentum", "admit-all logit"]} if len(scored) else
{}
    if acc:
        print(f"direction accuracy on the {len(scored)} non-abstaining calls: " + " | ".join(f"{k} {v:.2f}" for k, v in
acc.items()))
    else:
        print("direction accuracy: no non-abstaining calls this run")
    n_hold = int((dec["agent"] == "hold").sum())
    print(f"abstentions (hold = uncertain): {n_hold} of {len(dec)} sessions")
```

```
================================================================================
L11 EVALUATION — every baseline is computed as-of each session week; direction accuracy on non-abstentions, abstentions report
ed separately
================================================================================
```

	week	call	str	advocate	next up	excess up	SPYdn rule	agent	majority	momentum	admit-all logit	realized
0	2019-03-30/2019-04-05	hold	1	sell	True	True	ok	hold	ok	ok	miss	0.95
1	2019-07-20/2019-07-26	hold	1	sell	False	True	miss	hold	miss	miss	miss	-1.79
2	2019-09-07/2019-09-13	hold	1	sell	False	True	miss	hold	miss	miss	miss	-0.47
3	2020-01-18/2020-01-24	hold	1	buy	False	False	ok	hold	miss	ok	ok	-2.76
4	2020-02-15/2020-02-21	hold	1	buy	False	False	ok	hold	miss	ok	miss	-12.68
5	2020-09-05/2020-09-11	hold	1	buy	False	False	ok	hold	miss	ok	miss	-4.61

```
direction accuracy: no non-abstaining calls this run
abstentions (hold = uncertain): 6 of 6 sessions
```

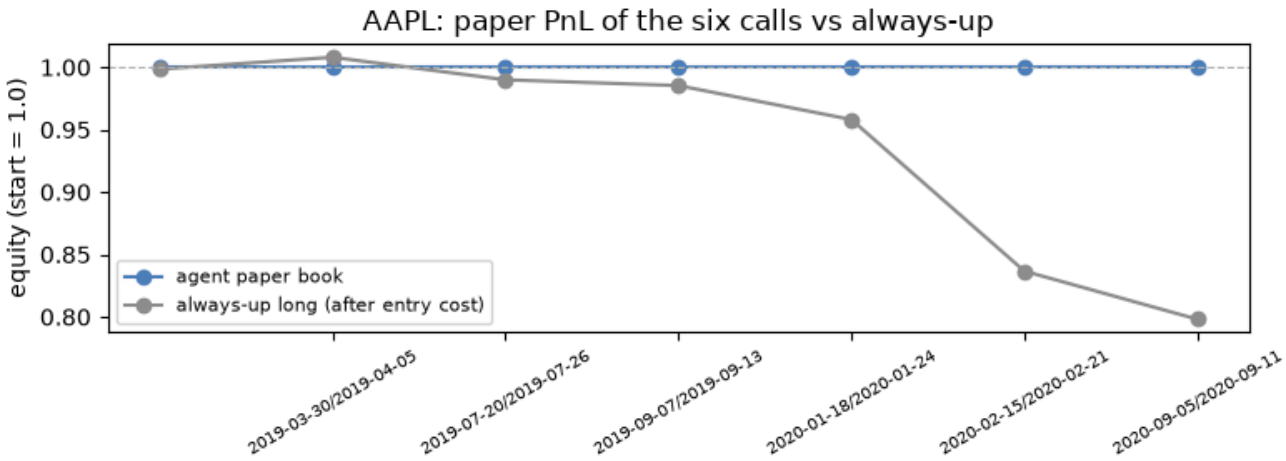
In [26]:

```
banner("L11.2 PAPER PnL — pre-registered: strength 1/2/3 = 25/50/100% of a unit position, sells short, holds flat, 15 bps
round-trip on turnover; always-up-long pays one entry cost")
SIZE_MAP = {1: 0.25, 2: 0.50, 3: 1.00}
COST = 0.0015
rows_, prev_pos, equity = [], 0.0, 1.0
eq_s, alw_s = [1.0], [1.0 - COST]
for s in session_log:
    size = SIZE_MAP.get(s["strength"], 0.25)
    posn = size if s["direction"] == "buy" else (-size if s["direction"] == "sell" else 0.0)
    turn = abs(posn - prev_pos)
    net = posn * s["fwd"] - turn * COST
    equity *= 1 + net
    eq_s.append(equity)
    alw_s.append(alw_s[-1] * (1 + s["fwd"]))
    rows_.append({"week": str(s["week"]), "call": s["direction"], "str": s["strength"],
                  "position": f"{posn:+.0%}", "gross": round(posn * s["fwd"], 4),
                  "cost": round(-turn * COST, 4), "net": round(net, 4), "equity": round(equity, 3)})
    prev_pos = posn
display(pd.DataFrame(rows_))
eq = np.array(eq_s)
peak = np.maximum.accumulate(eq)
mdd = float(((eq - peak) / peak).min())
print(f"paper PnL: agent book {eq[-1] - 1:+.1%} | always-up long {alw_s[-1] - 1:+.1%} | cash 0.0% | agent max drawdown
{mdd:.1%}")
fig, ax = plt.subplots(figsize=(8, 3))
ax.plot(range(len(eq_s)), eq_s, marker="o", color="#4f81bd", label="agent paper book")
ax.plot(range(len(alw_s)), alw_s, marker="o", color="#8c8c8c", label="always-up long (after entry cost)")
ax.axhline(1.0, color="#b3b3b3", lw=0.8, ls="--")
ax.set_xticks(range(1, len(session_log) + 1), [str(s["week"]) for s in session_log], fontsize=7, rotation=30)
ax.set_ylabel("equity (start = 1.0)")
ax.legend(fontsize=8)
ax.set_title(f"{TICKER}: paper PnL of the six calls vs always-up")
plt.tight_layout()
plt.show()
```

```
================================================================================
L11.2 PAPER PnL — pre-registered: strength 1/2/3 = 25/50/100% of a unit position, sells short, holds flat, 15 bps round-trip o
n turnover; always-up-long pays one entry cost
================================================================================
```

	week	call	str	position	gross	cost	net	equity
0	2019-03-30/2019-04-05	hold	1	+0%	0.0	-0.0	0.0	1.0
1	2019-07-20/2019-07-26	hold	1	+0%	-0.0	-0.0	-0.0	1.0
2	2019-09-07/2019-09-13	hold	1	+0%	-0.0	-0.0	-0.0	1.0
3	2020-01-18/2020-01-24	hold	1	+0%	-0.0	-0.0	-0.0	1.0
4	2020-02-15/2020-02-21	hold	1	+0%	-0.0	-0.0	-0.0	1.0
5	2020-09-05/2020-09-11	hold	1	+0%	-0.0	-0.0	-0.0	1.0

```
paper PnL: agent book +0.0% | always-up long -20.2% | cash 0.0% | agent max drawdown 0.0%
```



In [27]:
```
banner("L11.3 SPOT CHECK — a forward look for the written report, not an evaluation: did the price move as the call said
over the next week and the next month")


def call_consistency(direction, ret):
    if direction == "hold":
        return "abstained"
    want_up = direction == "buy"
    return "consistent" if (ret > 0) == want_up else "inconsistent"


rows_ = []
for s in session_log:
    w = s["week"]
    fwd4 = float((wkpx[TICKER].shift(-4) / wkpx[TICKER] - 1).loc[w])
    rows_.append({"week": str(w), "call": s["direction"], "str": s["strength"],
                  "next week": f"{s['fwd']:+.1%}", "next-week verdict": call_consistency(s["direction"], s["fwd"]),
                  "next month (4w)": f"{fwd4:+.1%}", "next-month verdict": call_consistency(s["direction"], fwd4)})
display(pd.DataFrame(rows_))
```

```
================================================================================
L11.3 SPOT CHECK — a forward look for the written report, not an evaluation: did the price move as the call said over the next
week and the next month
================================================================================
```

	week	call	str	next week	next-week verdict	next month (4w)	next-month verdict
0	2019-03-30/2019-04-05	hold	1	+0.9%	abstained	+7.5%	abstained
1	2019-07-20/2019-07-26	hold	1	-1.8%	abstained	-2.1%	abstained
2	2019-09-07/2019-09-13	hold	1	-0.5%	abstained	+8.0%	abstained
3	2020-01-18/2020-01-24	hold	1	-2.8%	abstained	-1.4%	abstained
4	2020-02-15/2020-02-21	hold	1	-12.7%	abstained	-26.8%	abstained
5	2020-09-05/2020-09-11	hold	1	-4.6%	abstained	+4.4%	abstained

In [28]:
```
banner("L11.4 ABSTENTION AUDIT — deterministic: holds carry no position, so they are described by calibration — what did
the system sit out, and on what states were the abstentions made")
abstention_rows = []
if not session_log:
    print("no decisions logged — abstention audit needs the sessions above")
else:
    rets_abs = wkpx[TICKER].pct_change()
    for s in session_log:
        w = s["week"]
        fwd = s["fwd"]
        hist = rets_abs.loc[:w].abs().dropna()
        pct = float((hist <= abs(fwd)).mean())
```

```
        n_, p_, g_ = bin_counts(w)
        v_ = float(wkpx["SPY_vol"].reindex(X0.index).loc[w])
        abstention_rows.append({"week": str(w), "call": s["direction"], "str": s["strength"],
                                "realized": f"{fwd:+.1%}", "|move| percentile as-of": f"{pct:.0%}",
                                "top-decile move": "yes" if pct >= 0.9 else "no",
                                "coverage at call": "quiet" if n_ == 0 else f"{n_} stories",
                                "volatility": "elevated" if v_ > 0.03 else "calm"})
    display(pd.DataFrame(abstention_rows))
    n_top = sum(r["call"] == "hold" and r["top-decile move"] == "yes" for r in abstention_rows)
    n_hold = sum(r["call"] == "hold" for r in abstention_rows)
```

```
================================================================================
L11.4 ABSTENTION AUDIT — deterministic: holds carry no position, so they are described by calibration — what did the system si
t out, and on what states were the abstentions made
================================================================================
```

	week	call	str	realized	|move| percentile as-of	top-decile move	coverage at call	volatility
0	2019-03-30/2019-04-05	hold	1	+0.9%	21%	no	2 stories	calm
1	2019-07-20/2019-07-26	hold	1	-1.8%	41%	no	2 stories	calm
2	2019-09-07/2019-09-13	hold	1	-0.5%	10%	no	2 stories	calm
3	2020-01-18/2020-01-24	hold	1	-2.8%	58%	no	1 stories	calm
4	2020-02-15/2020-02-21	hold	1	-12.7%	100%	yes	3 stories	calm
5	2020-09-05/2020-09-11	hold	1	-4.6%	80%	no	1 stories	calm

In [29]:

```
banner("L11.5 MEMORY ABLATION — pre-registered mechanism check: each session's decision rerun with the ledger and lessons
withheld from its tables; both branches reported")
ablation_rows = []
if not GEMINI_API_KEY or not session_log:
    print("GEMINI_API_KEY not set or no decisions logged — ablation skipped")
else:
    for si_ab, s in enumerate(session_log):
        tbl = syn_tables(s["week"], s["vt"], include_memory=False)
        syn_ab = call_llm("synthesis", SYNTHESIS_BLOCK, tbl) or {}
        d = str(syn_ab.get("direction", "hold")).lower()
        st_ = syn_ab.get("strength", 1)
        st_ = int(st_) if str(st_).isdigit() and 1 <= int(st_) <= 3 else 1
        ablation_rows.append({"week": str(s["week"]), "with memory": f"{s['direction']} {s['strength']}",
                              "without memory": f"{d} {st_}",
                              "changed": "yes" if (d, st_) != (s["direction"], s["strength"]) else "no"})
    display(pd.DataFrame(ablation_rows))
    n_ch = sum(r["changed"] == "yes" for r in ablation_rows)
```

```
================================================================================
L11.5 MEMORY ABLATION — pre-registered mechanism check: each session's decision rerun with the ledger and lessons withheld fro
m its tables; both branches reported
================================================================================
--- synthesis REPLY ---
```json
{
  "final_features": [],
  "removals": [
    {
      "term": "tablet",
      "reason": "No historical relation (q=0.84) and zero coverage this week."
    },
    {
      "term": "watch",
      "reason": "No historical relation (q=0.84) and zero coverage this week."
    },
    {
      "term": "COMPANY coverage",
      "reason": "No historical relation (q=0.94) after multiple testing."
    },
    {
      "term": "COMPANY tone",
      "reason": "Not significant after multiple testing (q=0.50) and tone uninformative."
    }
  ],
  "advocate_response": "The devil's advocate points out that company coverage is elevated at the 97th percentile, but since al
l surviving features lack significance after multiple testing (q > 0.05), no directional conviction can be manufactured.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "tablet coverage relation: q=0.84 — not significant after multiple testing",
    "watch coverage relation: q=0.84 — not significant after multiple testing",
    "COMPANY coverage relation: q=0.94 — not significant after multiple testing",
    "COMPANY tone relation: q=0.50 — not significant after multiple testing"
  ],
  "strength_boundary": "Any single feature achieving statistical significance (q < 0.05) would remove the requirement to hol
d.",
  "rationale": "All available features fail significance tests after multiple testing, and with no relation surviving, the hon
est call is hold. Therefore, we abstain from taking a directional position."
}
```
--- synthesis REPLY ---
```json
{
  "final_features": [
    "tablet",
    "watch",
    "COMPANY coverage",
    "COMPANY tone"
  ],
  "removals": [],
  "advocate_response": "The devil's advocate points out that all surviving features carry relations that are not significant a
fter multiple testing (q >= 0.50), meaning no relationship is statistically robust. We concede this point entirely and opt to
hold, as the rules state that with no relation surviving multiple testing, the honest call is hold.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "tablet coverage relation: no clear historical relation (q=0.84)",
    "watch coverage relation: higher coverage historically preceded weaker weeks (q=0.84)",
    "COMPANY coverage relation: no clear historical relation (q=0.94)",
    "COMPANY tone relation: more positive tone historically preceded weaker weeks (q=0.50)"
  ],
  "strength_boundary": "Any single feature achieving statistical significance after multiple testing would move the convictio
n.",
  "rationale": "All features across the frozen verdict table show relations that are not significant after multiple testing. P
er pipeline rules, when no relation survives multiple testing, the honest call is hold."
}
```
--- synthesis REPLY ---
```json
{"final_features": ["tablet", "watch", "COMPANY coverage", "COMPANY tone"], "removals": [], "advocate_response": "The devil's
advocate points out that multiple testing shows none of the relationships are significant (q >= 0.50), leaving no reliable his
torical directional edge on the printed evidence.", "direction": "hold", "strength": 1, "evidence": ["tablet: no clear histori
cal relation (q=0.84)", "watch: no clear historical relation (q=0.84)", "COMPANY coverage: no clear historical relation (q=0.9
4)", "COMPANY tone: not significant after multiple testing (q=0.50)"], "strength_boundary": "Any single feature achieving stat
istical significance with a clear historical relation.", "rationale": "With all evaluated signal features failing significance
tests after multiple testing, the honest call is to hold. No surviving directional edge is present in the printed tables."}
```
--- synthesis REPLY ---
```json
{
  "final_features": [
    "tablet",
    "watch",
```
```

```
    "COMPANY coverage",
    "COMPANY tone"
  ],
  "removals": [],
  "advocate_response": "The devil's advocate notes that every single feature carries no relation significant after multiple te
sting (q >= 0.50), making an honest call of hold the correct approach.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "tablet coverage relation: q=0.84 — not significant after multiple testing",
    "watch coverage relation: q=0.84 — not significant after multiple testing",
    "COMPANY coverage: q=0.94 — not significant after multiple testing",
    "COMPANY tone: q=0.50 — not significant after multiple testing"
  ],
  "strength_boundary": "Any single feature achieving statistical significance after multiple testing.",
  "rationale": "With no relationship surviving multiple testing across any feature in the verdict table, the honest call under
system rules is hold. Therefore, we abstain from directional conviction."
}
```
--- synthesis REPLY ---
```json
{
  "final_features": [
    "tablet",
    "watch",
    "COMPANY coverage",
    "COMPANY tone"
  ],
  "removals": [],
  "advocate_response": "The devil's advocate points to negative company tone and elevated headline volume; however, all histor
ical relations across the table are not significant after multiple testing and carry no surviving directional weight.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "tablet: no clear historical relation, q=0.84",
    "watch: no clear historical relation, q=0.84",
    "COMPANY coverage: no clear historical relation, q=0.94",
    "COMPANY tone: not significant after multiple testing, q=0.50"
  ],
  "strength_boundary": "Any single feature achieving statistical significance after multiple testing.",
  "rationale": "With no relationship surviving multiple testing across any feature, the honest call is hold. The printed table
s show no statistically significant directional edge."
}
```
--- synthesis REPLY ---
```json
{
  "final_features": [
    "tablet",
    "watch",
    "COMPANY coverage",
    "COMPANY tone"
  ],
  "removals": [],
  "advocate_response": "The devil's advocate notes that COMPANY tone shows a negative historical relation to forward returns a
nd is currently negative (-1.0000), while down market weeks historically precede stronger weeks. However, all featured relatio
ns are explicitly marked as not significant after multiple testing (q >= 0.50), leaving no statistically valid directional edg
e to override the baseline.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "tablet coverage relation: no clear historical relation (q=0.84)",
    "watch coverage relation: no clear historical relation (q=0.84)",
    "COMPANY coverage relation: no clear historical relation (q=0.94)",
    "COMPANY tone relation: not significant after multiple testing (q=0.50)"
  ],
  "strength_boundary": "Any single feature achieving statistical significance (q < 0.10) with a clear historical relation woul
d move strength to 2 or force a directional call.",
  "rationale": "All available signals carry relations that are explicitly not significant after multiple testing (q >= 0.50).
Per instructions, with no relation surviving multiple testing, the honest call is hold."
}
```

	week	with memory	without memory	changed
0	2019-03-30/2019-04-05	hold 1	hold 1	no
1	2019-07-20/2019-07-26	hold 1	hold 1	no
2	2019-09-07/2019-09-13	hold 1	hold 1	no
3	2020-01-18/2020-01-24	hold 1	hold 1	no
4	2020-02-15/2020-02-21	hold 1	hold 1	no
5	2020-09-05/2020-09-11	hold 1	hold 1	no

In [30]:

```
banner("L11.6 INJECTED-FAULT AUDIT — the revise branch verified on a deliberately corrupted copy of a real reply; labeled
synthetic, never counted as a system decision")
fault_evidence = False
if not GEMINI_API_KEY or not session_log:
    print("GEMINI_API_KEY not set or no decisions logged — fault audit skipped")
else:
    s0 = session_log[0]
    tbl = syn_tables(s0["week"], s0["vt"])
    corrupted = dict(s0["syn"])
    corrupted["final_features"] = list(corrupted.get("final_features", [])) + ["streaming"]
    corrupted["evidence"] = list(corrupted.get("evidence", [])) + ["tablet coverage at the 96th percentile this week
(r=+0.911)"]
    print("INJECTED FAULTS into the session-1 reply: (1) final_features lists 'streaming', absent from the verdict table;
(2) evidence cites r=+0.911, a value printed nowhere")
    ev = call_llm("evaluator", EVALUATOR_BLOCK, tbl + "\n\nDECISION UNDER AUDIT:\n" + json.dumps(corrupted)) or {}
    print(f"evaluator verdict on the corrupted reply: {ev.get('verdict')}")
    viol = ev.get("violations", [])
    print(f"violations: {viol if isinstance(viol, list) else [viol]}")
    if ev.get("verdict") == "revise":
        syn_fixed = call_llm("synthesis", SYNTHESIS_BLOCK, tbl + "\n\nEVALUATOR FEEDBACK (one revision allowed, address the
violations):\n" + json.dumps(ev))
        print(f"revised reply final_features: {(syn_fixed or {}).get('final_features')}")
        print(f"revised reply evidence: {(syn_fixed or {}).get('evidence')}")
        fault_evidence = True
    else:
        print("the evaluator accepted the corrupted reply — a miss to report in the limitations, not to hide")
print("note: the six session decisions above are untouched by this cell")
```

```
================================================================================
L11.6 INJECTED-FAULT AUDIT — the revise branch verified on a deliberately corrupted copy of a real reply; labeled synthetic, n
ever counted as a system decision
================================================================================
INJECTED FAULTS into the session-1 reply: (1) final_features lists 'streaming', absent from the verdict table; (2) evidence ci
tes r=+0.911, a value printed nowhere
--- evaluator REPLY ---
```json
{
  "checks": {
    "listed_valid": false,
    "survivors_accounted": true,
    "citations_match": false,
    "no_outside_knowledge": false,
    "no_arithmetic": true
  },
  "violations": [
    "Feature 'streaming' is included in final_features but does not exist in the verdict table.",
    "Cited value 'tablet coverage at the 96th percentile this week (r=+0.911)' does not match the printed tables (tablet share
is quiet, zero coverage; r=+0.911 does not appear in the tablet row).",
    "Outside knowledge/untraceable claim: 'streaming' is brought in without table support."
  ],
  "audit_note": "The decision is revised because it includes an untraceable feature ('streaming') and cites ungrounded statist
ics not present in the verdict table.",
  "verdict": "revise"
}
```
evaluator verdict on the corrupted reply: revise
violations: ["Feature 'streaming' is included in final_features but does not exist in the verdict table.", "Cited value 'table
t coverage at the 96th percentile this week (r=+0.911)' does not match the printed tables (tablet share is quiet, zero coverag
e; r=+0.911 does not appear in the tablet row).", "Outside knowledge/untraceable claim: 'streaming' is brought in without tabl
e support."]
--- synthesis REPLY ---
```json
{
  "final_features": [
    "tablet",
    "watch",
    "COMPANY coverage",
    "COMPANY tone"
  ],
  "removals": [],
  "advocate_response": "The devil's advocate notes that COMPANY coverage is elevated at the 97th percentile and COMPANY tone i
s uninformative, but no signal relations survive multiple testing (all q >= 0.50). On printed evidence, the complete absence o
f any statistically significant relation after multiple testing mandates an abstention to avoid manufacturing conviction.",
  "direction": "hold",
  "strength": 1,
  "evidence": [
    "tablet coverage relation: no clear historical relation (r=-0.043, r_spy=+0.020, r_exc=-0.064); q=0.84 — not significant a
fter multiple testing",
    "watch coverage relation: higher coverage historically preceded weaker weeks (r=-0.053, r_spy=+0.021, r_exc=-0.077); q=0.8
4 — not significant after multiple testing",
    "COMPANY coverage: no clear historical relation (r=-0.011, r_spy=+0.079, r_exc=-0.062); q=0.94 — not significant after mul
tiple testing",
    "COMPANY tone: more positive tone historically preceded weaker weeks (r=-0.081, r_spy=-0.064, r_exc=-0.057); q=0.50 — not
significant after multiple testing"
  ],
  "strength_boundary": "No lower strength grade exists below 1; any directional call would require a surviving significant rel
ation which is absent here.",
  "rationale": "All available signal relations are not significant after multiple testing, with q-values ranging from 0.50 to
0.94. With no surviving relational edge, the honest and correct decision is to hold."
}
```
revised reply final_features: ['tablet', 'watch', 'COMPANY coverage', 'COMPANY tone']
revised reply evidence: ['tablet coverage relation: no clear historical relation (r=-0.043, r_spy=+0.020, r_exc=-0.064); q=0.8
4 — not significant after multiple testing', 'watch coverage relation: higher coverage historically preceded weaker weeks (r=-
0.053, r_spy=+0.021, r_exc=-0.077); q=0.84 — not significant after multiple testing', 'COMPANY coverage: no clear historical r
elation (r=-0.011, r_spy=+0.079, r_exc=-0.062); q=0.94 — not significant after multiple testing', 'COMPANY tone: more positive
tone historically preceded weaker weeks (r=-0.081, r_spy=-0.064, r_exc=-0.057); q=0.50 — not significant after multiple testin
g']
note: the six session decisions above are untouched by this cell
```

In [31]:

```python
banner("L11.1 VISUALIZATIONS — the session log rendered: decisions on the price path, the news bins the decisions read, and
where every candidate died")
if session_log:
    fig, ax = plt.subplots(figsize=(8.5, 3.8))
    close_ = wkpx[TICKER].reindex(wk_index[TE])
    ax.plot(close_.index.to_timestamp(), close_.values, color="#8fa8c9", lw=1.3, label=f"{TICKER} weekly close (test
window)")
    for s in session_log:
        ts = s["week"].to_timestamp(how="end")
        y = float(close_.loc[s["week"]])
        col_ = {"buy": "#4f81bd", "sell": "#c0504d"}.get(s["direction"], "#8c8c8c")
        ax.scatter([ts], [y], color=col_, s=70, zorder=3, marker="^" if s["direction"] != "hold" else "o",
edgecolors="white", linewidths=0.6)
```

```
        ax.annotate(f"{s['direction']} {s['strength']}\nrealized {s['fwd']:+.1%}", (ts, y), textcoords="offset points",
                    xytext=(0, 12), fontsize=7, ha="center", color=col_)
    ax.set_title(f"{TICKER}: the six demonstration sessions on the price path")
    ax.set_ylabel("weekly close")
    ax.legend(fontsize=8, loc="upper left")
    plt.tight_layout()
    plt.show()

    fig, ax = plt.subplots(figsize=(8.5, 3.4))
    w_last = session_log[-1]["week"]
    win = wk_index[wk_index.get_loc(w_last) - 39:wk_index.get_loc(w_last) + 1]
    shares = company_series["COMPANY|share"].loc[win]
    ax.bar(win.to_timestamp(), shares.values, width=5, color="#8fa8c9", label="company coverage share of the week's
headlines")
    ax2 = ax.twinx()
    tone_x, tone_y = [], []
    for ww in win:
        if int(hits.loc[ww]) > 0:
            tone_x.append(ww.to_timestamp())
            tone_y.append(float(company_series["COMPANY|senti"].loc[ww]))
    ax2.scatter(tone_x, tone_y, color="#c0504d", s=26, zorder=3, label="weekly tone (undefined on zero-coverage weeks)")
    ax2.set_ylim(-1.3, 1.3)
    ax2.set_ylabel("tone (pos - neg)/stories")
    for s in session_log:
        ax.axvline(s["week"].to_timestamp(how="end"), color="#555555", lw=0.7, ls="--", alpha=0.6)
    ax.set_title(f"{TICKER}: the company bin the decision layer reads — coverage and tone, last 40 weeks (dashed =
sessions)")
    ax.set_ylabel("share of week's headlines")
    h1, l1 = ax.get_legend_handles_labels()
    h2, l2 = ax2.get_legend_handles_labels()
    ax.legend(h1 + h2, l1 + l2, fontsize=8, loc="upper left")
    plt.tight_layout()
    plt.show()

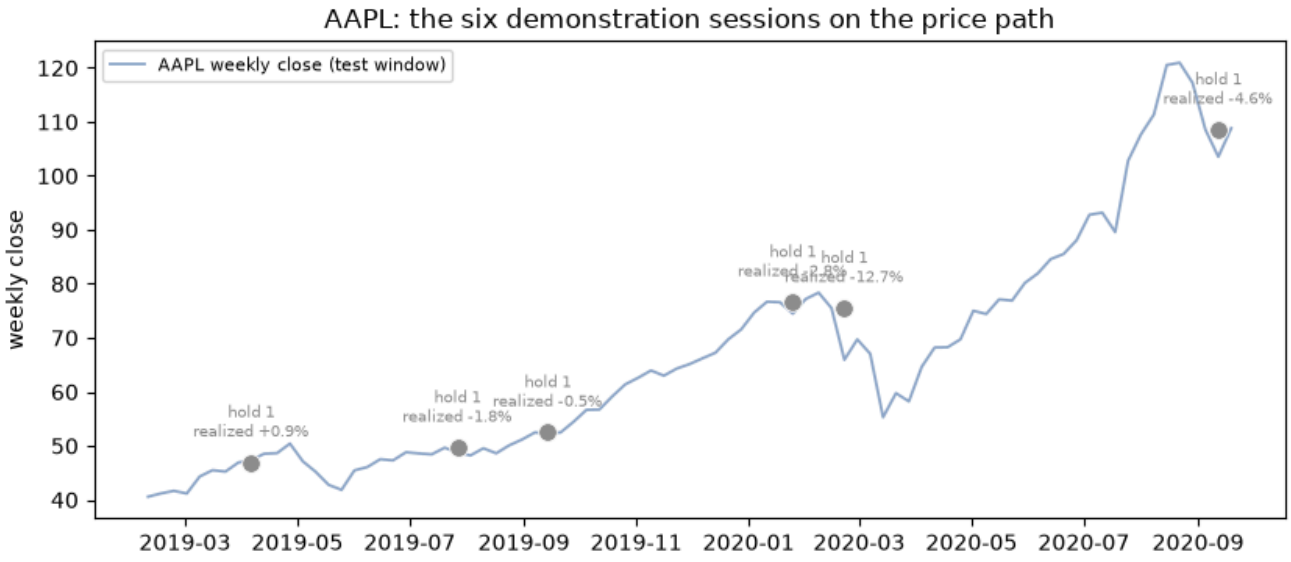
if fate_log:
    display(pd.DataFrame(fate_log))
    fig, ax = plt.subplots(figsize=(8.5, 0.42 * len(fate_log) + 1.6))
    stages = ["candidate", "admission gates", "planner", "news desk", "earnings desk", "frozen survivor"]

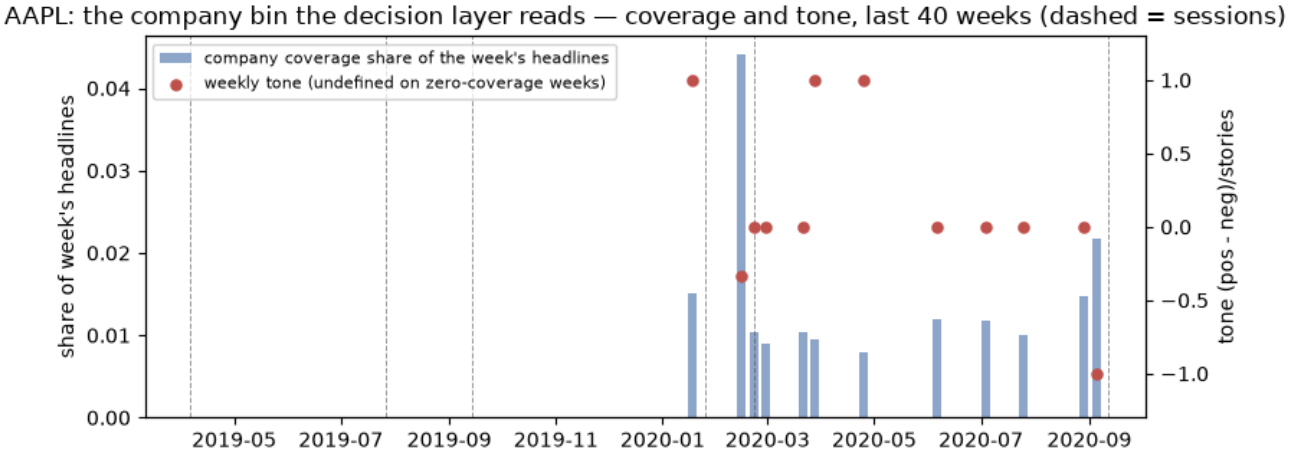
    def stage_of(fr):
        f = fr["fate"] or ""
        if f.startswith("died at admission"):
            return 1
        if f.startswith("dropped by planner"):
            return 2
        if f.startswith("struck"):
            return 4
        if f.startswith("kept"):
            return 4
        return 5

    def color_of(fr):
        f = fr["fate"] or ""
        if f.startswith("died at admission"):
            return "#b3b3b3"
        if f.startswith("dropped by planner"):
            return "#d28b36"
        if f.startswith("struck"):
            return "#c0504d"
        if f.startswith("kept"):
            return "#8fa8c9"
        return "#4f81bd"

    for y_, fr in enumerate(fate_log):
        ax.hlines(y_, 0, stage_of(fr), color=color_of(fr), lw=5)
        ax.scatter([stage_of(fr)], [y_], color=color_of(fr), s=42, zorder=3)
    ax.set_yticks(range(len(fate_log)), [fr["term"] for fr in fate_log], fontsize=8)
    ax.set_xticks(range(len(stages)), stages, fontsize=8)
    ax.set_xlim(-0.2, 5.6)
    ax.invert_yaxis()
    ax.set_title(f"candidate fate: how far each of the {len(fate_log)} candidates got (gray = admission, orange = planner,
red = two-axis strike, blue = survivor)")
    plt.tight_layout()
    plt.show()
```

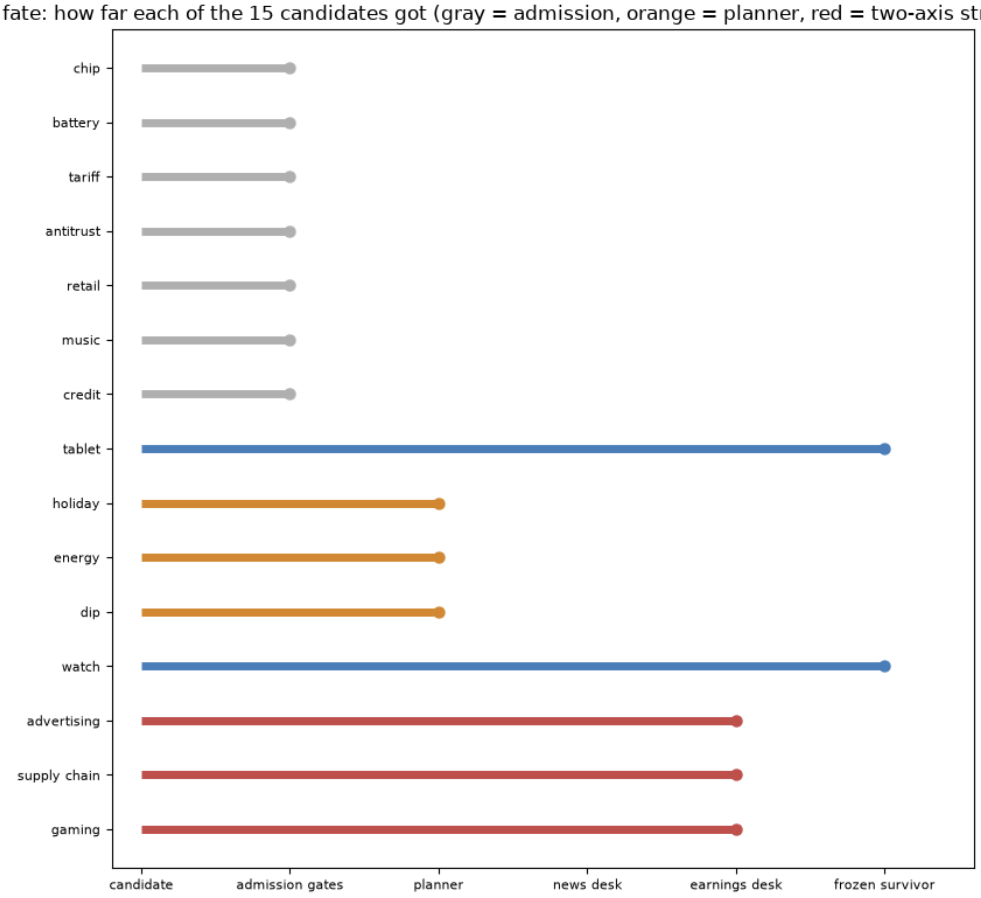
```
================================================================================
L11.1 VISUALIZATIONS — the session log rendered: decisions on the price path, the news bins the decisions read, and where ever
y candidate died
================================================================================
```


AAPL: the six demonstration sessions on the price path
AAPL weekly close (test window)
weekly close
hold 1
realized +0.9%
hold 1
realized -1.8%
hold 1
realized -0.5%
hold 1
realized -2.8%
hold 1
realized -12.7%
hold 1
realized -4.6%
2019-03
2019-05
2019-07
2019-09
2019-11
2020-01
2020-03
2020-05
2020-07
2020-09


AAPL: the company bin the decision layer reads — coverage and tone, last 40 weeks (dashed = sessions)
company coverage share of the week's headlines
weekly tone (undefined on zero-coverage weeks)
share of week's headlines
tone (pos - neg)/stories
2019-05
2019-07
2019-09
2019-11
2020-01
2020-03
2020-05
2020-07
2020-09

	term	provenance	planner	planner_note	news	channel	fate
0	chip	proposed	NaN		NaN	NaN	died at admission (died on vocabulary test (margin 0.24))
1	battery	proposed	NaN		NaN	NaN	died at admission (died on support (19 headlines, 15 weeks; floor 25))
2	tariff	proposed	NaN		NaN	NaN	died at admission (died on vocabulary test (margin -0.22))
3	antitrust	proposed	NaN		NaN	NaN	died at admission (died on support (2 headlines, 2 weeks; floor 25))
4	retail	proposed	NaN		NaN	NaN	died at admission (died on vocabulary test (margin -0.29))
5	music	proposed	NaN		NaN	NaN	died at admission (died on support (19 headlines, 18 weeks; floor 25))
6	credit	proposed	NaN		NaN	NaN	died at admission (died on vocabulary test (margin -0.19))
7	tablet	mined	full	Core hardware product line directly impacting revenue and shipment statistics.	NaN	revenue	frozen survivor
8	holiday	mined	drop	Calendar language representing seasonal shopping periods.	NaN	NaN	dropped by planner: Calendar language representing seasonal shopping periods.
9	energy	mined	drop	Generic sector or commodity-related term not specific to company operations.	NaN	NaN	dropped by planner: Generic sector or commodity-related term not specific to com
10	dip	mined	drop	Generic market-process language describing price movements.	NaN	NaN	dropped by planner: Generic market-process language describing price movements.
11	watch	mined	full	Core wearable device line driving ecosystem and hardware revenue.	NaN	revenue	frozen survivor
12	advertising	proposed	read	Suspect captures digital services revenue growth and privacy policy impacts.	contaminated	none	struck on two axes: news contaminated + earnings channel none
13	supply chain	proposed	read	Suspect captures manufacturing bottlenecks, logistics costs, and component availability.	contaminated	none	struck on two axes: news contaminated + earnings channel none
14	gaming	proposed	read	Suspect captures App Store commission trends and subscription gaming performance.	contaminated	none	struck on two axes: news contaminated + earnings channel none

candidate fate: how far each of the 15 candidates got (gray = admission, orange = planner, red = two-axis strike, blue = survivor)



11. The deliverable, the AAPL research card

Renders the final session's decision as the user-facing card. The card prints the market state, the recent company bins, each surviving keyword's relations, the call with its strength, and the memory lines consulted.

In [32]:

```
banner(f"FINAL OUTPUT — RESEARCH CARD: {TICKER}, final session; plain-language binned news sentiment, keyword relations to next-week return, market context, and the call")
import textwrap
STRENGTH_GLOSS = {1: "low conviction", 2: "moderate conviction", 3: "high conviction"}
if not session_log:
    print(f"no decisions logged for {TICKER} — the research card needs GEMINI_API_KEY and the sessions above")
else:
    last = session_log[-1]
    syn, vt, w, st = last["syn"], last["vt"], last["week"], last["strength"]

    def box(txt):
        for ln in textwrap.wrap(str(txt), 94) or [""]:
            print("| " + ln.ljust(94) + " |")

    edge = "+" + "-" * 96 + "+"
    callword = "UNCERTAIN (hold)" if last["direction"] == "hold" else last["direction"].upper()
    mkt = (f"SPY {'up' if float(wkpx['SPY_ret'].reindex(X0.index).loc[w]) > 0 else 'down'} this week; "
           f"volatility {'elevated' if float(wkpx['SPY_vol'].reindex(X0.index).loc[w]) > 0.03 else 'calm'}; "
           f"10y yield {'rising' if float(wkpx['dgs10_chg'].reindex(X0.index).loc[w]) > 0 else 'falling'}")
    ret_a = wkpx[TICKER].pct_change().loc[:w].iloc[-52:]
    ret_s = wkpx["SPY"].pct_change().loc[:w].iloc[-52:]
    rho = float(np.corrcoef(ret_a, ret_s)[0, 1])
    tg = "closely" if rho >= 0.5 else ("moderately" if rho >= 0.3 else "loosely")
    print(edge)
    box(f"{TICKER} — INVESTMENT RESEARCH CARD  (final session; week {week_off(w)} relative to corpus end)")
    print(edge)
    box(f"MARKET THIS WEEK: {mkt}")
    box(f"TICKER-MARKET LINK: {TICKER}'s weekly returns correlate {rho:+.2f} with SPY over the trailing 52 weeks — {TICKER} {tg} tracks the market")
    box("RECENT NEWS ON THIS COMPANY (last 4 weekly bins, classifier-labeled):")
    pw = wk_index.get_loc(w)
    for j in range(max(0, pw - 3), pw + 1):
        ww = wk_index[j]
        n_, p_, g_ = bin_counts(ww)
        if n_ >= 3:
```

```
            box(f"  week {week_off(ww):>4}: {n_} stories — {p_} positive, {g_} negative, {n_ - p_ - g_} neutral | tone
{float(X0['COMPANY|senti'].loc[ww]):+.2f}")
        elif n_:
            box(f"  week {week_off(ww):>4}: {n_} stor{'y' if n_ == 1 else 'ies'} — {p_} positive, {g_} negative, {n_ - p_ -
g_} neutral | tone uninformative (fewer than 3 stories)")
        else:
            box(f"  week {week_off(ww):>4}: no stories | tone undefined")
    rec = sorted([i for i in np.where(m_mask)[0] if 0 <= (w - week_of[i]).n <= 4], key=lambda i: week_of[i])[-5:]
    for i in rec:
        box(f"  [{labels_all[i]:>8}] {clean['headline'].iloc[i][:88]}")
    q_min_card = min((float(m.group(1)) for r in (vt["rows"] or [])
                      if (m := re.search(r"q=([0-9.]+)", r))), default=1.0)
    box(f"KEYWORD SIGNALS — each survivor's coverage and tone vs next-week return, judged on the window to session 1;
nothing survives multiple testing (min q={q_min_card:.2f}) — historical co-movement, not a proven predictor:")
    for term in syn.get("final_features", []):
        if term not in vt["survivors"]:
            continue
        ch = (vt.get("chan", {}).get(term, {}) or {})
        box(f"  {term} [channel: {ch.get('channel', '?')} {ch.get('direction', '?')}] — captures: {vt.get('caps',
{}).get(term, '')}")
        box(f"    tone relation: {vt.get('senti_rel', {}).get(term, 'not computed')}")
        box(f"    coverage relation: {vt.get('cov_rel', {}).get(term, 'not computed')}")
        rec_ = tw[term][w]
        if rec_[0] >= 3:
            box(f"    this week: ACTIVE — {rec_[0]} stories, tone {float(X0[f'{term}|senti'].loc[w]):+.2f} ({rec_[1]}
positive, {rec_[2]} negative)")
        elif rec_[0]:
            box(f"    this week: ACTIVE — {rec_[0]} stories, tone uninformative (fewer than 3 stories)")
        else:
            nz = X0[f"{term}|share"].loc[:w]
            last_nz = nz[nz > 0]
            la = f"week {week_off(last_nz.index[-1])}" if len(last_nz) else "never in the corpus"
            box(f"    this week: quiet (zero coverage; last active {la})")
    for r in vt["rows"]:
        if r.startswith("- COMPANY"):
            box("  " + r[len("- COMPANY "):].replace(" | ", " ; "))
    print(edge)
    box(f"CALL: {callword}   strength {st}/3 — {STRENGTH_GLOSS.get(st, '')}")
    _size = SIZE_MAP.get(st, 0.25)
    _pos = {"buy": _size, "sell": -_size}.get(last["direction"], 0.0)
    box(f"PAPER BOOK: this week sized {abs(_pos):.0%} {'short' if _pos < 0 else ('long' if _pos > 0 else 'flat')} (pre-
registered strength map; 15 bps round trip; the paper-PnL section keeps the account)")
    box(f"BOUNDARY: {syn.get('strength_boundary', '')}")
    box(f"RATIONALE: {syn.get('rationale', '')}")
    led = [r for r in memory["ledger"] if r["ticker"] == TICKER]
    if led:
        box(f"MEMORY: {len(led)} prior realized calls consulted; latest: session {led[-1]['session']} — {led[-1]
['gloss']}")
    else:
        box("MEMORY: (no prior calls)")
    if memory.get("lessons"):
        box(f"SELF-WRITTEN LESSON: {memory['lessons'][-1]}")
    print(edge)
```

```
================================================================================
FINAL OUTPUT — RESEARCH CARD: AAPL, final session; plain-language binned news sentiment, keyword relations to next-week retur
n, market context, and the call
================================================================================
+-----------------------------------------------------------------------------------------------------+
| AAPL — INVESTMENT RESEARCH CARD  (final session; week -2 relative to corpus end)                    |
+-----------------------------------------------------------------------------------------------------+
| MARKET THIS WEEK: SPY down this week; volatility calm; 10y yield falling                            |
| TICKER-MARKET LINK: AAPL's weekly returns correlate +0.79 with SPY over the trailing 52 weeks       |
| — AAPL closely tracks the market                                                                    |
| RECENT NEWS ON THIS COMPANY (last 4 weekly bins, classifier-labeled):                               |
|   week   -5: no stories | tone undefined                                                            |
|   week   -4: no stories | tone undefined                                                            |
|   week   -3: 1 story — 0 positive, 0 negative, 1 neutral | tone uninformative (fewer than 3         |
| stories)                                                                                            |
|   week   -2: 1 story — 0 positive, 1 negative, 0 neutral | tone uninformative (fewer than 3         |
| stories)                                                                                            |
|   [ neutral] Apple, Tesla Shares Keep Rising After Stock Splits                                     |
|   [negative] Tesla, Apple, Amazon Down Premarket                                                    |
| KEYWORD SIGNALS — each survivor's coverage and tone vs next-week return, judged on the window       |
| to session 1; nothing survives multiple testing (min q=0.50) — historical co-movement, not a        |
| proven predictor:                                                                                   |
|   tablet [channel: revenue helps] — captures: co-occurs with this company's mention headlines      |
|     tone relation: no clear historical relation (r=+0.032, r_spy=+0.112, r_exc=-0.031);             |
| specificity: mostly mirrors the market; significance: q=0.84 — not significant after multiple      |
| testing                                                                                             |
|     coverage relation: no clear historical relation (r=-0.043, r_spy=+0.020, r_exc=-0.064);         |
| specificity: ticker-specific, more than the market; significance: q=0.84 — not significant         |
| after multiple testing                                                                              |
|     this week: quiet (zero coverage; last active week -179)                                         |
|   watch [channel: revenue helps] — captures: co-occurs with this company's mention headlines       |
|     tone relation: no clear historical relation (r=+0.006, r_spy=-0.013, r_exc=+0.015);             |
| specificity: mostly mirrors the market; significance: q=0.94 — not significant after multiple      |
| testing                                                                                             |
|     coverage relation: higher coverage historically preceded weaker weeks (r=-0.053,                |
| r_spy=+0.021, r_exc=-0.077); specificity: ticker-specific, more than the market; significance:     |
| q=0.84 — not significant after multiple testing                                                     |
|     this week: quiet (zero coverage; last active week -221)                                         |
|   coverage: captures: the volume of headlines naming the company ; no clear historical             |
| relation (r=-0.011, r_spy=+0.079, r_exc=-0.062); specificity: unclear specificity;                  |
| significance: q=0.94 — not significant after multiple testing                                       |
|   tone: captures: the tone of headlines naming the company ; more positive tone historically       |
| preceded weaker weeks (r=-0.081, r_spy=-0.064, r_exc=-0.057); specificity: unclear                  |
| specificity; significance: q=0.50 — not significant after multiple testing                          |
+-----------------------------------------------------------------------------------------------------+
| CALL: UNCERTAIN (hold)   strength 1/3 — low conviction                                              |
| PAPER BOOK: this week sized 0% flat (pre-registered strength map; 15 bps round trip; the            |
| paper-PnL section keeps the account)                                                                |
| BOUNDARY: No relation surviving multiple testing permits a directional call; any single             |
| significant relation would move strength to 2.                                                      |
| RATIONALE: With no relation surviving multiple testing across the verdict table, the honest         |
| call is hold. The memory ledger and current sparse statistics do not override the lack of           |
| statistically significant directional relations.                                                    |
| MEMORY: 6 prior realized calls consulted; latest: session 6 — abstained — no position held;         |
| the ticker moved -4.6% that week (a down move sat out, not a loss)                                  |
| SELF-WRITTEN LESSON: Across 6 abstentions, the pipeline consistently held no position during        |
| thin and active coverage periods while sitting out down weeks averaging -4.5% and up weeks          |
| averaging +0.9%.                                                                                    |
+-----------------------------------------------------------------------------------------------------+
```

12. Cross-ticker memory trace

Runs a second ticker's selection stage on the memory written during this run. The persistent bans enter admission and the planner prompt. No decision sessions run for the second ticker.

In [33]:

```
banner("L12 CROSS-TICKER MEMORY TRACE — a second ticker's selection stage runs on the persistent memory the AAPL sessions
wrote (the ban ledger is supplied to admission and planning; for MSFT it excluded nothing); selection only, so API cost
stays bounded")
TRACE_OK = False
trace_ticker = "MSFT"
if not GEMINI_API_KEY:
    print("GEMINI_API_KEY not set — trace skipped; the single-ticker memory evidence above stands alone")
else:
    bans_before = len(banned_words())
    try:
        info2 = yf.Ticker(trace_ticker).info or {}
    except Exception:
        info2 = {}
    names2 = {info2.get("shortName"), info2.get("longName")} - {None, ""}
    words2 = set()
    for name in names2:
```

```
        for frag in re.findall(r"[A-Z][a-z]+|[A-Z]{2,}", str(name)):
            w_ = frag.lower()
            if len(w_) > 2 and w_ not in NAME_STOP:
                words2.add(w_)
    resolve2 = sorted(words2)
    m_mask2 = np.array([bool(s & set(resolve2)) for s in tok_sets])
    print(f"{trace_ticker} resolved words: {', '.join(resolve2) or '(none — metadata unavailable)'} | {int(m_mask2.sum())}
headlines name it")
    mined2_raw = mine_seeds(mask=m_mask2, self_words=resolve2)
    banned_now = banned_words()
    bites = [r for r in mined2_raw if r["seed"] in banned_now or (set(r["seed"].split()) & banned_now)]
    for b in bites:
        print(f"BAN BITE: '{b['seed']}' mined for {trace_ticker} ({b['mention_hits']} co-occurrences) excluded by the
persistent ledger an earlier session of this system wrote")
    mined2 = [r for r in mined2_raw if r not in bites]
    print(f"mined {len(mined2_raw)} seeds for {trace_ticker}; {len(bites)} excluded by the persistent ban ledger")
    props2 = propose_terms(trace_ticker)
    roster2 = [{"seed": r["seed"], "provenance": "mined", "gloss": "co-occurs with this company's mention headlines"} for r
in mined2]
    ar2 = admit_proposals(props2)
    display(pd.DataFrame(ar2))
    for r in ar2:
        if r["fate"] == "admitted":
            roster2.append({"seed": r["seed"], "provenance": "proposed", "gloss": r["hunch"]})
    print("--- PLANNER TABLES for the second ticker (verbatim; the persistent-memory line is what the AAPL sessions wrote)
---")
    print(planner_tables(roster2, trace_ticker))
    plan2 = plan_research(roster2, trace_ticker)
    if plan2:
        roster2 = dispatch_plan(plan2, roster2)
        if plan2.get("escalate"):
            print("TOOL CALL: remine(level='escalated', bans=memory + planner strikes) — the second ticker's planner
requested it")
            repl2 = remine_escalated({r["seed"] for r in roster2} | banned_words(), mask=m_mask2, self_words=resolve2)
            print("replacements: " + (", ".join(r["seed"] for r in repl2) or "none surfaced"))
            if repl2:
                dispatch_plan(plan_research(repl2, trace_ticker), repl2)
        memory["planner_notes"].append(f"{trace_ticker}: {str(plan2.get('plan_summary', ''))[:200]}")
        MEMORY_FILE.write_text(json.dumps(memory, indent=1), encoding="utf-8")
        print(f"persistent bans: {bans_before} before the trace -> {len(banned_words())} after")
    esc = bool(plan2 and plan2.get("escalate"))
    TRACE_OK = True
```

```
================================================================================
L12 CROSS-TICKER MEMORY TRACE — a second ticker's selection stage runs on the persistent memory the AAPL sessions wrote (the b
an ledger is supplied to admission and planning; for MSFT it excluded nothing); selection only, so API cost stays bounded
================================================================================
```

MSFT resolved words: microsoft | 450 headlines name it
mined 0 seeds for MSFT; 0 excluded by the persistent ban ledger
--- PROPOSER TABLES (verbatim — the profile text is the proposer's entire input) ---
COMPANY PROFILE:
Microsoft Corporation Technology Software - Infrastructure Microsoft Corporation, a technology company, develops and supports a portfolio of technology solutions for individuals and businesses worldwide. Its products include operating systems, server applications, business solution applications, software development tools, desktop and server management tools, and video games; and devices, such as PCs, tablets, gaming and entertainment consoles, other intelligent devices, and related accessories. The company's Productivity and Business Processes segment offers Microsoft 365 Commercial, Enterprise Mobility + Security, Power BI, Exchange, SharePoint, Microsoft Teams, Microsoft 365 Security and Compliance, Microsoft 365 Copilot, and Windows Commercial on-premises and Office licensed on-premises. This segment also provides Microsoft 365 Consumer products and cloud services; LinkedIn, including talent solutions, marketing solutions, subscriptions, and sales solutions; Dynamics 365, a set of cloud-based applications; and on-premises ERP and CRM applications. Its Intelligent Cloud segment offers server products and cloud services; cloud and AI consumption-based services, GitHub cloud services, health and life sciences cloud services, as well as virtual desktop offerings, and other cloud services; SQL Server, Windows Server, Visual Studio, System Center, and related client access licenses; and enterprise support services, industry solutions, Microsoft partner network, and learning experience. The company's More Personal Computing segment provides Windows OEM licensing and devices comprising Surface and PC accessories; XBOX hardware, and XBOX first- and third-party content and services; XBOX Game Pass and other subscriptions; XBOX Cloud Gaming, advertising, and other cloud servic
--- proposer REPLY ---
```json
{
  "terms": [
    {
      "term": "chip",
      "hunch": "Changes in semiconductor supply and pricing directly impact the hardware manufacturing costs for Surface devices and Xbox consoles."
    },
    {
      "term": "datacenter",
      "hunch": "Expansion or operational bottlenecks in physical infrastructure directly affect the capacity and margins of Azure cloud services."
    },
    {
      "term": "gaming",
      "hunch": "Shifts in consumer spending on interactive entertainment directly drive revenue for Xbox hardware, content, and subscriptions."
    },
    {
      "term": "advertising",
      "hunch": "Macroeconomic shifts in corporate marketing budgets directly impact ad revenue generated across search and LinkedIn platforms."
    },
    {
      "term": "tariff",
      "hunch": "Import duties and trade restrictions on consumer electronics increase the cost of imported hardware like PCs and gaming consoles."
    },
    {
      "term": "cybersecurity",
      "hunch": "Rising threat levels and enterprise demand for digital protection drive sales of Microsoft 365 Security and enterprise compliance tools."
    },
    {
      "term": "software",
      "hunch": "Shifts in enterprise IT spending cycles alter subscription and licensing volumes for operating systems and productivity suites."
    },
    {
      "term": "antitrust",
      "hunch": "Regulatory scrutiny and legal rulings can alter business practices, acquisition strategies, or operational structures in cloud and software."
    },
    {
      "term": "telecom",
      "hunch": "Network infrastructure performance and connectivity standards influence the adoption and latency-dependent delivery of cloud and gaming services."
    }
  ]
}
```

	seed	hunch	fate
0	chip	Changes in semiconductor supply and pricing directly impact the hardware manufacturing...	died on vocabulary test (margin 0.24)
1	datacenter	Expansion or operational bottlenecks in physical infrastructure directly affect the ca...	died on support (17 headlines, 14 weeks; floor 25)
2	gaming	Shifts in consumer spending on interactive entertainment directly drive revenue for Xb...	admitted
3	advertising	Macroeconomic shifts in corporate marketing budgets directly impact ad revenue generat...	admitted
4	tariff	Import duties and trade restrictions on consumer electronics increase the cost of impo...	died on vocabulary test (margin -0.22)
5	cybersecurity	Rising threat levels and enterprise demand for digital protection drive sales of Micro...	died on vocabulary test (margin 0.22)
6	software	Shifts in enterprise IT spending cycles alter subscription and licensing volumes for o...	admitted
7	antitrust	Regulatory scrutiny and legal rulings can alter business practices, acquisition strate...	died on support (2 headlines, 2 weeks; floor 25)
8	telecom	Network infrastructure performance and connectivity standards influence the adoption a...	died on vocabulary test (margin 0.18)

```
--- PLANNER TABLES for the second ticker (verbatim; the persistent-memory line is what the AAPL sessions wrote) ---
COMPANY: MSFT
CANDIDATE ROSTER (3):
- gaming [proposed]: Shifts in consumer spending on interactive entertainment directly drive revenue for Xbox hardware, conten
t, and subscrip
- advertising [proposed]: Macroeconomic shifts in corporate marketing budgets directly impact ad revenue generated across sear
ch and LinkedIn plat
- software [proposed]: Shifts in enterprise IT spending cycles alter subscription and licensing volumes for operating systems
and productivity
WORDS ALREADY BANNED BY EARLIER SESSIONS OF THIS SYSTEM (persistent memory; never re-admit them): holiday, dip
--- planner REPLY ---
```
```json
{
  "actions": [
    {
      "term": "gaming",
      "action": "full",
      "reason": "Core segment tied to Xbox hardware and content revenue.",
      "generalize": false
    },
    {
      "term": "advertising",
      "action": "full",
      "reason": "Major revenue stream tied to search and LinkedIn marketing budgets.",
      "generalize": false
    },
    {
      "term": "software",
      "action": "full",
      "reason": "Foundational product category driving enterprise licensing and cloud revenue.",
      "generalize": false
    }
  ],
  "escalate": true,
  "plan_summary": "All three proposed terms represent major operational and revenue segments for Microsoft. Routing all to the
full statistics and earnings lanes to capture detailed financial signals, and escalating to surface additional specific keywor
d candidates since the initial roster is very thin."
}
```
```
PLANNER plan_summary: All three proposed terms represent major operational and revenue segments for Microsoft. Routing all to
the full statistics and earnings lanes to capture detailed financial signals, and escalating to surface additional specific ke
yword candidates since the initial roster is very thin.
dispatch: gaming->full, advertising->full, software->full
TOOL CALL: remine(level='escalated', bans=memory + planner strikes) — the second ticker's planner requested it
replacements: none surfaced
persistent bans: 393 before the trace -> 393 after
```

13. System validation

Checks the run against the component list and prints the evaluator scoreboard, the twin flags, the tool-call log, and the call counts.

In [34]:
```
banner("SYSTEM VALIDATION — the run checks itself; every required component must show evidence in the outputs above")
checks = {
```

```
    "every session decided (no failures)": len(session_log) + len(session_failures) == N_SESSIONS and not session_failures,
    "a frozen roster exists": frozen is not None,
    "memory ledger holds glossed realized rows for every session": len([r for r in memory["ledger"] if r["ticker"] ==
TICKER]) >= len(session_log) and all("gloss" in r for r in memory["ledger"]),
    "persistent ban memory has agent-written entries (learning across runs)": any(b["origin"] != "deterministic" for b in
memory["banned"]),
    "reflection wrote lessons from realized outcomes (learning across runs)": bool(memory.get("lessons")),
    "evaluator audited every decision": sum(1 for a, _ in CALL_LOG if a == "evaluator") >= len(session_log),
    "ledger rows were injected into later sessions (trace above)": len(memory["ledger"]) >= 1 and len(session_log) >= 2,
    "abstention audit computed (holds described by calibration, not PnL)": len(abstention_rows) == len(session_log),
    "memory ablation ran (ledger-withheld arm; skipped without a key)": bool(ablation_rows) or not GEMINI_API_KEY,
    "injected-fault audit demonstrated the revise branch (skipped without a key)": bool(fault_evidence) or not
GEMINI_API_KEY,
    "cross-ticker memory trace ran (second-ticker selection on persistent memory)": TRACE_OK or not GEMINI_API_KEY,
}
display(pd.DataFrame({"check": list(checks), "passed": list(checks.values())}))
for e in EVAL_LOG:
    print(f"   session {e['session']}: {e.get('verdict')} — {str(e.get('note', ''))[:110]}")
print(f"devil's advocate: {sum(1 for a, _ in CALL_LOG if a == 'advocate')} sessions argued; {len(ADVOCATE_MISSES)} strength
caps for unaddressed briefs" + (f" (sessions {ADVOCATE_MISSES})" if ADVOCATE_MISSES else ""))
print(f"sign twin: {len(SIGN_FLAGS)} direction/sign contradictions flagged" + (f" {SIGN_FLAGS}" if SIGN_FLAGS else ""))
print(f"advocate citation twin: {len(ADVOCATE_CITE_MISSES)} brief(s) with numbers absent from the tables" + (f" —
{ADVOCATE_CITE_MISSES}" if ADVOCATE_CITE_MISSES else ""))
print(f"news desk tool-loop: {len(NEWS_TOOL_CALLS)} agent-chosen evidence requests this run" + (f" — {NEWS_TOOL_CALLS}" if
NEWS_TOOL_CALLS else ""))
print(f"call log: {len(CALL_LOG)} agent calls this run ({sum(1 for _, s in CALL_LOG if s == 'live')} live, {sum(1 for _, s
in CALL_LOG if s == 'cache')} cached)")
print("all validation checks completed" if all(checks.values()) else "SOME CHECKS FAILED — see the session output above for
the cause")
```

```
================================================================================
SYSTEM VALIDATION — the run checks itself; every required component must show evidence in the outputs above
================================================================================
```

	check	passed
0	every session decided (no failures)	True
1	a frozen roster exists	True
2	memory ledger holds glossed realized rows for every session	True
3	persistent ban memory has agent-written entries (learning across runs)	True
4	reflection wrote lessons from realized outcomes (learning across runs)	True
5	evaluator audited every decision	True
6	ledger rows were injected into later sessions (trace above)	True
7	abstention audit computed (holds described by calibration, not PnL)	True
8	memory ablation ran (ledger-withheld arm; skipped without a key)	True
9	injected-fault audit demonstrated the revise branch (skipped without a key)	True
10	cross-ticker memory trace ran (second-ticker selection on persistent memory)	True

```
   session 1: accept — Acceptance rests on all features, removals, citations, and non-significance claims strictly matching th
e verdi
   session 2: accept — Every feature, value, and claim strictly matches the printed tables and no outside knowledge or unautho
rized a
   session 3: accept — All listed features, values, and absence of statistical significance are faithfully sourced from the pr
ovided
   session 4: accept — The decision correctly lists all valid features, accounts for all survivors, cites verifiable table and
pipeli
   session 5: accept — All listed features and citations accurately reflect the verdict table, with no outside knowledge or un
authori
   session 6: accept — The decision is fully grounded in the provided verdict table and pipeline statistics with all features
and cit
devil's advocate: 6 sessions argued; 0 strength caps for unaddressed briefs
sign twin: 0 direction/sign contradictions flagged
advocate citation twin: 2 brief(s) with numbers absent from the tables — [(1, ['-0.81']), (3, ['-0.81'])]
news desk tool-loop: 3 agent-chosen evidence requests this run — [('advertising', -500, 6), ('supply chain', -500, 6), ('gamin
g', -500, 6)]
call log: 39 agent calls this run (0 live, 39 cached)
all validation checks completed
```

14. Limitations

- The corpus is one market-wide newswire source (NIFTY, 2010-2020). Template and vocabulary filters remove most repeated wire-service text, and the filters print what they remove.

- Mention-based seed mining produced 5 seeds for AAPL, few by construction. Most proposer terms are rejected for insufficient corpus support or by the vocabulary test, which removes terms that postdate the corpus window. Lexicon fixed phrases carry a lowered support floor of 15 headlines to protect them from the counting threshold.
- The six sessions run on the test weeks with the most company-naming coverage. Coverage is an input, so the selection uses no outcome information, but the weeks are unusually active and the sample is small. No result here is statistically powered, and the 85-week baseline table in section 7 remains the reference. Because the selected weeks are eventful and volatile, the session record cannot be separated from market beta.
- Headline timestamps are day-level. A story published after a week's closing print enters that week's signal, while prices are measured from that close. The publication lag is acknowledged and not corrected.
- Weekly tone bins often hold one or two stories. Below three stories the pipeline marks tone as uninformative, because earlier diagnostic runs showed a single headline's tone label moving decisions.
- With 68.2 percent of test weeks up, sign accuracy rewards always predicting up. The evaluation therefore also reports excess return over SPY and the paper portfolio return.
- The paper portfolio charges a 15 basis point round trip, models sells as short positions, and maps strength to pre-registered position fractions. Strength grades conviction and does not size risk. The model omits slippage, borrow cost, and financing.
- A deterministic sign check flags decisions that contradict the sign of a tone relation they cite. The evaluator verifies citations, and this check verifies signs.
- The evaluator accepted every decision this run. The revision path remains available, and the injected-fault audit demonstrates that it triggers.
- Selection is frozen at session 1 as an API-cost compromise. Later sessions re-decide only direction and strength on frozen features.
- The devil's advocate argues the opposite case at every session. A synthesis reply that ignores its strongest item has its strength capped by contract, and a relation printed as no clear historical relation cannot support a directional call.
- The reflection agent's lessons are model-written text over ledger rows that the pipeline annotates relative to each call. For an abstention, the printed move is context, not PnL. A lesson drawn from six rows can overgeneralize, and each lesson is labeled as system output rather than verified analysis.
- The news desk's evidence loop is bounded at three request rounds. Its samples are seeded and stratified, and the boundary audit prints further containing headlines for manual review.
- yfinance profiles are fetched as of the run date and are not point-in-time. EDGAR filings are fetched with a declared contact identity and full paginated history. Filings predate the session weeks, so that input is point-in-time for the window. Insider forms are excluded as noise, and a refused request returns no filings.
- The sentiment classifier transfers at 0.62 accuracy on disputed labels with 79 percent neutral predictions, so weekly tone is coarse in thin weeks. Zero-coverage weeks report tone as undefined rather than neutral.
- Agent-visible inputs carry no calendar dates, but period-identifying content such as pandemic coverage or product names can still date the window. Date stripping reduces this channel without eliminating it.

- LLM replies are cached at temperature zero. The model is not bit-deterministic across service changes, so a rerun from an empty cache may differ in wording while the computed tables are unchanged.
- The memory ablation reruns six decisions with the ledger and lessons withheld. At n = 6 it can detect whether memory changes a decision but cannot estimate an effect size, and the result is printed in either case.
- The injected-fault audit corrupts one real reply deliberately with a fabricated citation and a feature absent from the verdict table. The audit shows that the evaluator's revision path triggers. The corrupted reply is labeled synthetic and is never counted as a system decision.
- The cross-ticker trace runs the second ticker's selection stage only, to bound API cost. Persistent bans enter that ticker's admission gates and planner prompt, and the escalated remine tool runs on the planner's request. No second-ticker decision sessions run, so cross-ticker learning is not measured on outcomes.
- The devil's advocate brief enters the synthesis prompt without an audit, because the evaluator checks only the synthesis reply. A deterministic citation check flags brief numbers absent from the tables. This run it caught two briefs quoting r=-0.081 as r=-0.81, and the decision layer did not carry those values forward.
- The EDGAR filing list enters every earnings-desk prompt as recency context. No earnings verdict cites it, so it never visibly drove a channel judgment, and it is not treated as evidence.